▼ All-or-nothing atomicity: consistency under failure

1 ✦ All-or-nothing atomicity: consistency under failure

▼ Consistency under failure

2 ✦ Motivation case: bank transfer

3 ✦ Question: what could happen if the machine fails?

4 ✦ A naive implementation of the bank

5 ✦ Analysis: what would happen if a crash happens during the execution?

6 ✦ Deconstructing fsync: execution behind the fsync

7 ✦ Ideal system property: all-or-nothing atomicity

▼ Shadow copy

8 ✦ Shadow copy: one way to realize atomic file updates

9 ✦ Analysis of crash during shadow copy: fsync and fcopy

10 ✦ Question: what about a crash during rename?

11 ✦ Deconstructing rename to see what could happen during a crash

12 ✦ Question: crash at which step causes a problem?

13 ✦ Example: crash after step 1 — two names, one refcount

14 ✦ Possible solution: let the application clean up

▼ Journaling: making the file system atomic

15 ✦ Journaling: record before update

16 ✦ Case #1: rename via journaling

17 ✦ Case #1, crashed: recovering via the journal

18 ✦ Case #2: appending to a file, via journaling

19 ✦ The cost of journaling: everything is written twice

20 ✦ Mitigation: selective journaling (e.g., only the metadata)

21 ✦ data=writeback: journal the metadata only

22 ✦ data=ordered: the data goes out first

23 ✦ data=journal: the data goes through the journal too

24 ✦ Question: what if we crash while writing the commit?

▼ Limits of shadow copy

25 ✦ Question: two clients modify the same file

26 ✦ A case study of two operations using shadow copy

27 ✦ Three limitations of shadow copy

▼ Logging

28 ✦ Logging: generalizing journaling to any data modifications

29 ✦ Transaction and commit point

30 ✦ Redo-only logging #1: buffer the updates in memory, log at commit

31 ✦ Redo-only logging #2: sync the log to disk first

32 ✦ Question: how do we know a log entry is complete?

33 ✦ Redo-only logging #3: then update the data on disk

34 ✦ Question: do we need an fsync on the bank file?

35 ✦ Crash recovery: replay the log from the start

36 ✦ Question: where is the commit point of this transaction?

37 ✦ Quick summary: redo-only logging

38 ✦ (Aside) Value log vs. operator log

39 ✦ Pros & cons of redo-only logging

40 ✦ Coping with insufficient memory: write uncommitted updates to disk

41 ✦ Undo-redo logging: the log can also roll back

42 ✦ Implementation: write an undo record before each update

43 ✦ Undo-redo logging: record redo entries

44 ✦ Recap: log entry vs. log record in redo-only vs. undo-redo logging

45 ✦ Inside a log record

46 ✦ Two logging rules

47 ✦ Recovery rules: walk the log backwards

▼ Checkpoint

48 ✦ The remaining problem: the log file keeps growing

49 ✦ What is a checkpoint

50 ✦ Checkpoint to truncate the log

51 ✦ Question: what if a TX is ongoing during the checkpoint? (redo-only)

52 ✦ Question: what if a TX is ongoing during the checkpoint? (undo-redo)

53 ✦ Naive solution: stop all transactions, then do the checkpoint

54 ✦ Solution: keep the in-progress TXs' log in a CKPT record

55 ✦ Recovery with a checkpoint

56 ✦ Undo-redo vs. redo-only: how to choose

57 ✦ (Aside) What about undo-only logging

58 ✦ Summary

59 ✦ 参考文献

All-or-nothing atomicity: consistency under failure

Xingda Wei

Motivation case: bank transfer

A bank application, with two requirements

- The data must be persistently stored: the money survives a power loss
- It has to support operations (read/update accounts), e.g., `transfer`
- The operations must report/leave the data correct (e.g., money is not lost)




```
transfer(bank, a, b, amt):
    bank[a] = bank[a] - amt
    bank[b] = bank[b] + amt
```

To simplify the problem, assume only one machine as the server executes the operations

- With a single machine, linearizability comes for free

Question: what could happen if the machine fails?

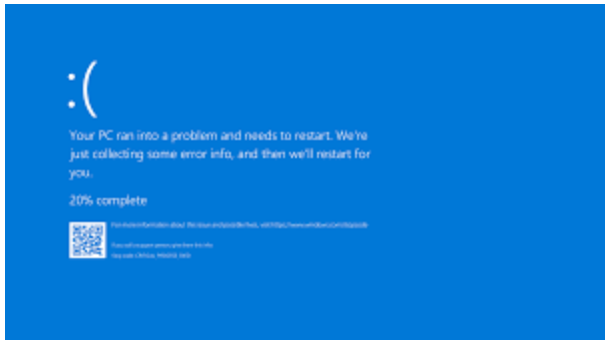
```
transfer(bank, a, b, amt):
    bank[a] = bank[a] - amt
    bank[b] = bank[b] + amt
```

The correctness semantic we need from the application is one (or many) invariants

- Example: Money never appears or disappears: `bank[a] + bank[b]` never changes if we only have `transfer`

Suppose that upon failure, all system components stop together (fail-stop)

- No case where half the components keep running and writing garbage



Can we preserve this semantic upon failures? This depends on the system implementation

A naive implementation of the bank

Suppose that all the bank's data is stored in a file

- So data is persistent by default

```
transfer(bank, a, b, amt):
    records = mmap(bank, ...)
    records[a] = records[a] - amt      # new value: in memory
    records[b] = records[b] + amt      # new value: in memory
    fsync(bank, ...)                   # new value: on disk
    notify_the_user()                  # the user is told "done"
```

Two system calls

- `mmap`: operate the file like memory; changes first land in the page cache
- `fsync`: flush the changes from the page cache to the disk, page by page

Analysis: what would happen if a crash happens during the execution?

```
transfer(bank, a, b, amt):
    records = mmap(bank, ...)
    records[a] = records[a] - amt      # new value: in memory
    records[b] = records[b] + amt      # new value: in memory
    fsync(bank, ...)                   # new value: on disk
    notify_the_user()                  # the user is told "done"
```

Crash before fsync: nothing happens

- Not a single byte on disk has been modified
- If the disk was consistent before the crash, it still is

Question: what if the crash happens in the middle of `fsync`?

What does "in the middle of fsync" even mean?

Deconstructing fsync: execution behind the fsync

The application modified two disk pages cached in the page cache; fsync flushes the cache



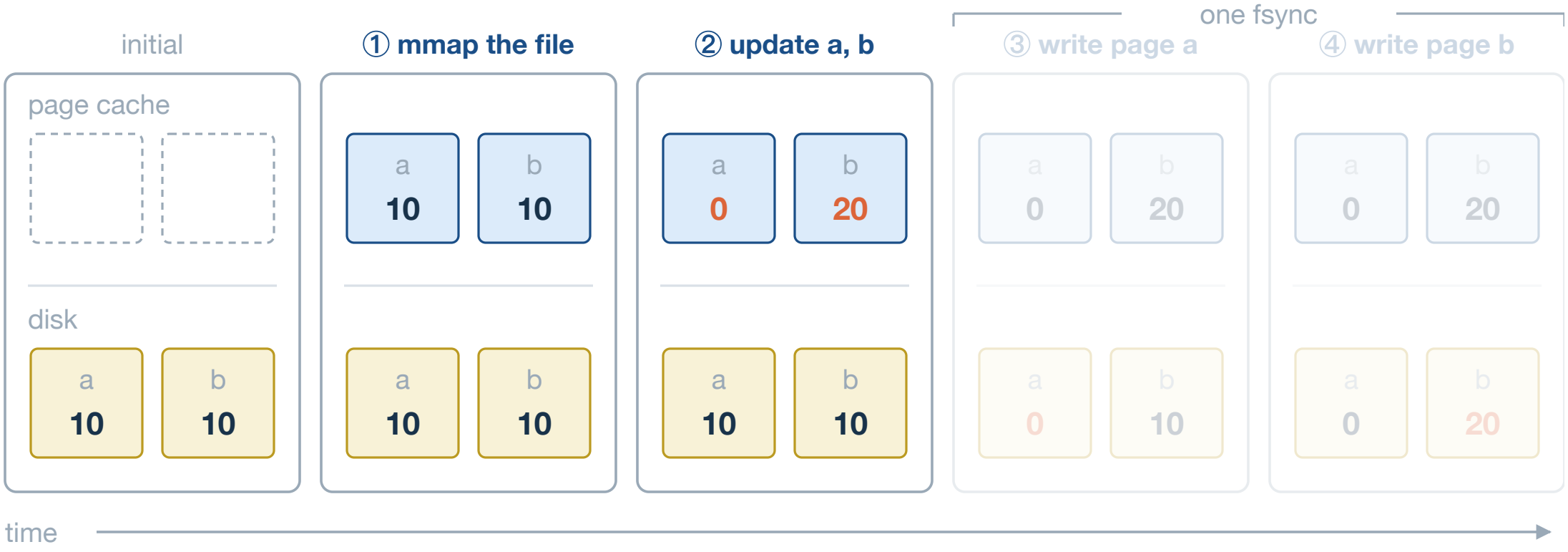
Deconstructing fsync: execution behind the fsync

The application modified two disk pages cached in the page cache; fsync flushes the cache



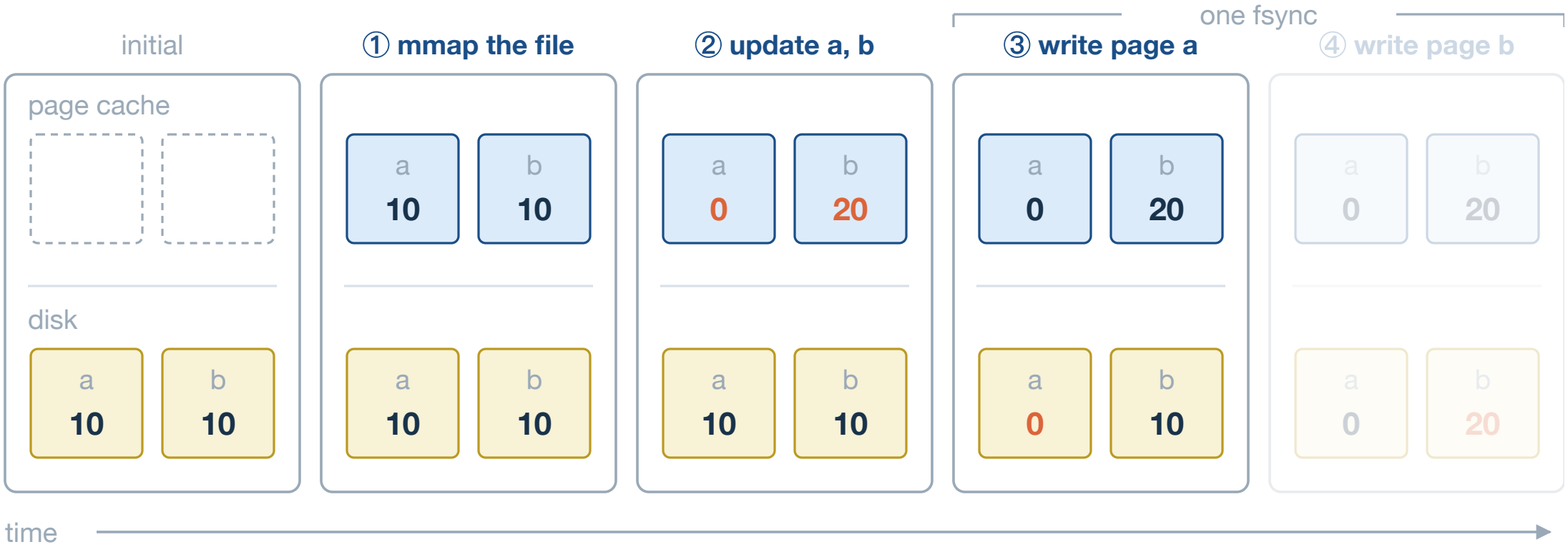
Deconstructing fsync: execution behind the fsync

The application modified two disk pages cached in the page cache; fsync flushes the cache



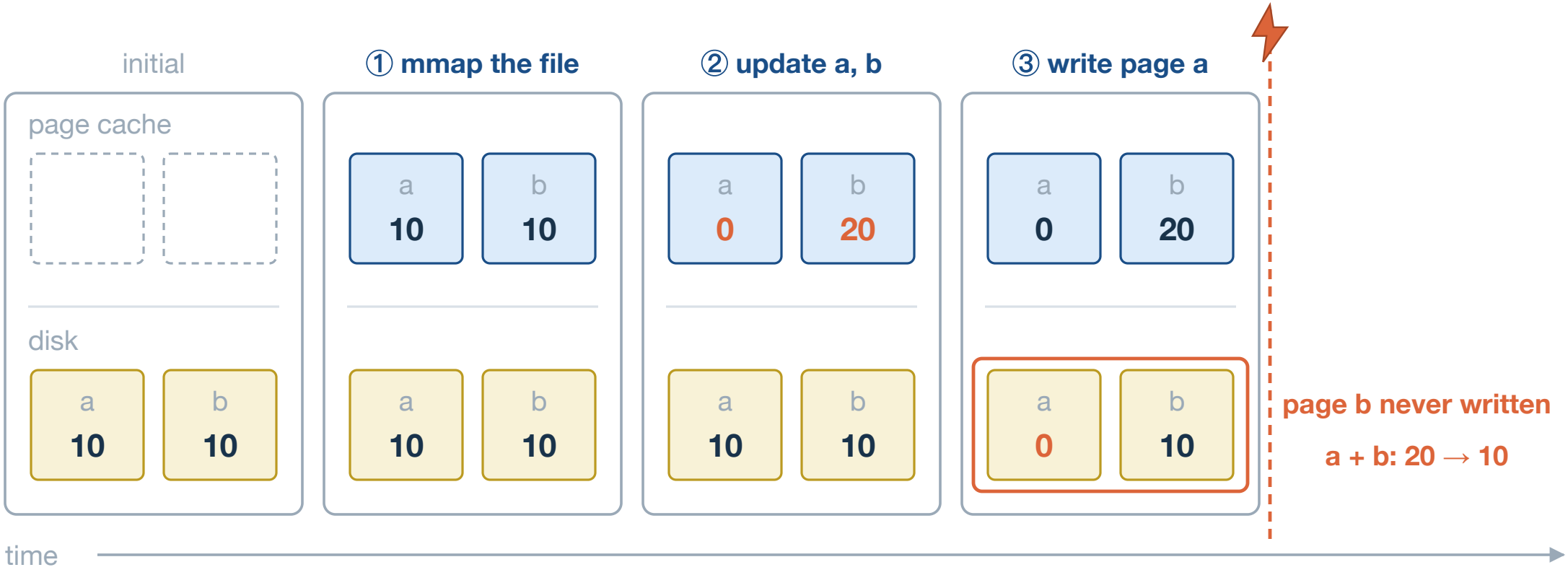
Deconstructing fsync: execution behind the fsync

The application modified two disk pages cached in the page cache; fsync flushes the cache



Deconstructing fsync: execution behind the fsync

The application modified two disk pages cached in the page cache; fsync flushes the cache



The disk can only update one page/sector at a time (or a fixed granularity)

Ideal system property: all-or-nothing atomicity

all-or-nothing atomicity
a set of operations happens completely or not at all

With it, reasoning about failures leaves only two cases

- the operations happened
- the operations did not happen
- no more “partially happened” — and there are too many kinds of “partial” to enumerate one by one

Shadow copy: one way to realize atomic file updates

High-level idea: copy-on-write

- Do not modify the old copy — there is always a consistent copy on disk (assuming the initial is consistent)
- All changes go to a separate copy (shadow copy) to isolate any possible inconsistent state caused by failures
- Replace the original file only after all the writes have succeeded

```
transfer(bank, a, b, amt):
    fcopy(bank, bank_temp)
    records = mmap(bank_temp, ...)
    records[a] = records[a] - amt
    records[b] = records[b] + amt
    fsync(bank_temp, ...)
    rename(bank_temp, bank)
```

Two new lines compared to before: `fcopy` at the start, `rename` at the end

Analysis of crash during shadow copy: fsync and fcopy

```
transfer(bank, a, b, amt):
    fcopy(bank, bank_temp)
    records = mmap(bank_temp, ...)
    records[a] = records[a] - amt
    records[b] = records[b] + amt
    fsync(bank_temp, ...)
    rename(bank_temp, bank)
```

No problem

- `bank` was never touched; it is still one complete old copy
- After reboot, remove `bank_temp` — as if the transfer never happened

Question: what about a crash during rename?

```
transfer(bank, a, b, amt):
    fcopy(bank, bank_temp)
    records = mmap(bank_temp, ...)
    records[a] = records[a] - amt
    records[b] = records[b] + amt
    fsync(bank_temp, ...)
    rename(bank_temp, bank)
```

For the bank transfer it is fine: `bank` is either the old file or the new one

Question: can other states become inconsistent?

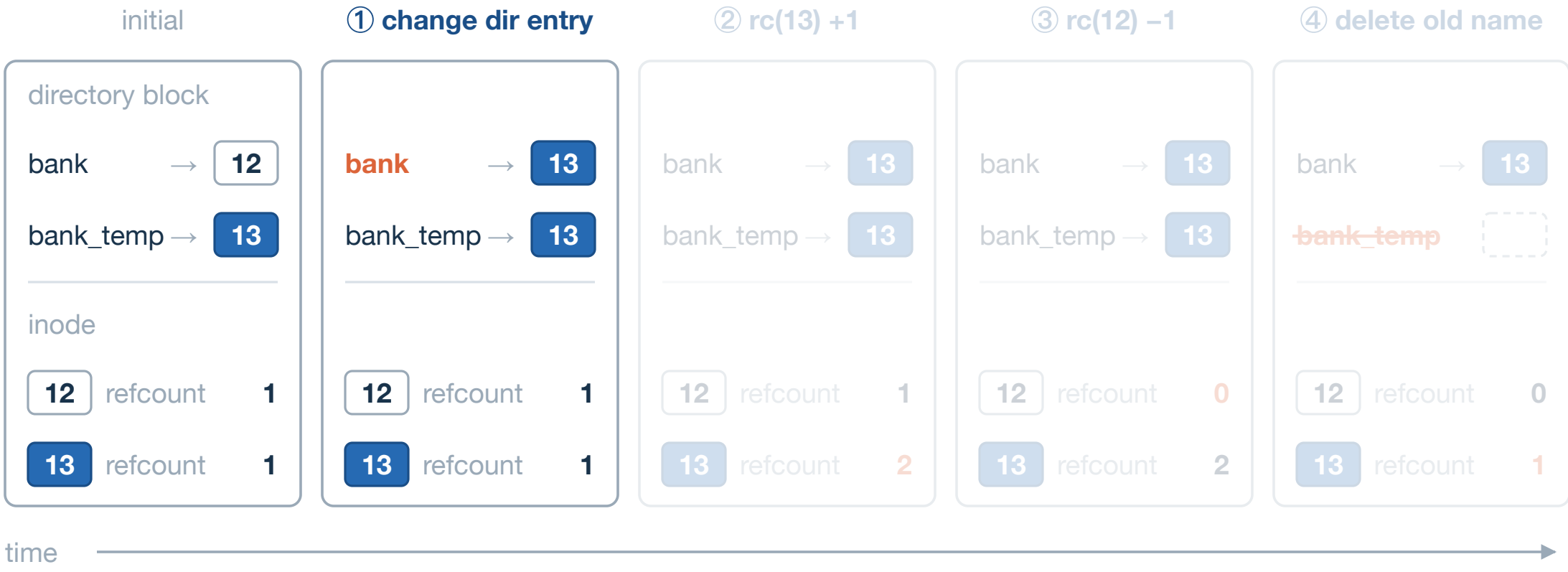
Deconstructing rename to see what could happen during a crash

`rename(bank_temp, bank)` modifies four places on disk



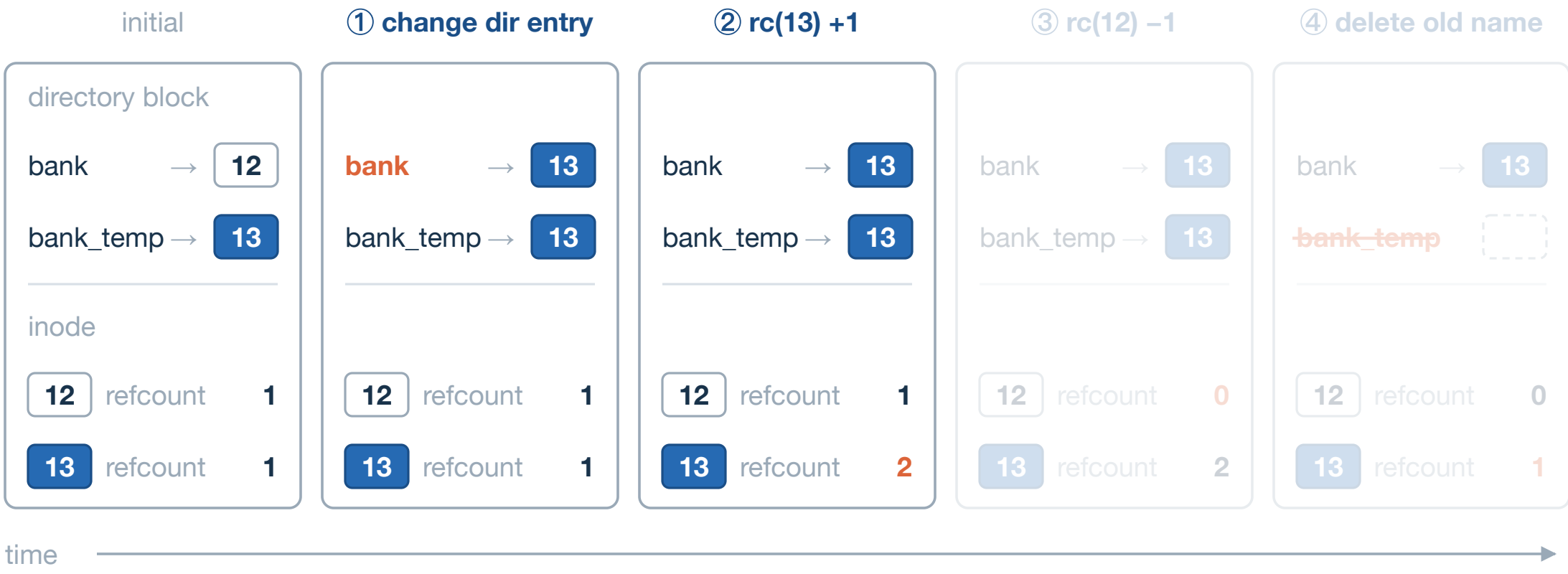
Deconstructing rename to see what could happen during a crash

`rename(bank_temp, bank)` modifies four places on disk



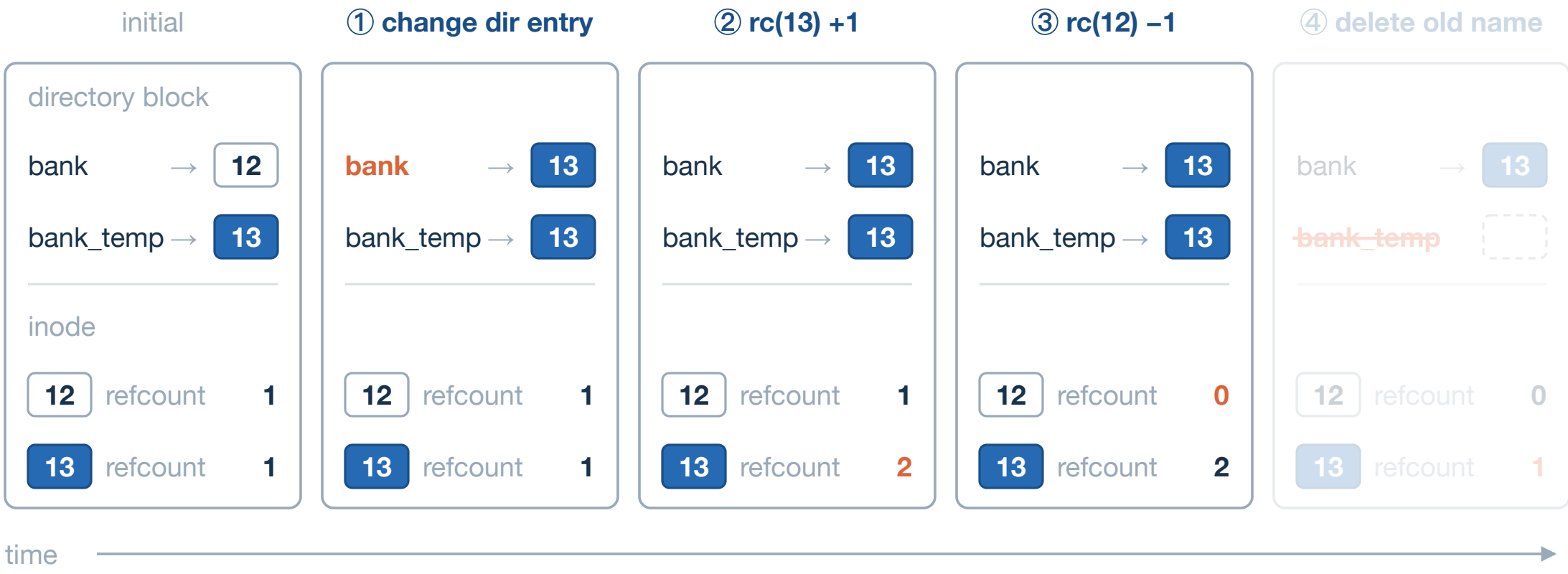
Deconstructing rename to see what could happen during a crash

`rename(bank_temp, bank)` modifies four places on disk



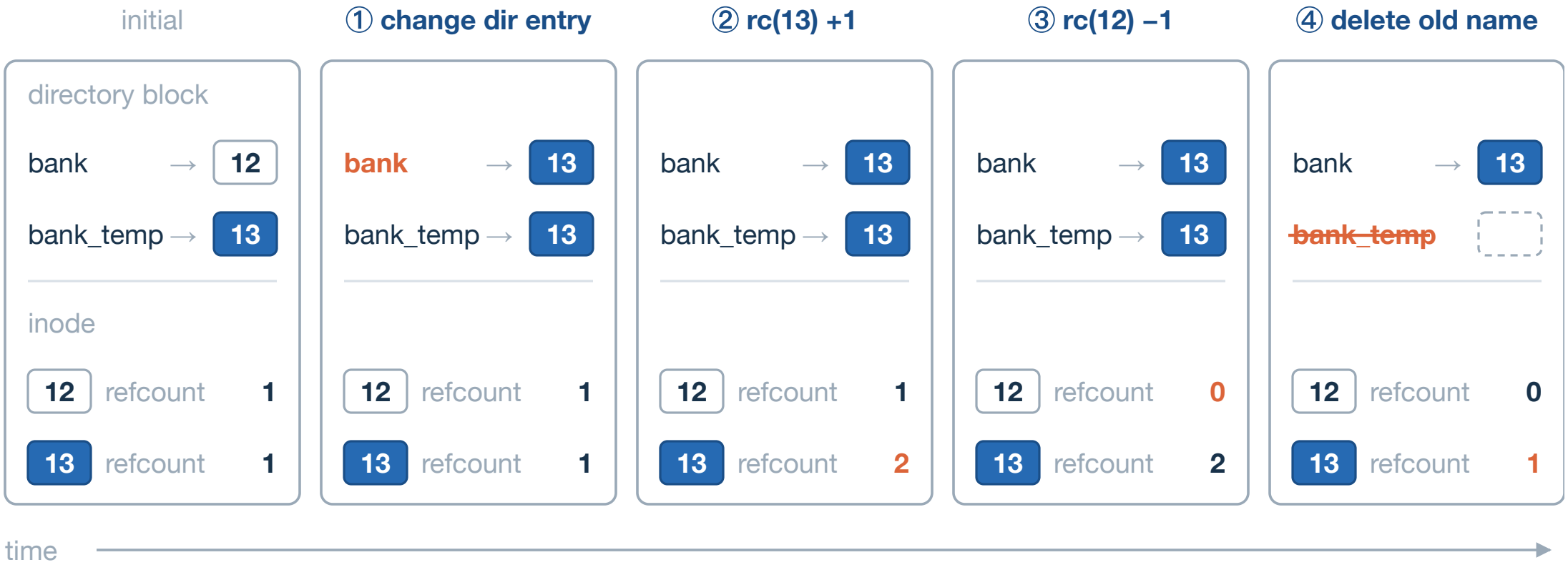
Deconstructing rename to see what could happen during a crash

`rename(bank_temp, bank)` modifies four places on disk



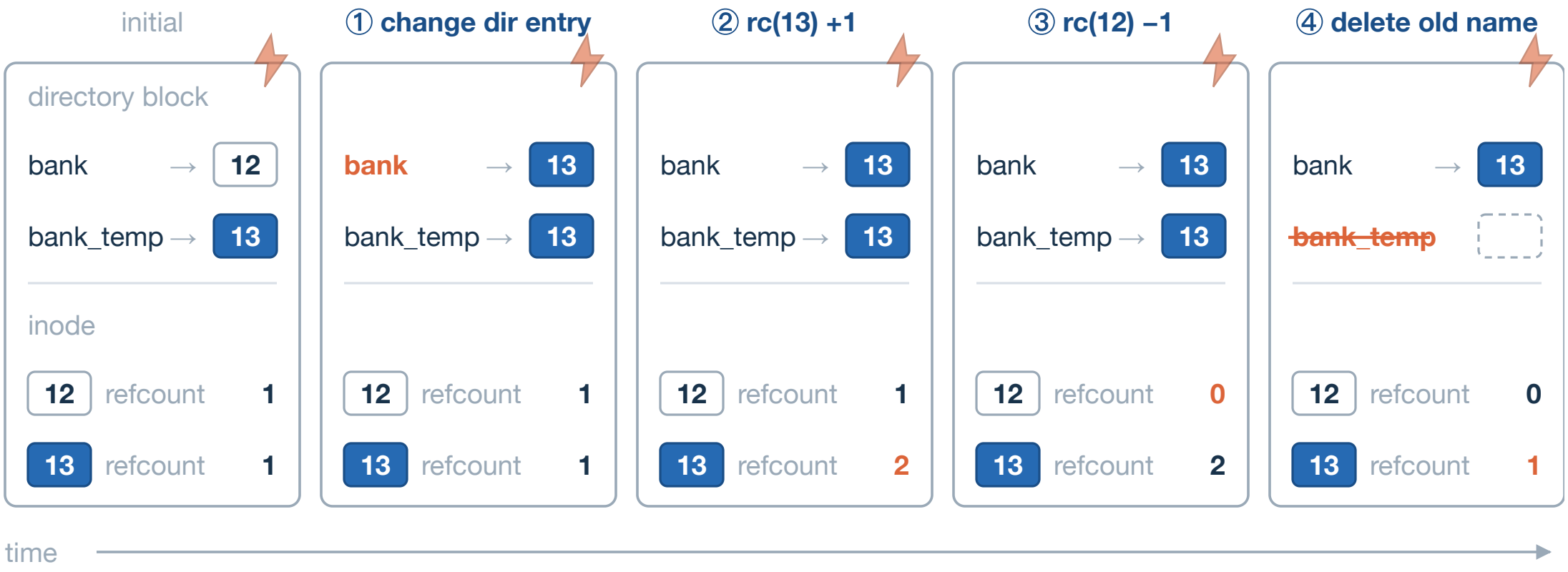
Deconstructing rename to see what could happen during a crash

`rename(bank_temp, bank)` modifies four places on disk

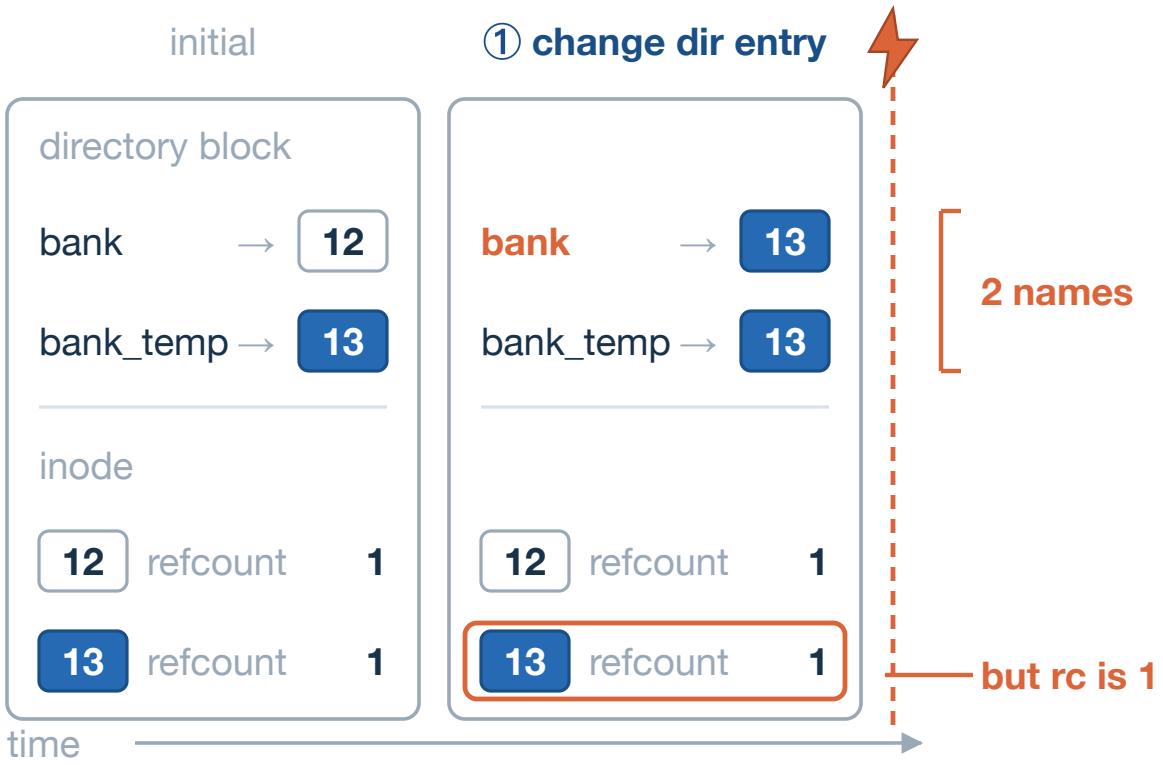


Question: crash at which step causes a problem?

Disk can crash at any of these intermediate step!



Example: crash after step 1 — two names, one refcount



Two names point to inode 13, but its refcount is still 1

- Delete either name, and the refcount drops to 0 — the file is reclaimed
- The other name still points to it, so accessing it is dangerous

Possible solution: let the application clean up

One idea: after reboot, run a sanity check on the filesystem state, and then ask the application to clean up

- The application knows it created `bank_temp`, so it deletes it after reboot

It is not recommended: the file system state's correctness must not depend on applications behaving

- If the application crashes and never comes back, those inodes leak forever
- A malicious user can keep leaking until the file system runs out of inodes

The correctness of file system recovery must be decoupled from application recovery

Shadow copy is only atomic if rename is

So it is ideal to provide All-or-nothing for all

filesystem APIs

Journaling: record before update

Approach: before modifying data on disk, first record “what I am about to change” into an area called the journal

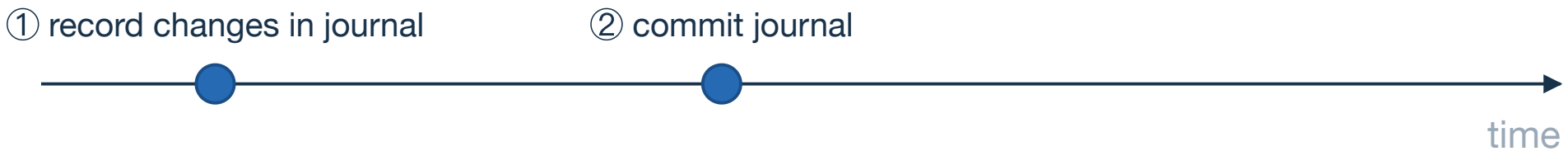
- The method is similar to shadow copy: before we are sure we can recover to a correct state, don't modify the correct copy



Journaling: record before update

Approach: before modifying data on disk, first record "what I am about to change" into an area called the journal

- The method is similar to shadow copy: before we are sure we can recover to a correct state, don't modify the correct copy



Journaling: record before update

Approach: before modifying data on disk, first record “what I am about to change” into an area called the journal

- The method is similar to shadow copy: before we are sure we can recover to a correct state, don't modify the correct copy



Journaling: record before update

Approach: before modifying data on disk, first record "what I am about to change" into an area called the journal

- The method is similar to shadow copy: before we are sure we can recover to a correct state, don't modify the correct copy



Journaling: record before update

Approach: before modifying data on disk, first record "what I am about to change" into an area called the journal

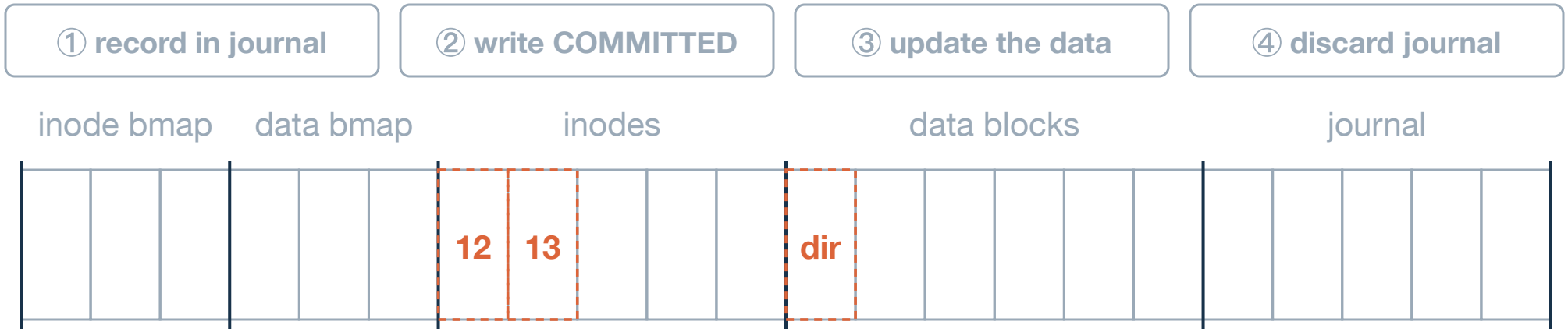
- The method is similar to shadow copy: before we are sure we can recover to a correct state, don't modify the correct copy



One portion of the disk is left as the journaling area

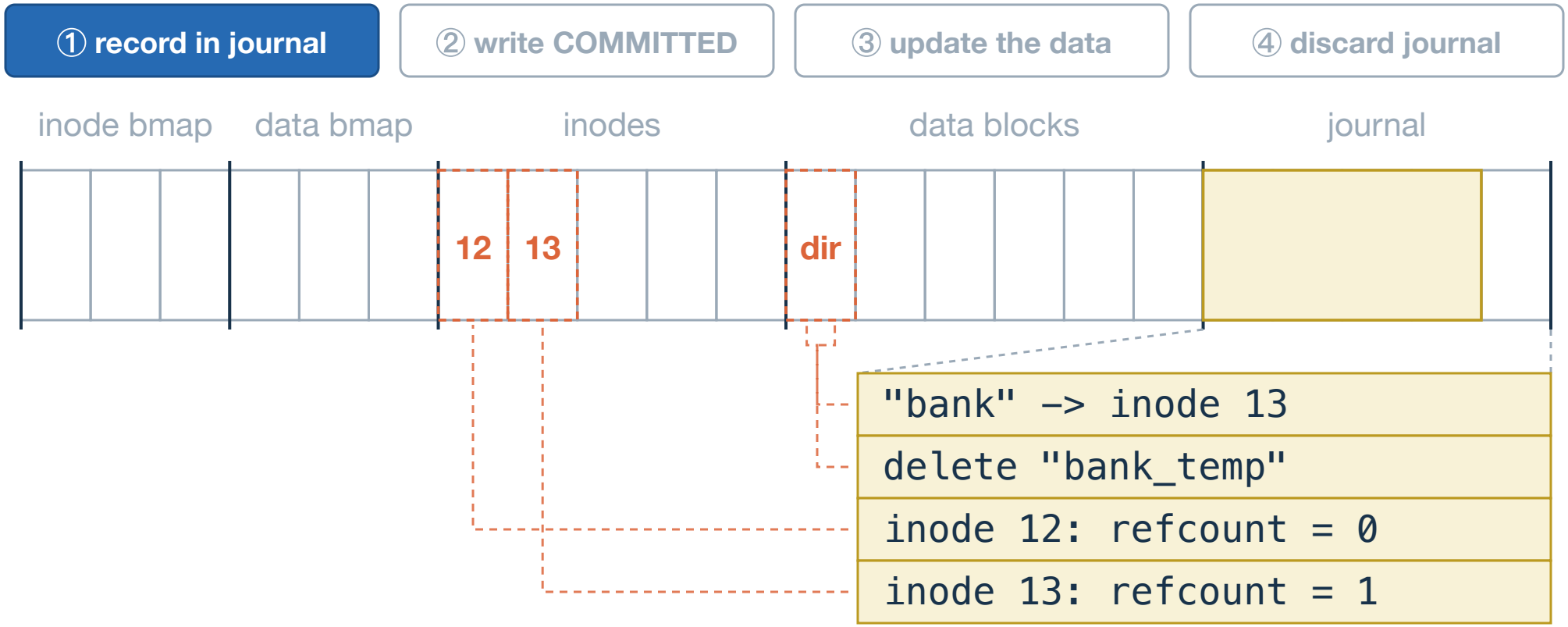
Case #1: rename via journaling

One record per change: which block it hits, and what that block becomes



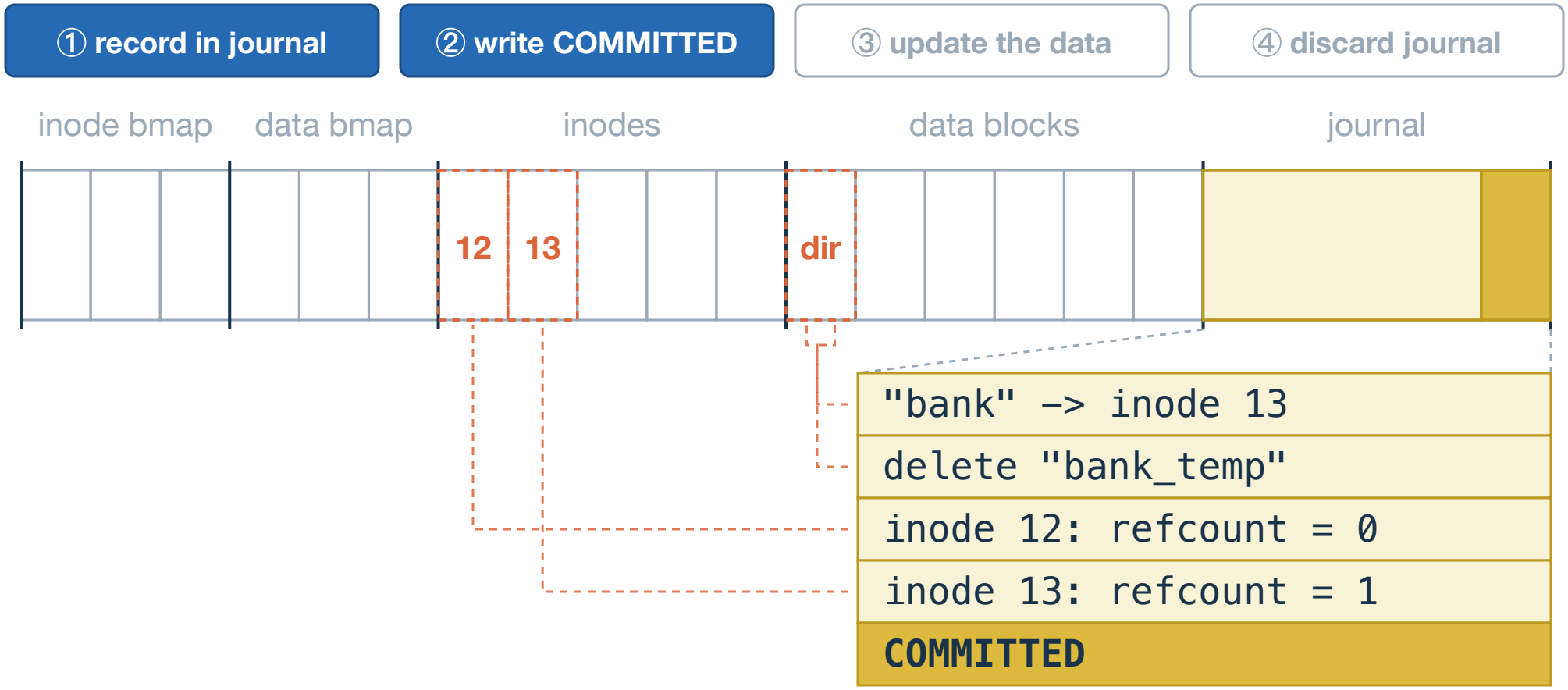
Case #1: rename via journaling

One record per change: which block it hits, and what that block becomes



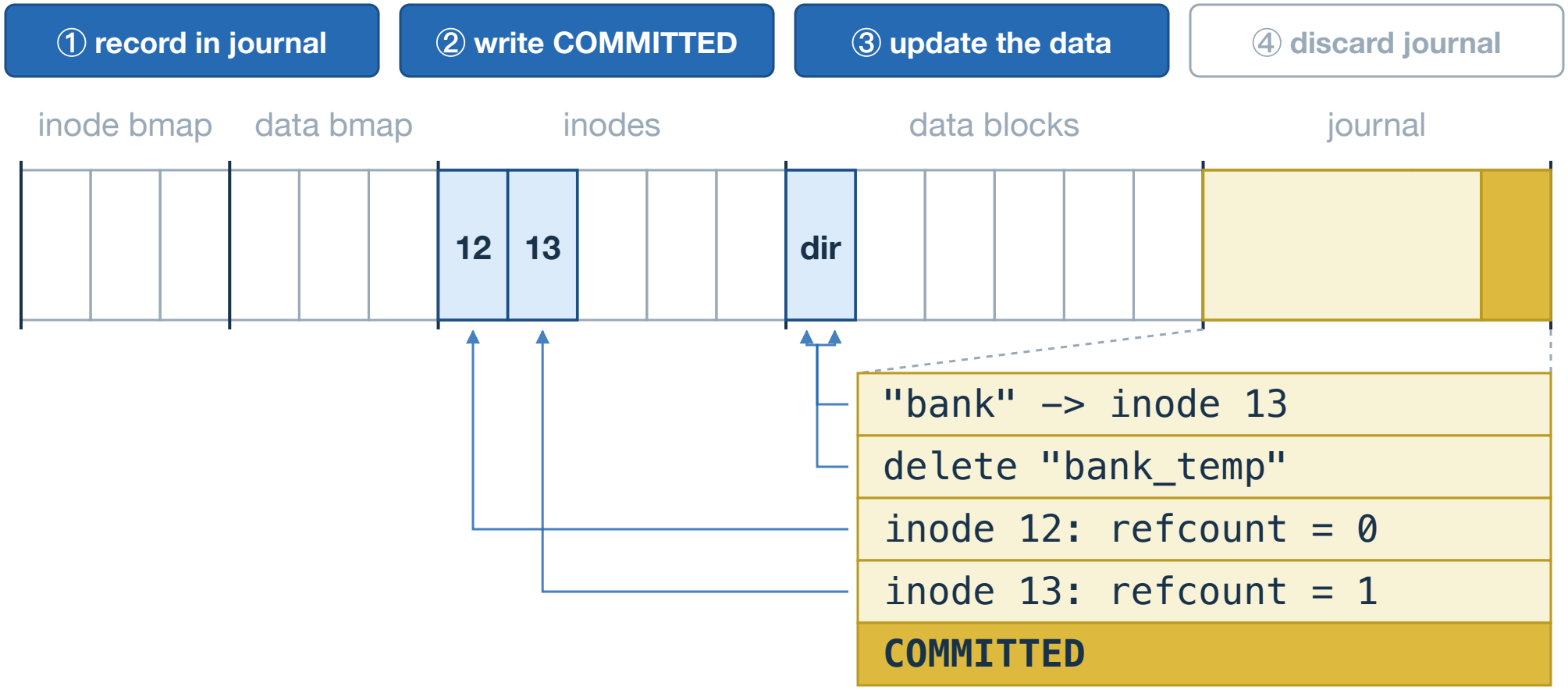
Case #1: rename via journaling

One record per change: which block it hits, and what that block becomes



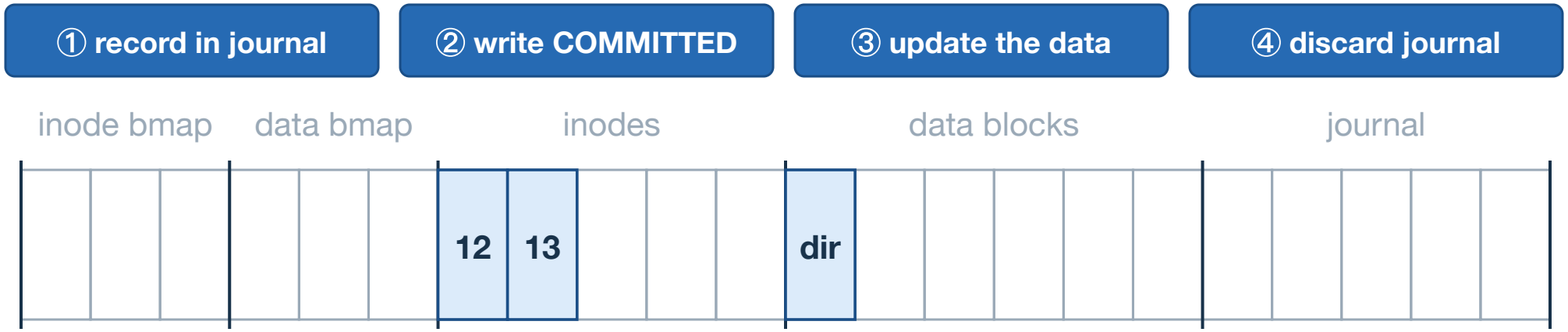
Case #1: rename via journaling

One record per change: which block it hits, and what that block becomes



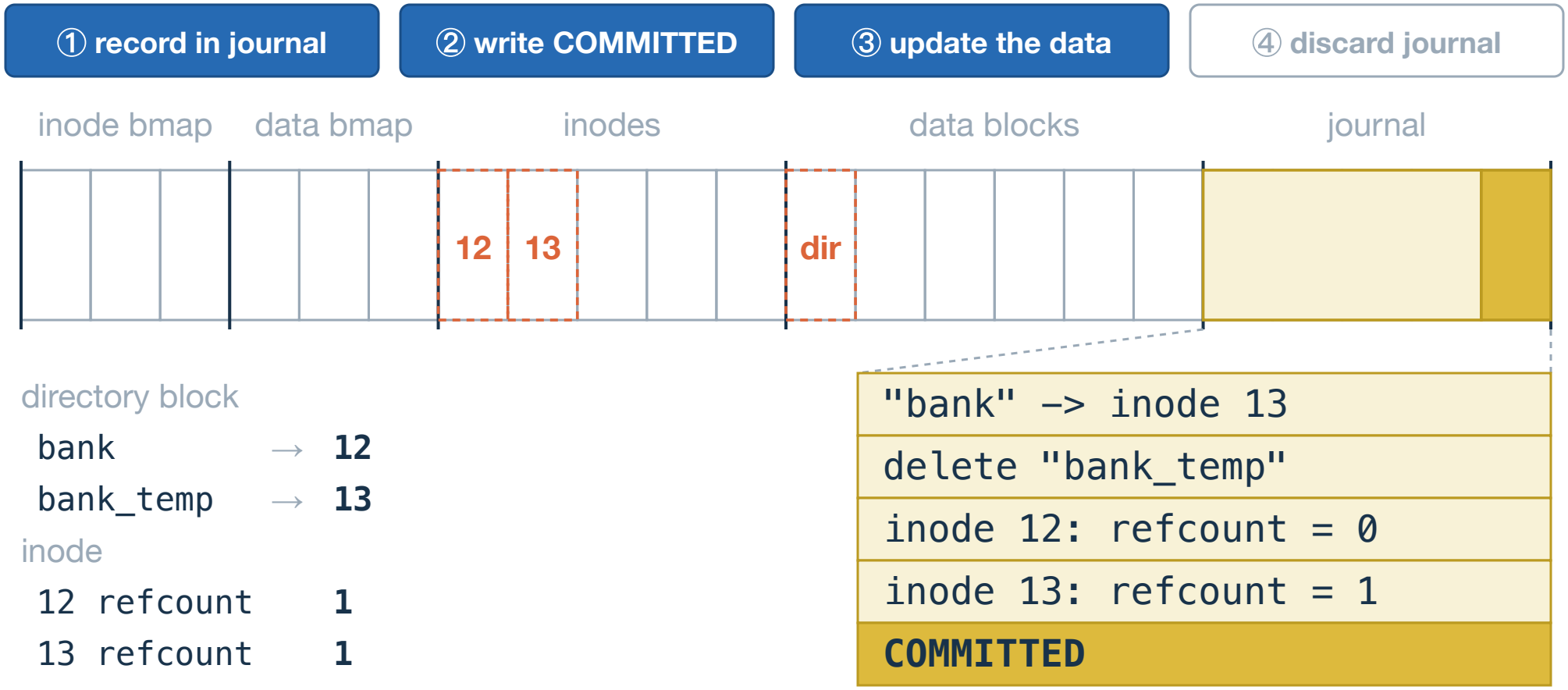
Case #1: rename via journaling

One record per change: which block it hits, and what that block becomes



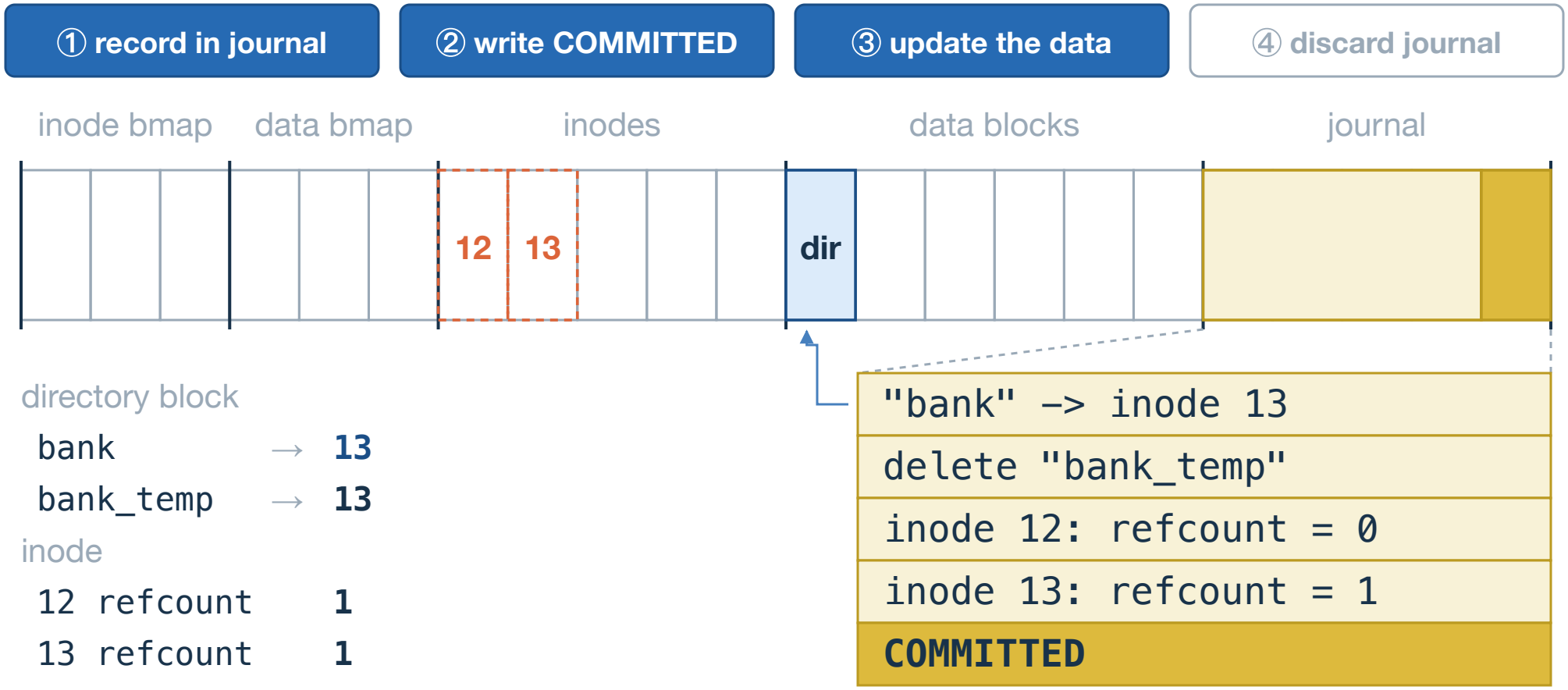
Case #1, crashed: recovering via the journal

Suppose we have met the inconsistency described before



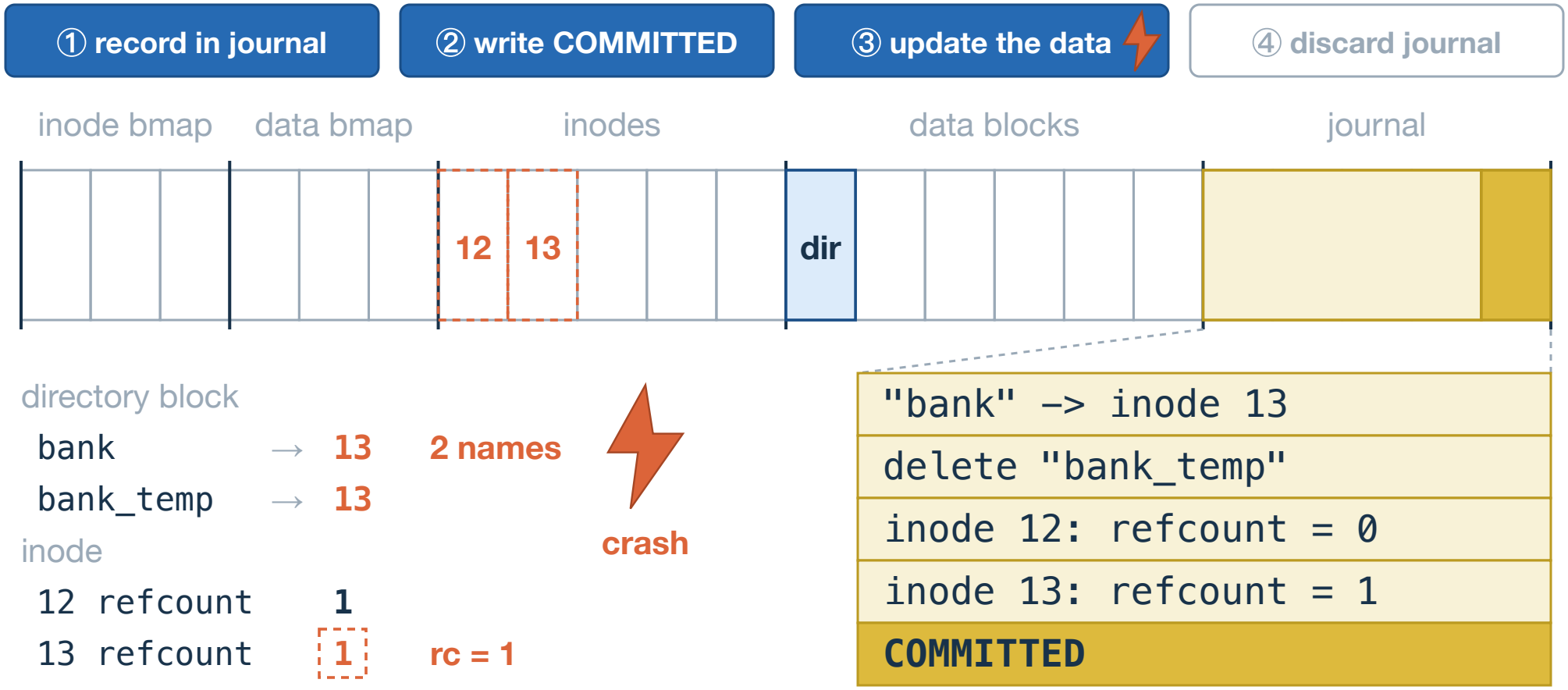
Case #1, crashed: recovering via the journal

Suppose we have met the inconsistency described before



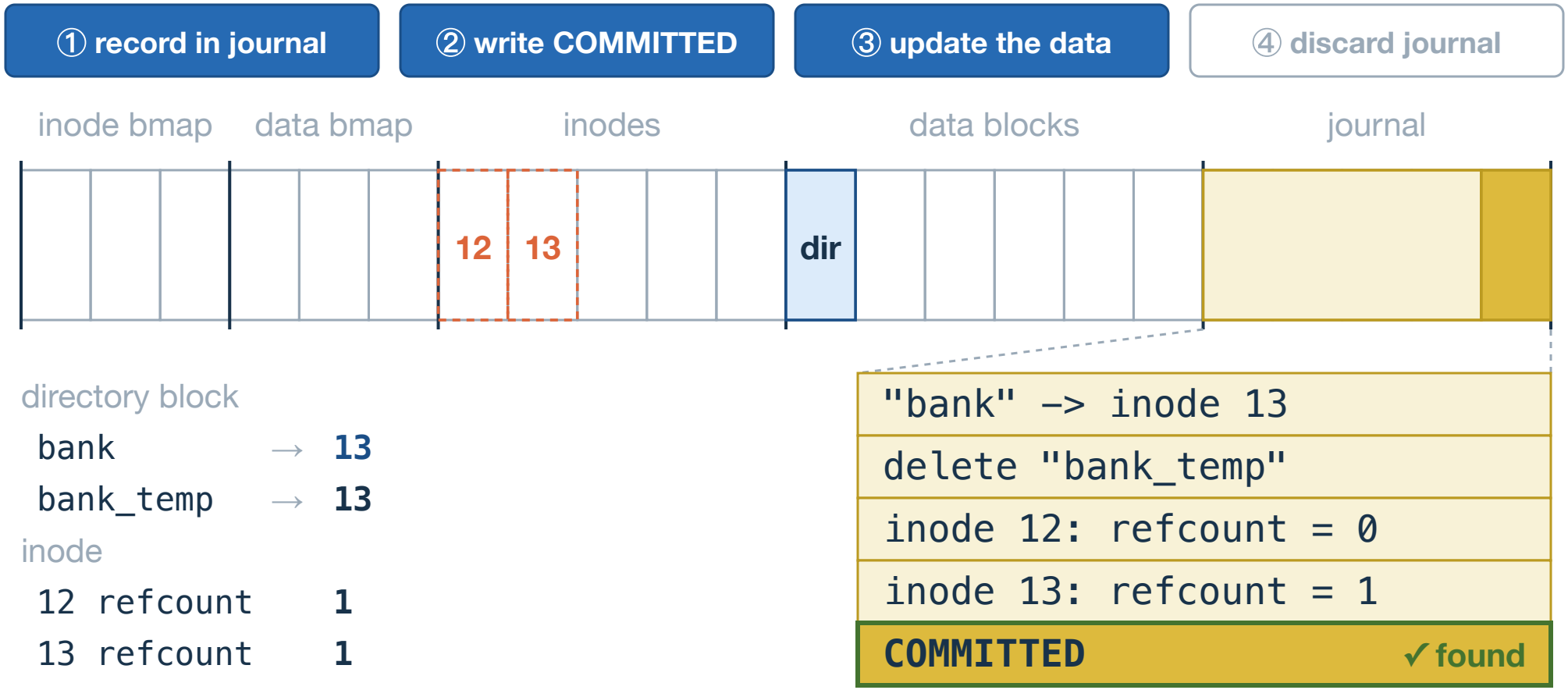
Case #1, crashed: recovering via the journal

Suppose we have met the inconsistency described before



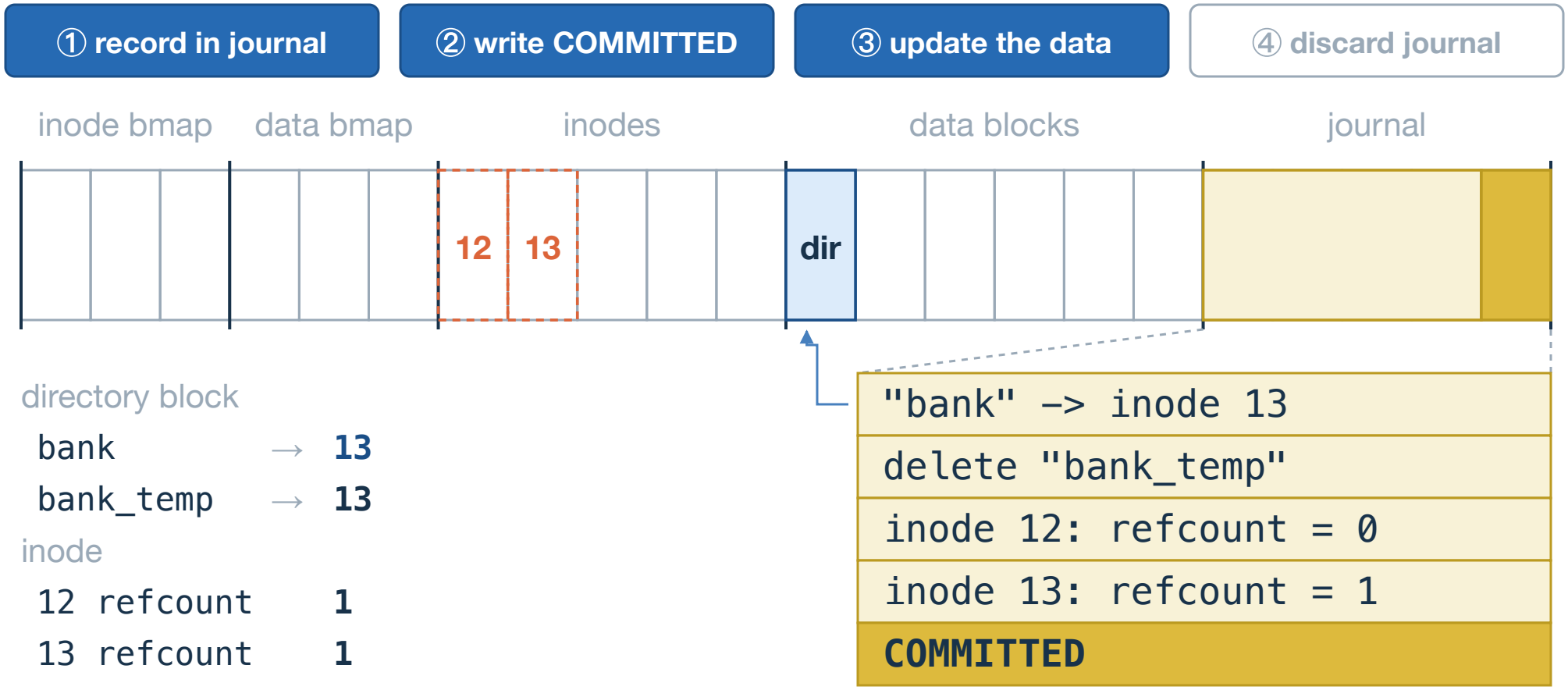
Case #1, crashed: recovering via the journal

Suppose we have met the inconsistency described before



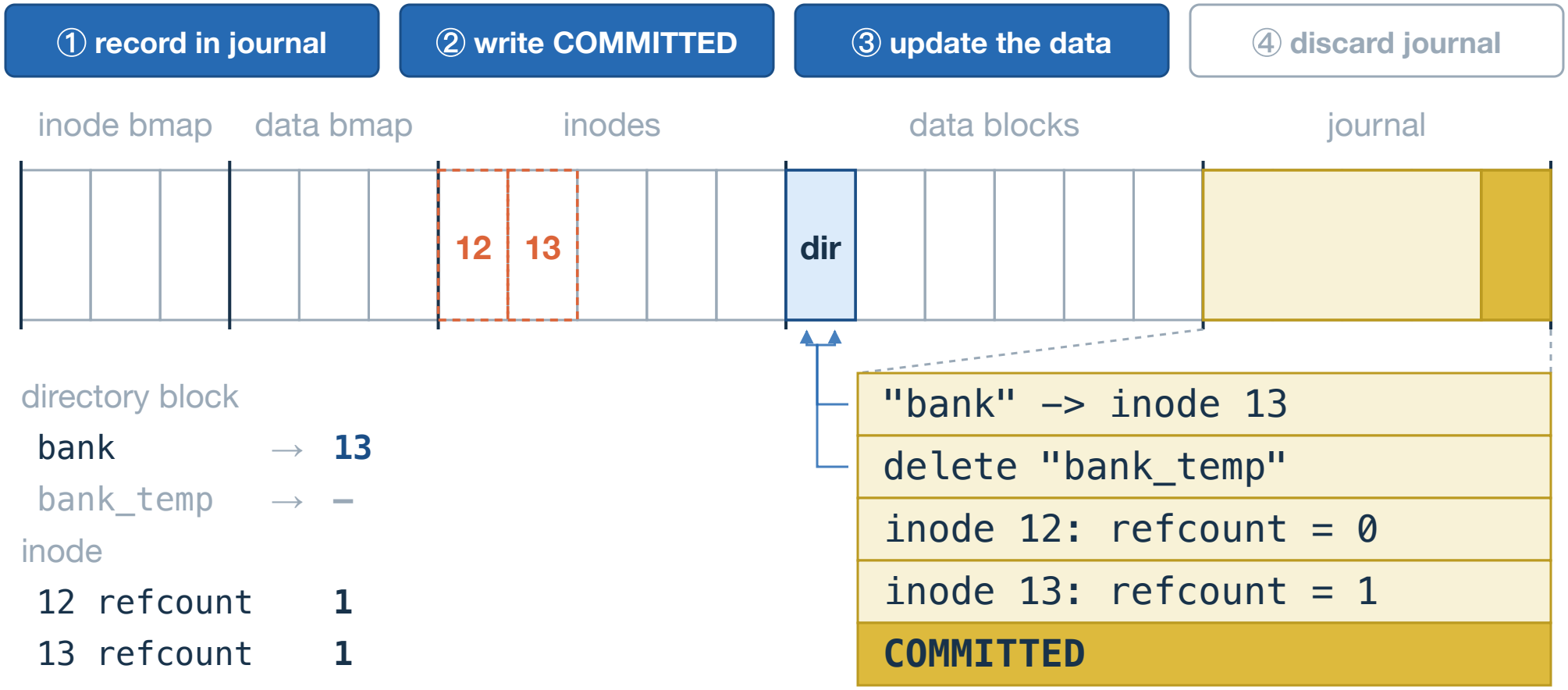
Case #1, crashed: recovering via the journal

Suppose we have met the inconsistency described before



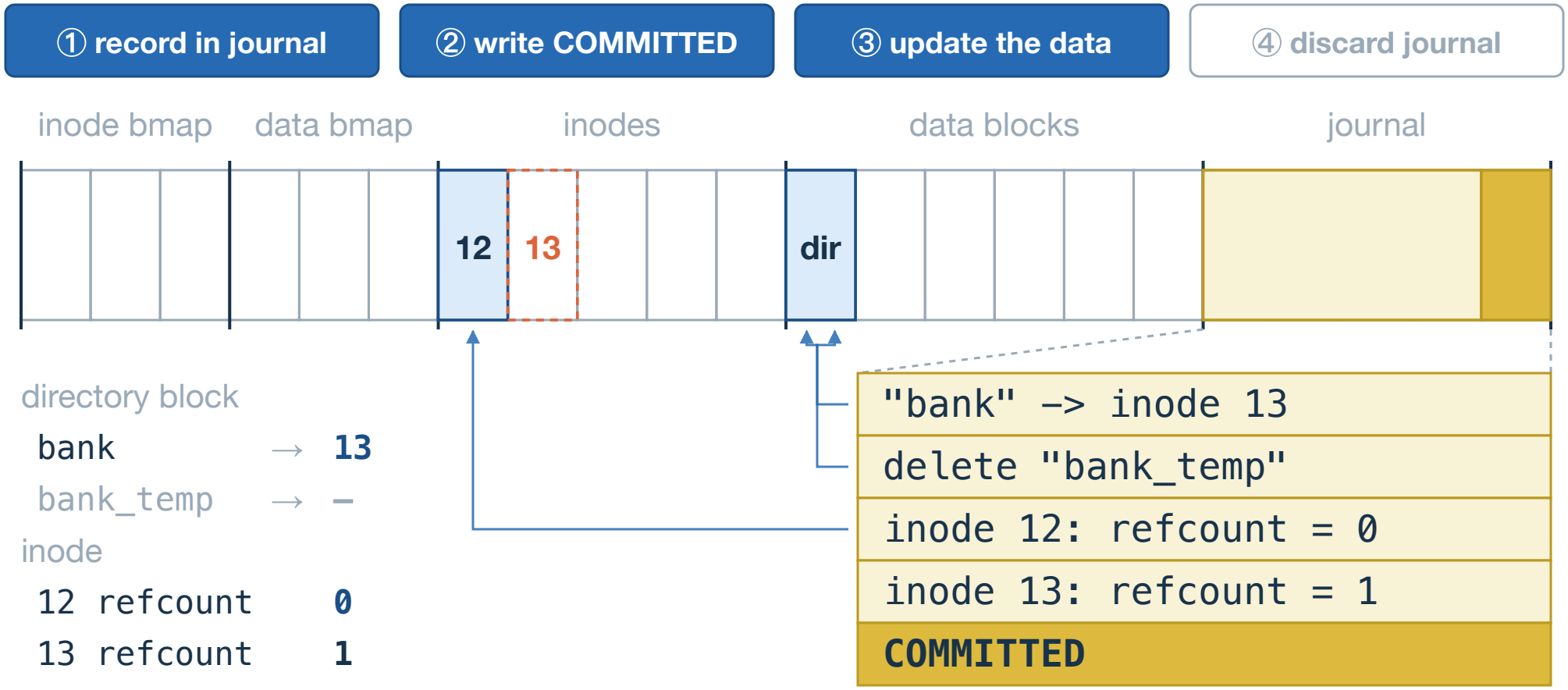
Case #1, crashed: recovering via the journal

Suppose we have met the inconsistency described before



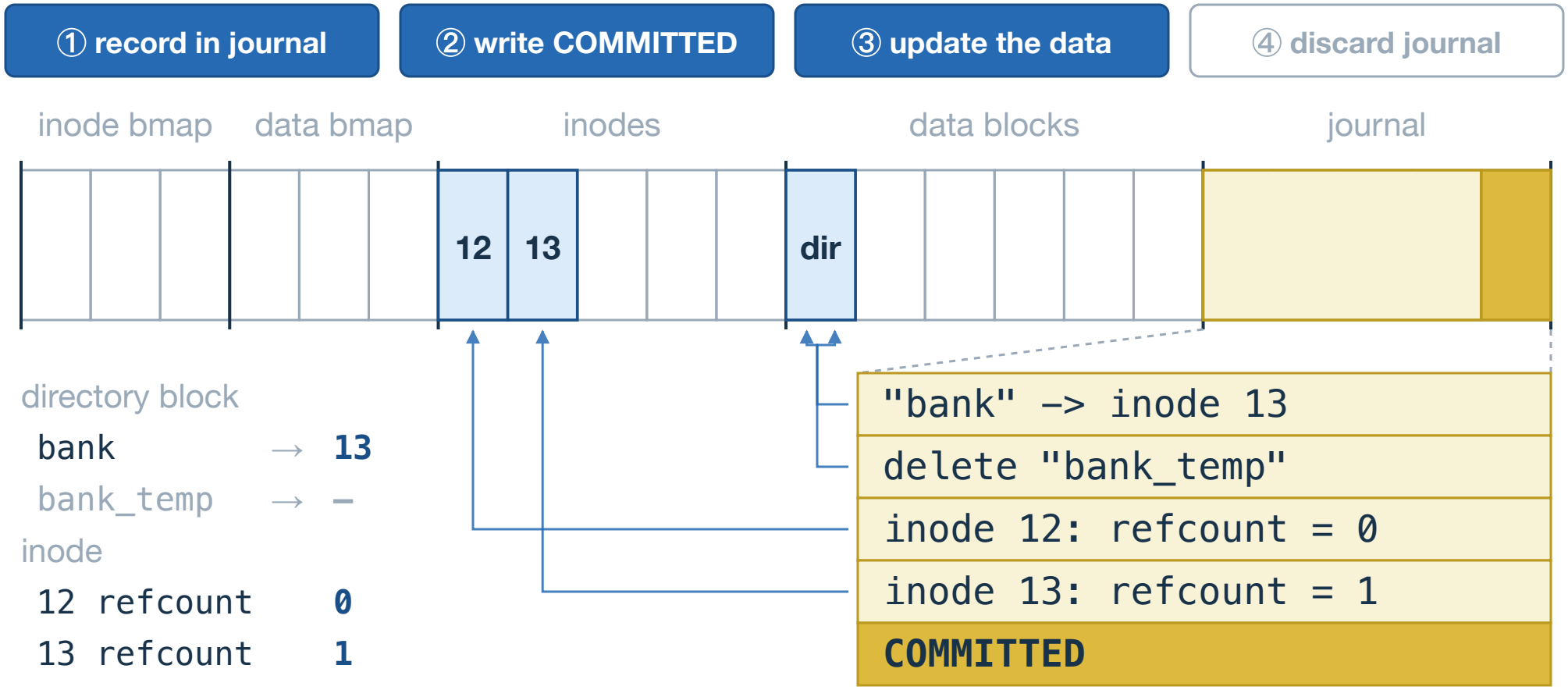
Case #1, crashed: recovering via the journal

Suppose we have met the inconsistency described before



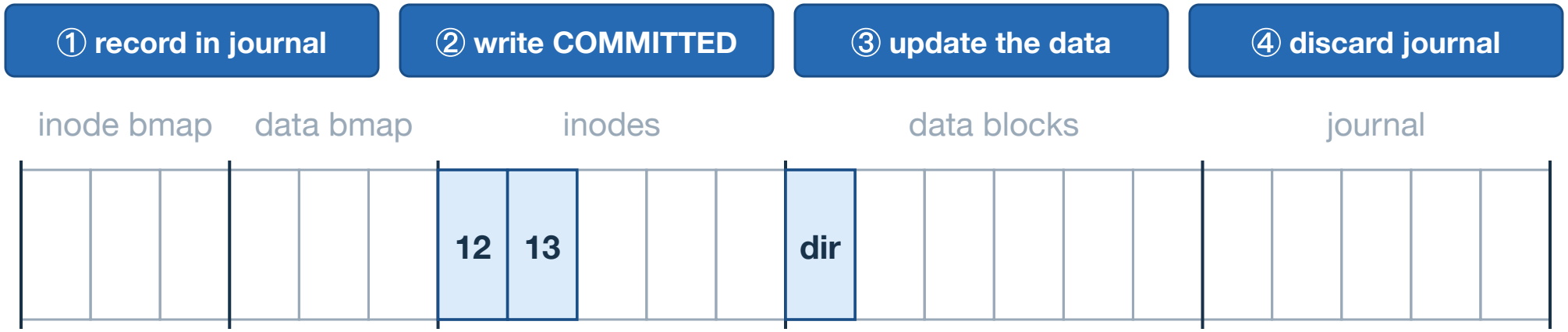
Case #1, crashed: recovering via the journal

Suppose we have met the inconsistency described before



Case #1, crashed: recovering via the journal

Suppose we have met the inconsistency described before



directory block

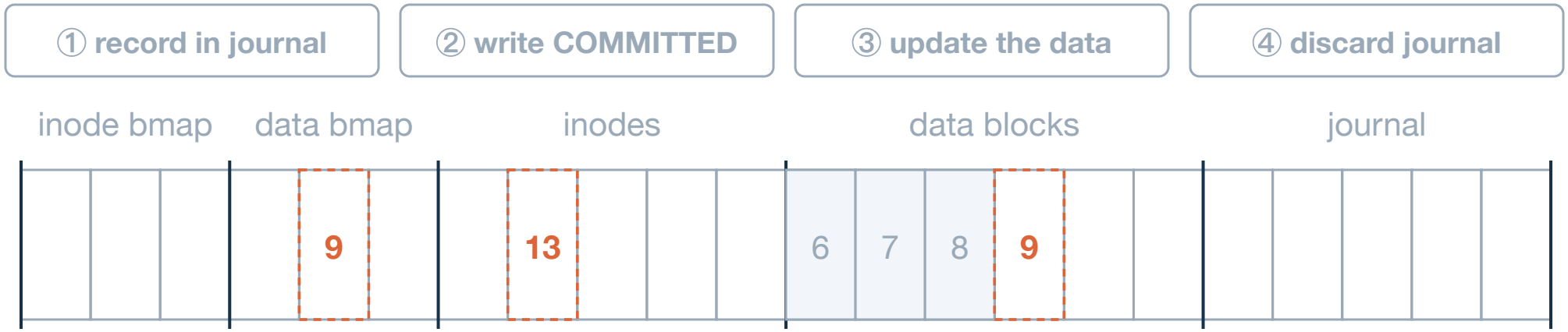
```
bank        →  13
bank_temp   →  –
```

inode

```
12 refcount    0
13 refcount    1
```

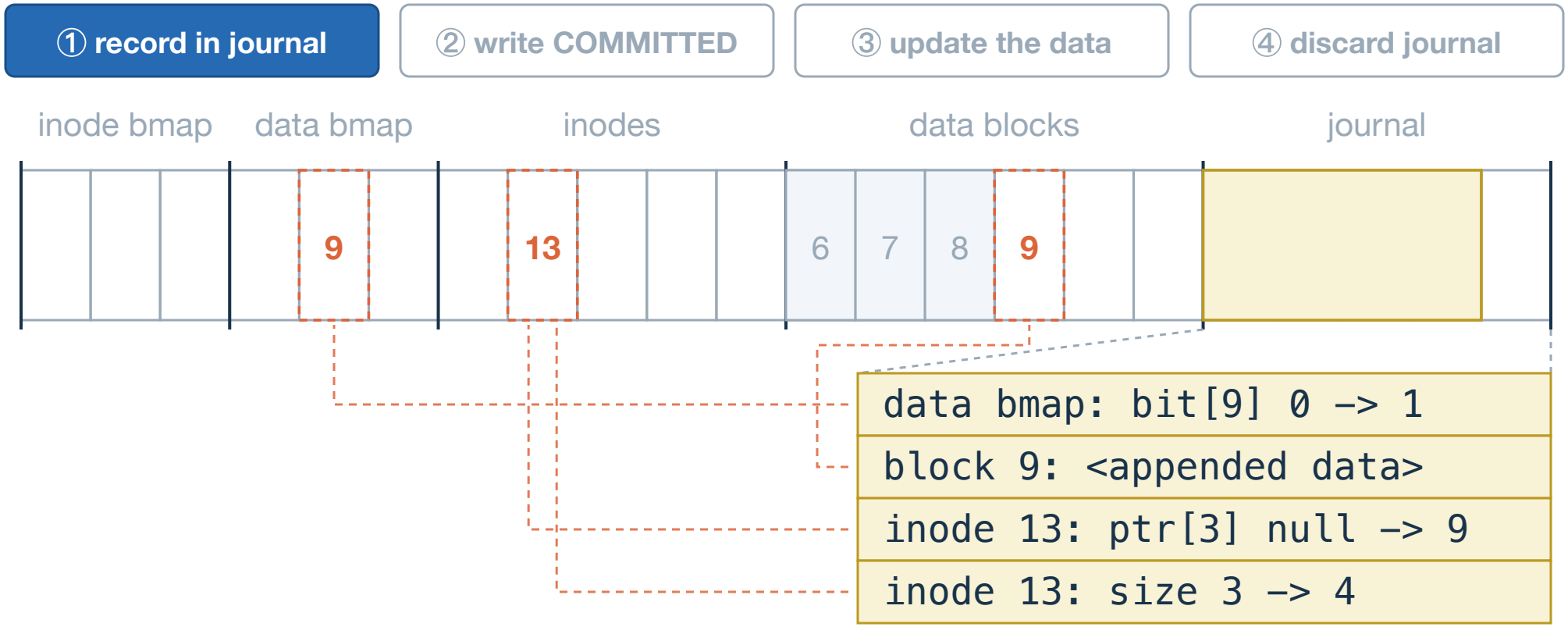
Case #2: appending to a file, via journaling

Any file system operation's modifications can be recorded in the journal



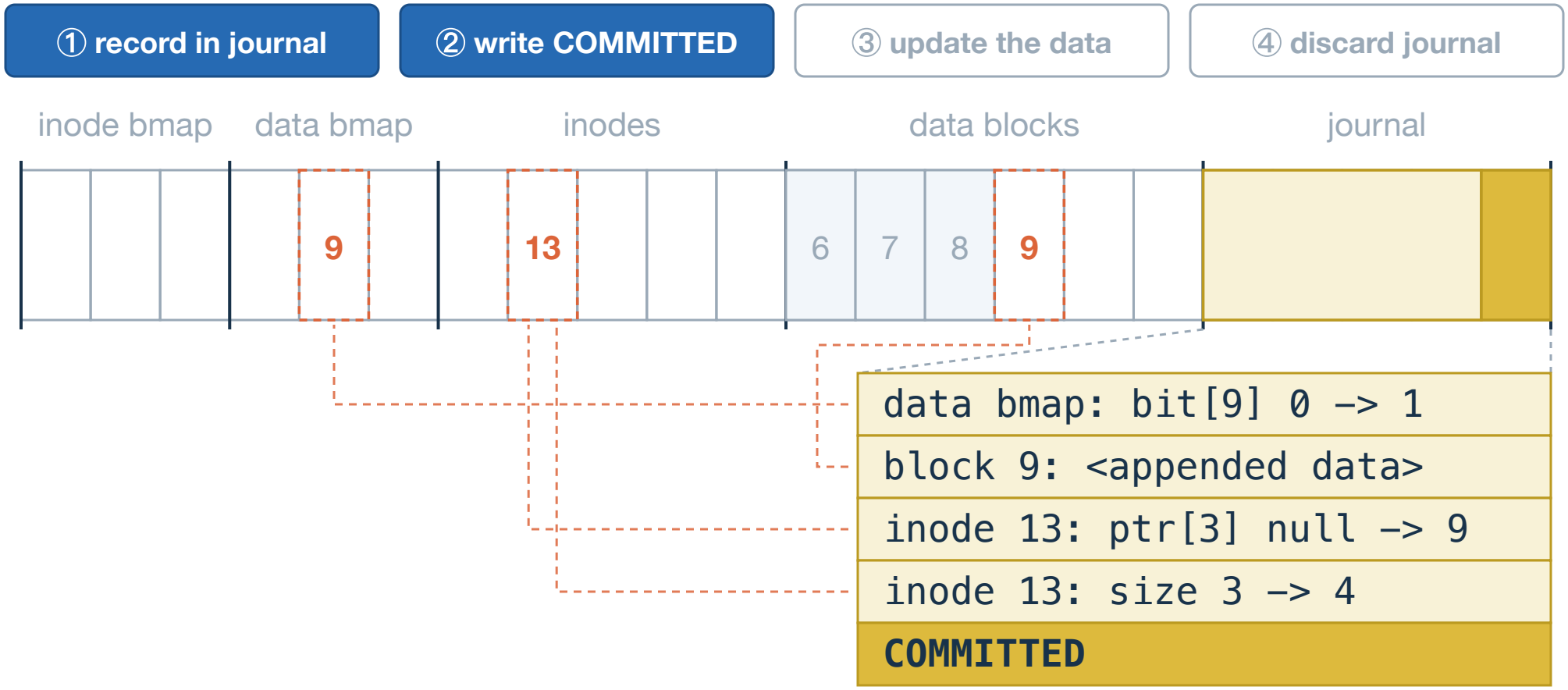
Case #2: appending to a file, via journaling

Any file system operation's modifications can be recorded in the journal



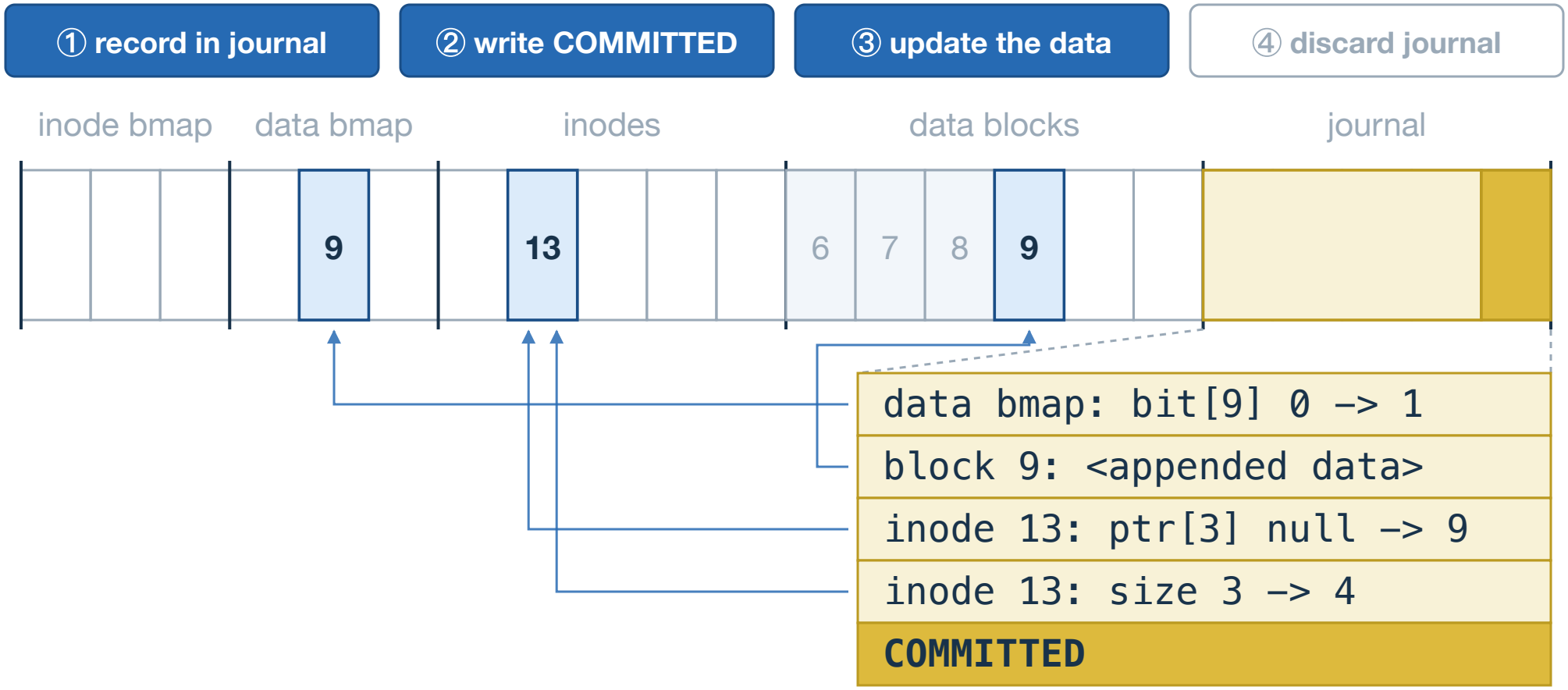
Case #2: appending to a file, via journaling

Any file system operation's modifications can be recorded in the journal



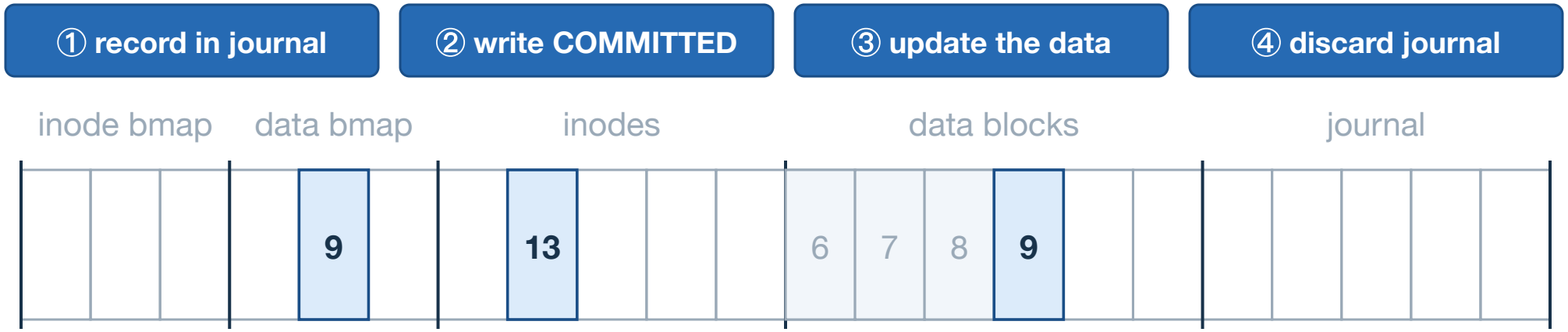
Case #2: appending to a file, via journaling

Any file system operation's modifications can be recorded in the journal



Case #2: appending to a file, via journaling

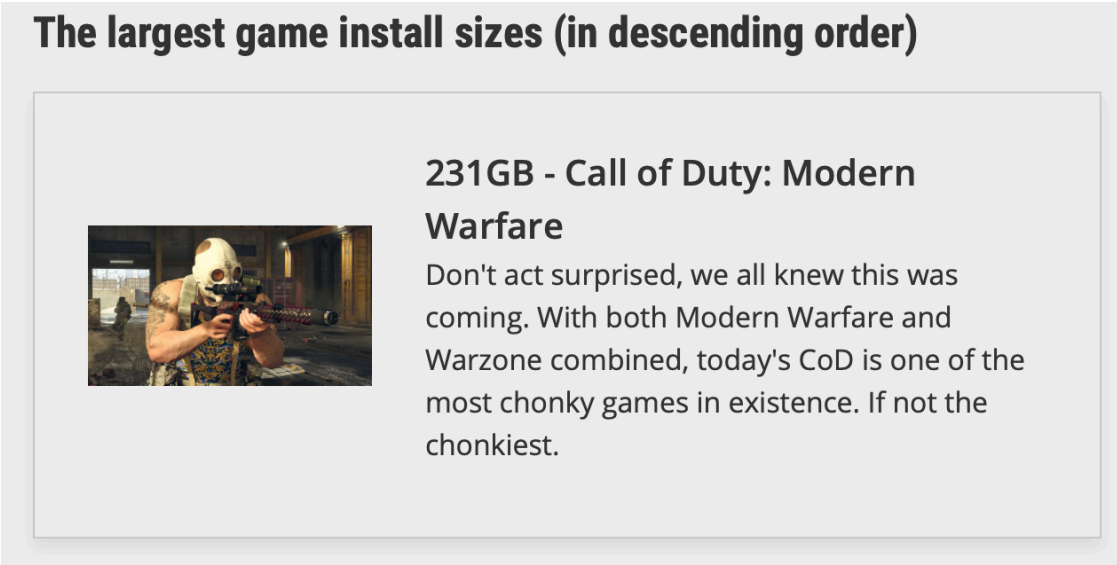
Any file system operation's modifications can be recorded in the journal



The cost of journaling: everything is written twice

Every piece of update writes the disk twice

- Once in the journal
- Once at its home location



[1]

When files are large, this 2X becomes expensive

[1] Screenshot of a game install-size ranking, to give a sense of how large files can be

Mitigation: selective journaling (e.g., only the metadata)

Observation: not everything in a file system is equally important

- Broken metadata breaks the whole file system (e.g., which block belongs to which inode)
- Broken data breaks only that one file's content, and can be detected by the application itself, e.g., via a checksum

Approach: only metadata goes through the journal; data is written only once

- Metadata is typically small

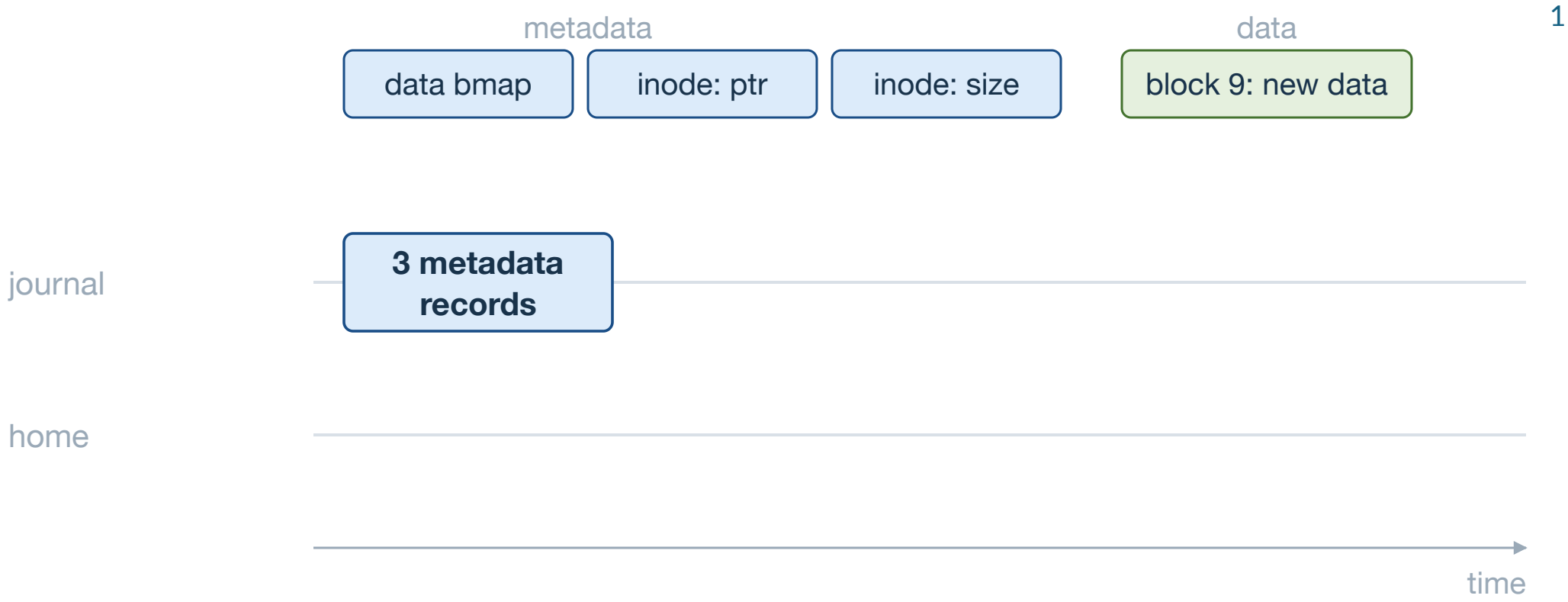
What if the data is also important?

- ext4 also provides options: `data=journal` / `ordered` / `writeback`
- The application can also handle it itself: `fsync` the data first, then proceed

data=writeback: journal the metadata only

Example: append block 9 to the bank file

- The journal carries only the three metadata changes

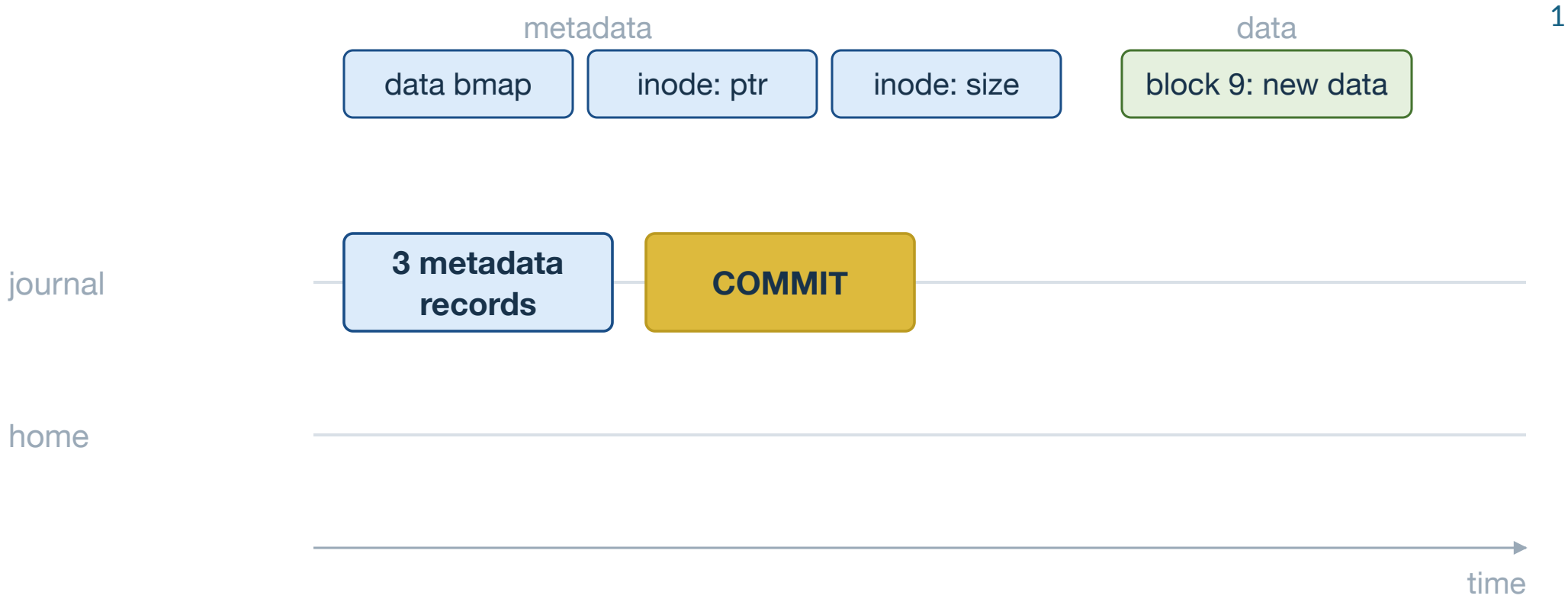


[1] Option semantics: Linux kernel ext4 documentation, mount options

data=writeback: journal the metadata only

Example: append block 9 to the bank file

- The journal carries only the three metadata changes



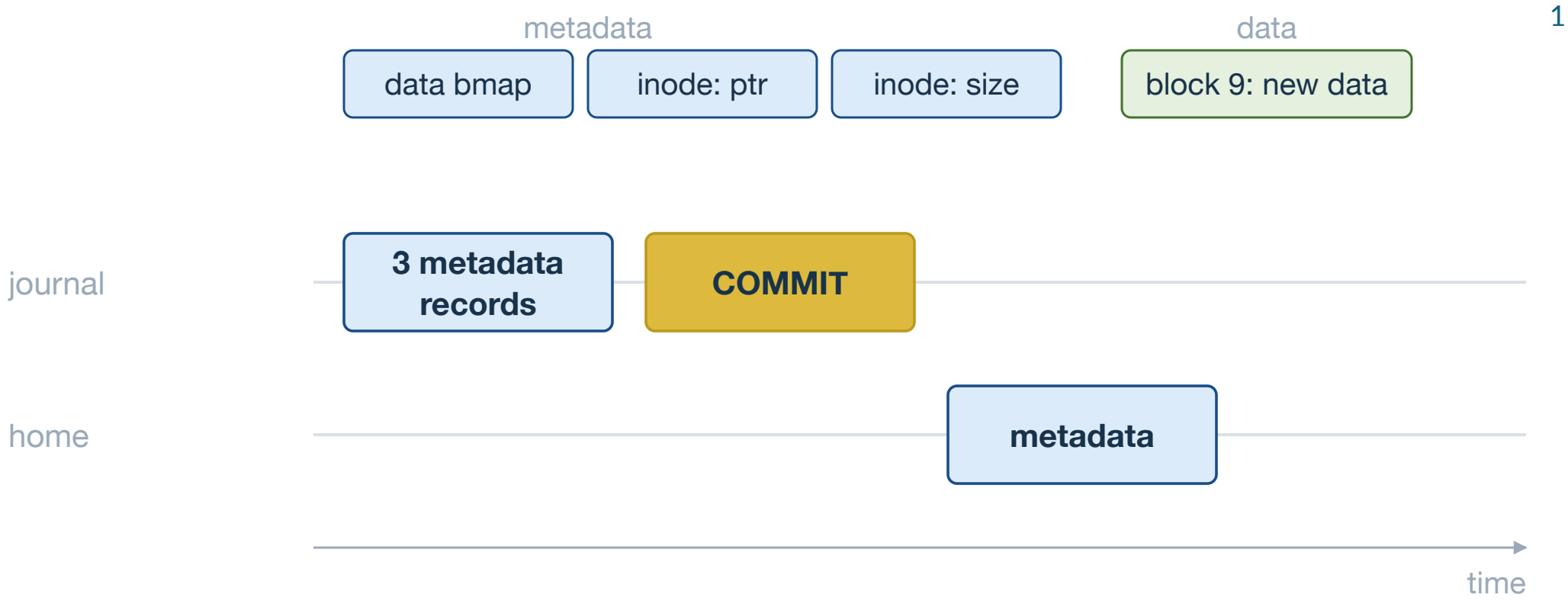
[1]

[1] Option semantics: Linux kernel ext4 documentation, mount options

data=writeback: journal the metadata only

Example: append block 9 to the bank file

- The journal carries only the three metadata changes



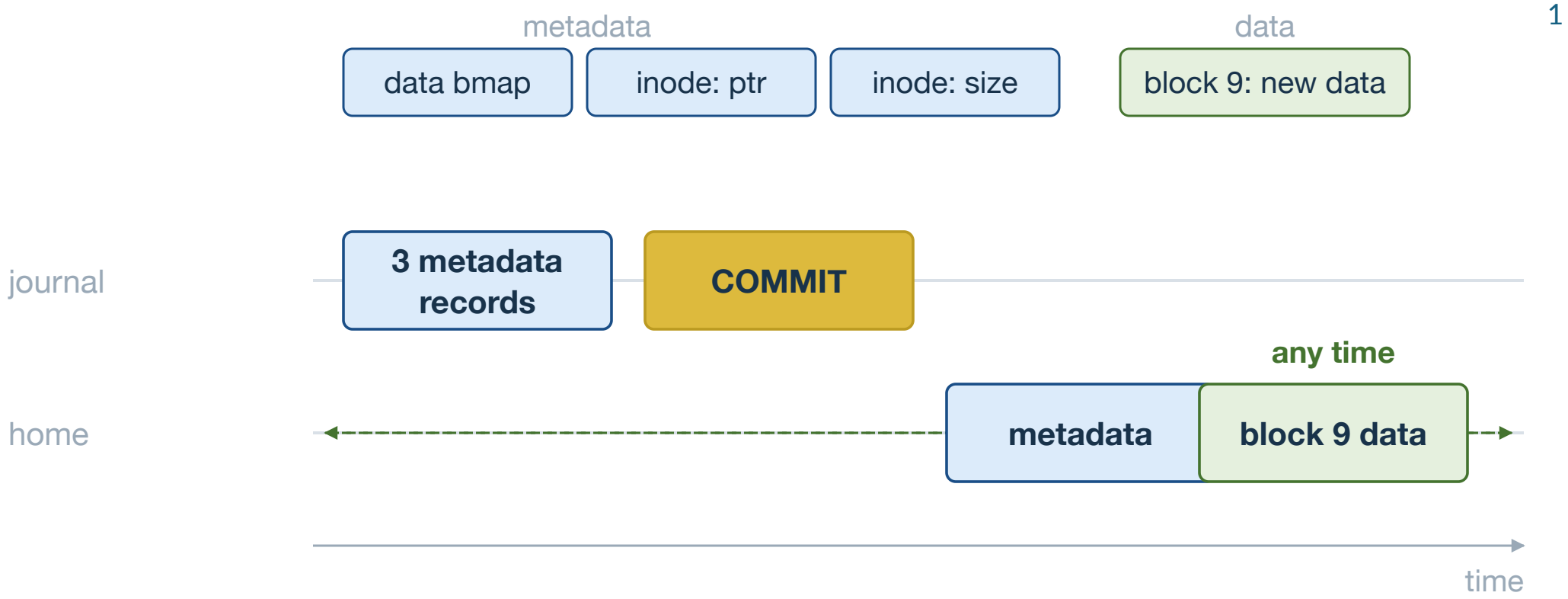
[1] Option semantics: Linux kernel ext4 documentation, mount options

data=writeback: journal the metadata only

Example: append block 9 to the bank file

- The journal carries only the three metadata changes

[1]



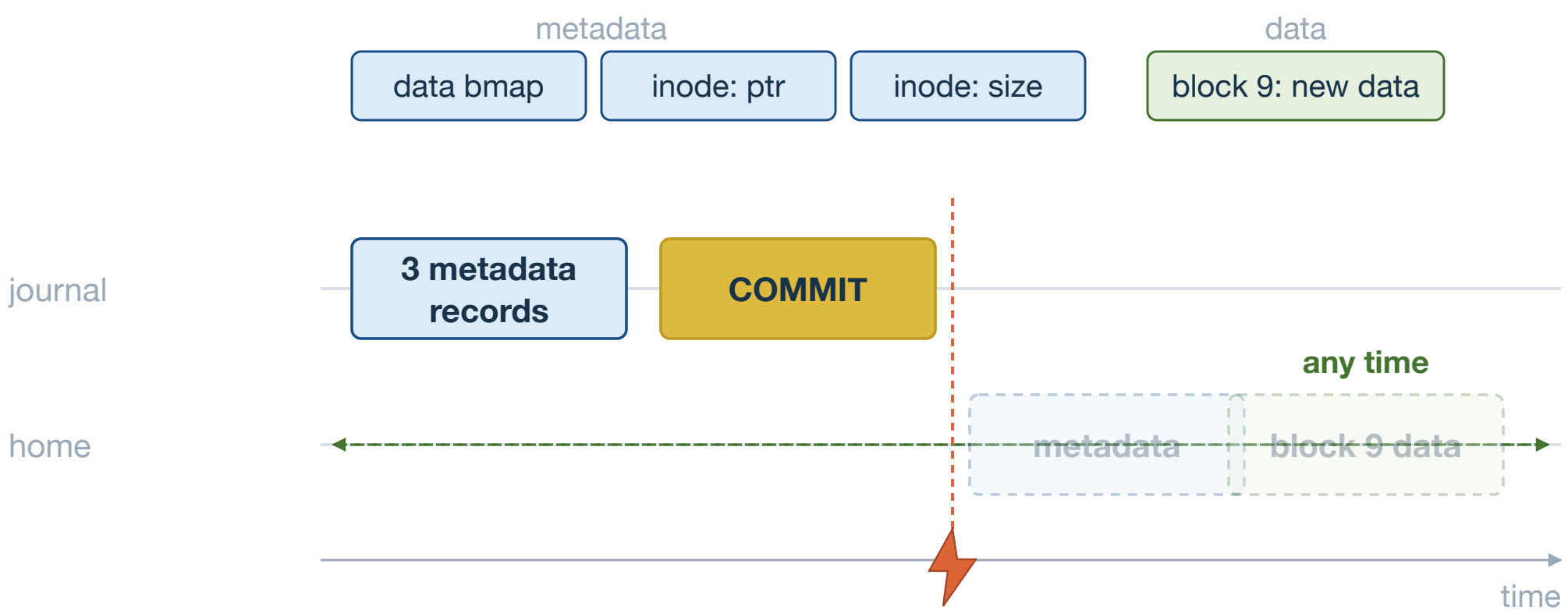
[1] Option semantics: Linux kernel ext4 documentation, mount options

data=writeback: journal the metadata only

Example: append block 9 to the bank file

- The journal carries only the three metadata changes

[1]



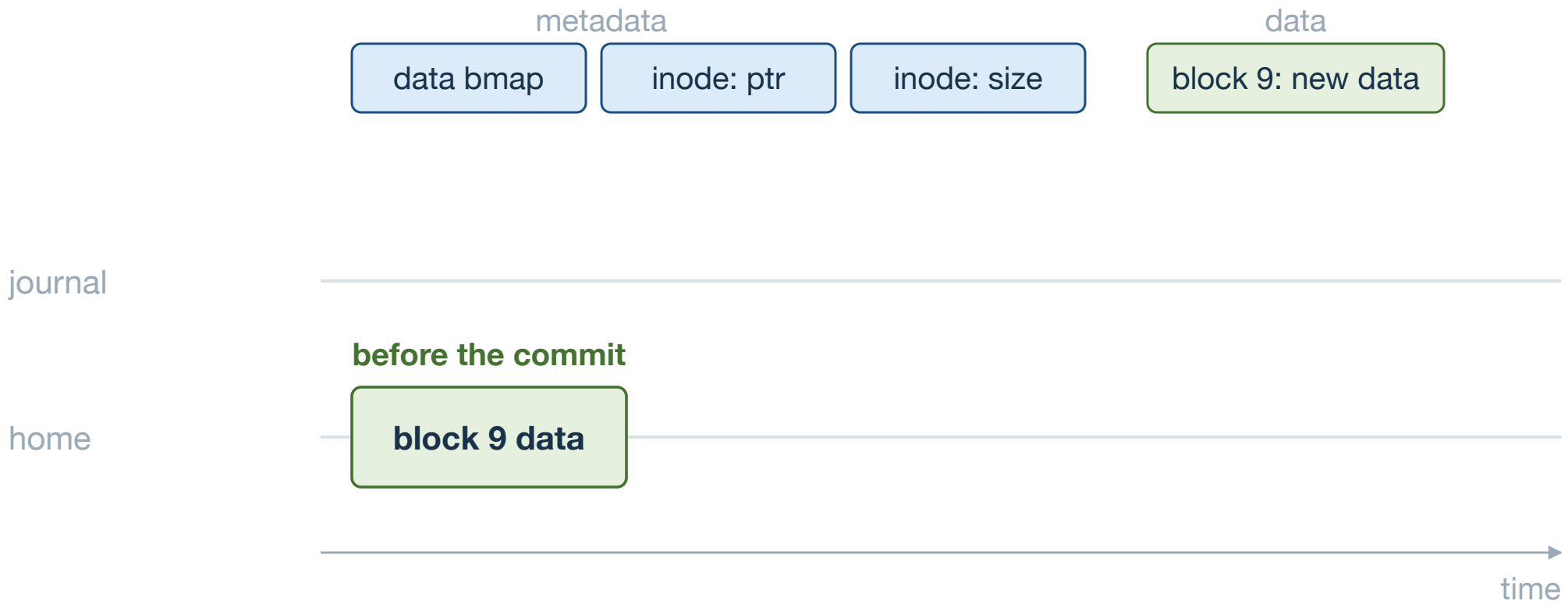
replay the journal: the file is now 4 blocks long

block 9 still holds whatever was there before

[1] Option semantics: Linux kernel ext4 documentation, mount options

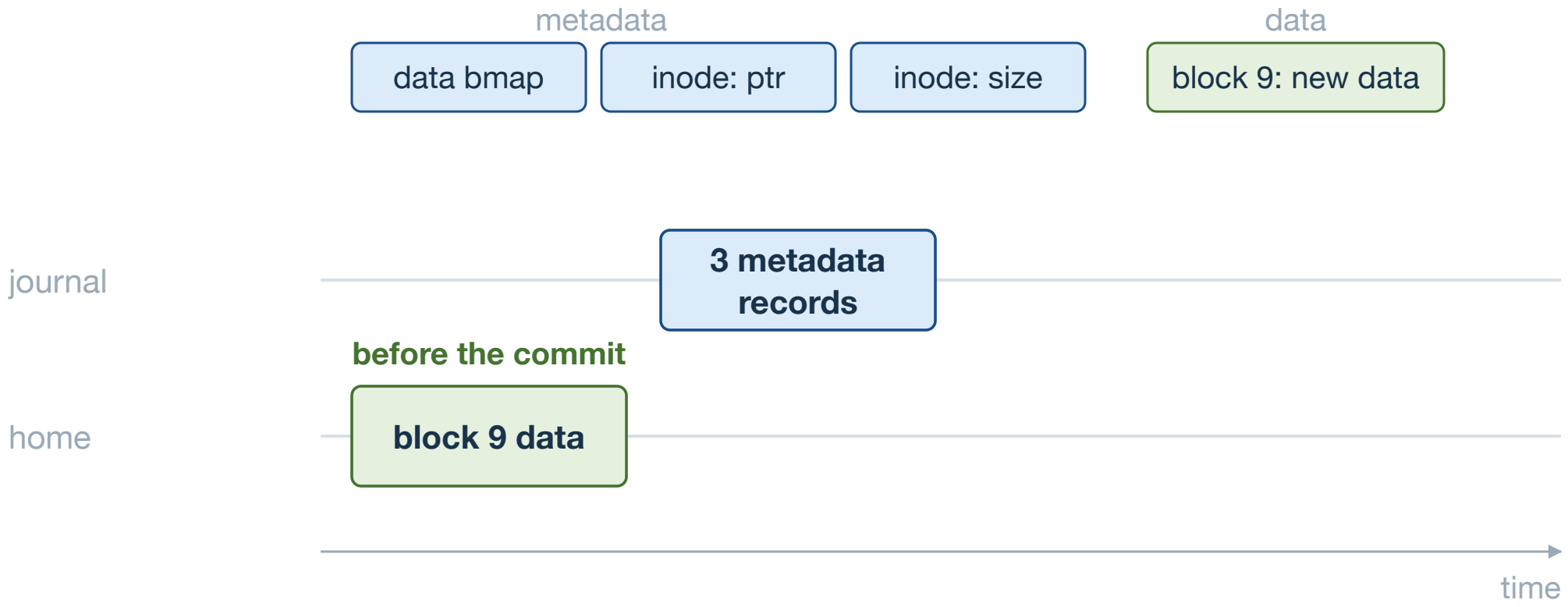
data=ordered: the data goes out first

Example: append block 9 to the bank file. If the file exists, then the data is correct



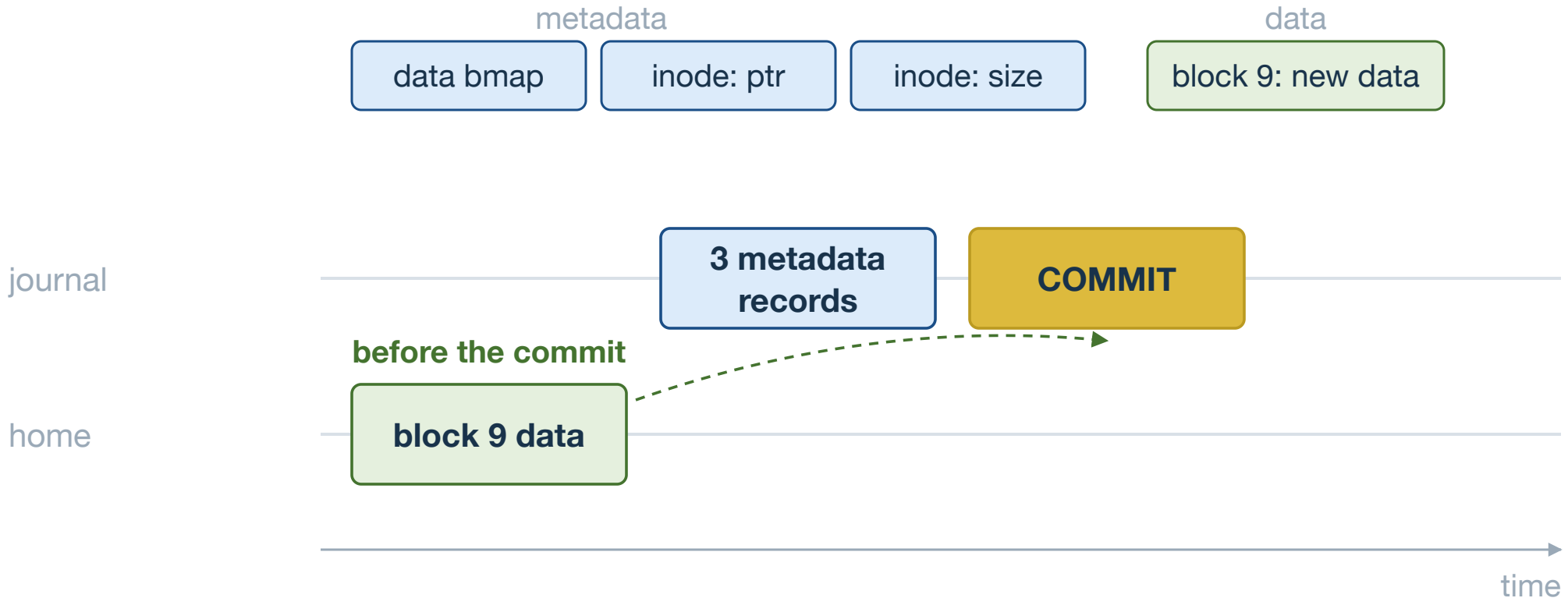
data=ordered: the data goes out first

Example: append block 9 to the bank file. If the file exists, then the data is correct



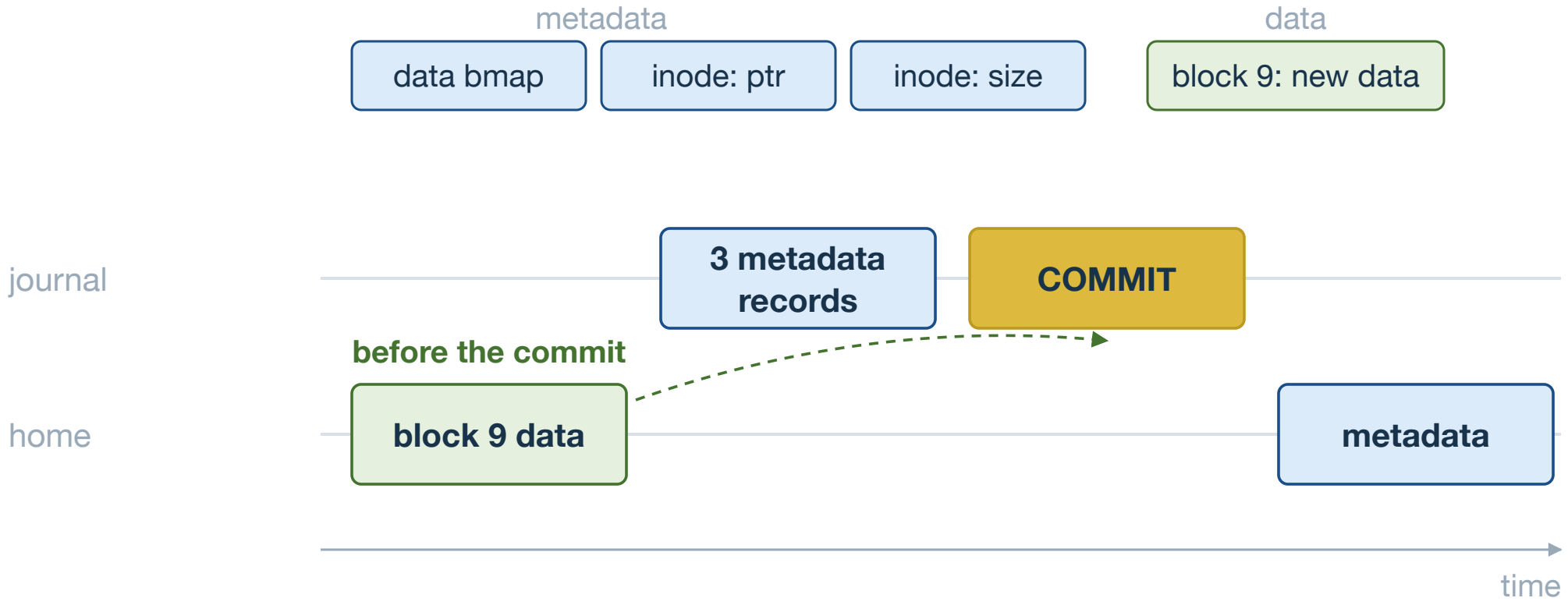
data=ordered: the data goes out first

Example: append block 9 to the bank file. If the file exists, then the data is correct



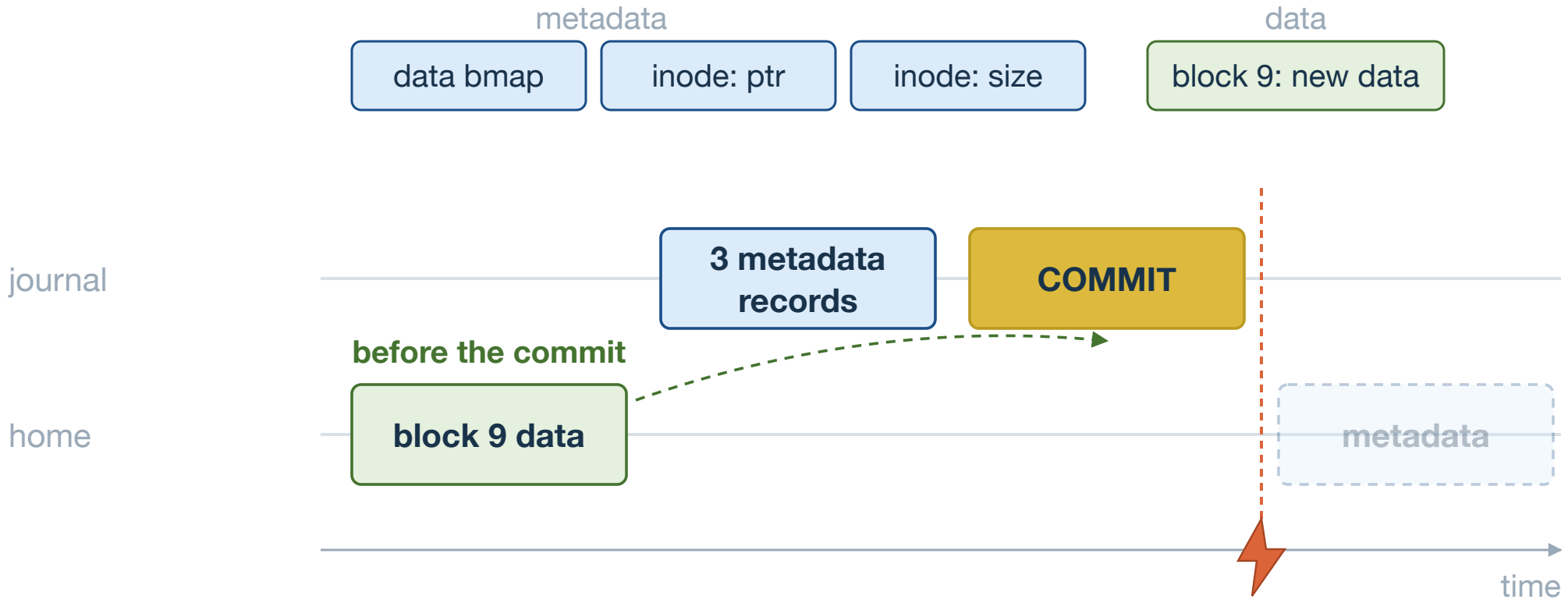
data=ordered: the data goes out first

Example: append block 9 to the bank file. If the file exists, then the data is correct



data=ordered: the data goes out first

Example: append block 9 to the bank file. If the file exists, then the data is correct

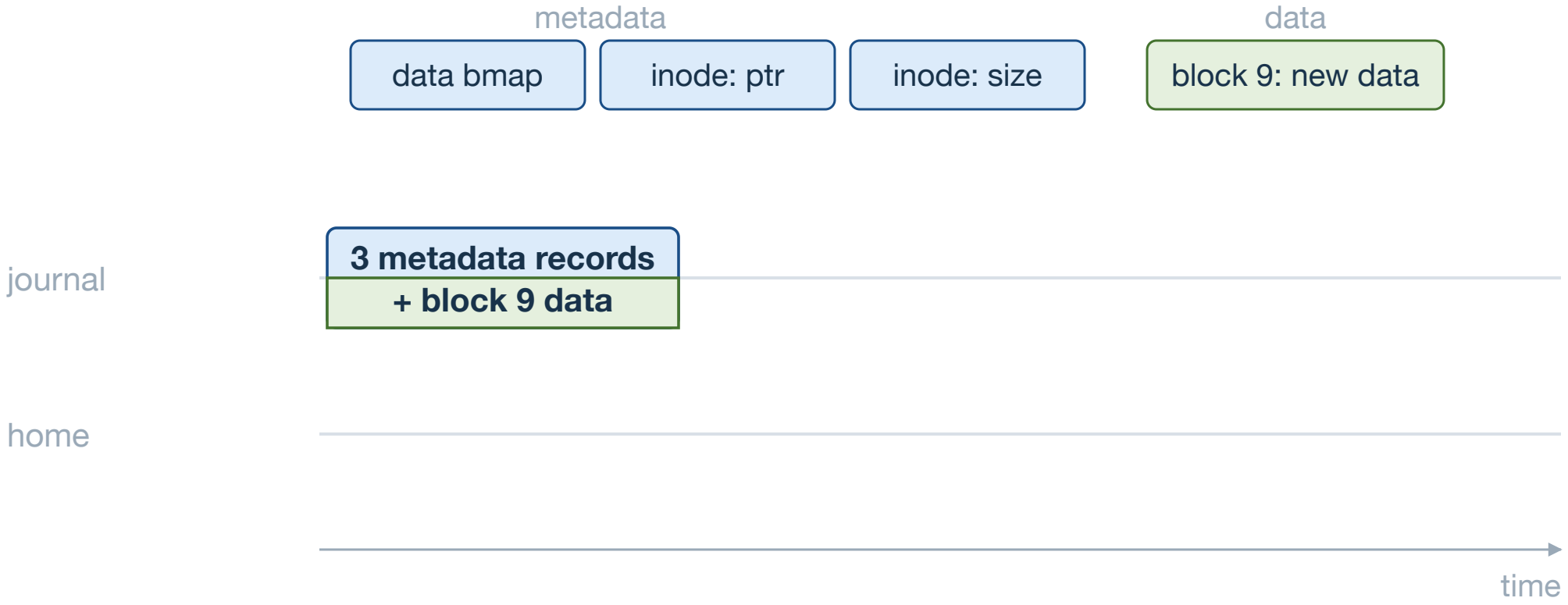


replay the journal: the file is now 4 blocks long

block 9 already holds the new data

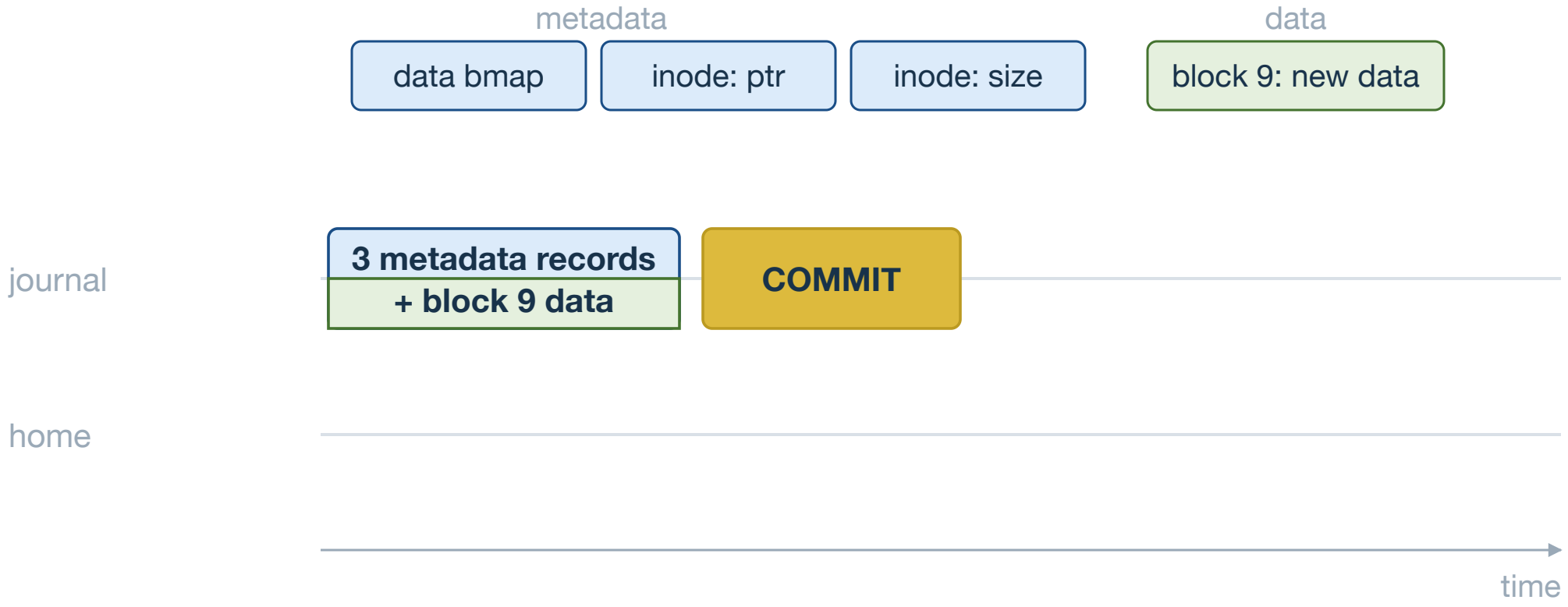
data=journal: the data goes through the journal too

The data goes into the journal as well: written twice, and now the data is all-or-nothing too



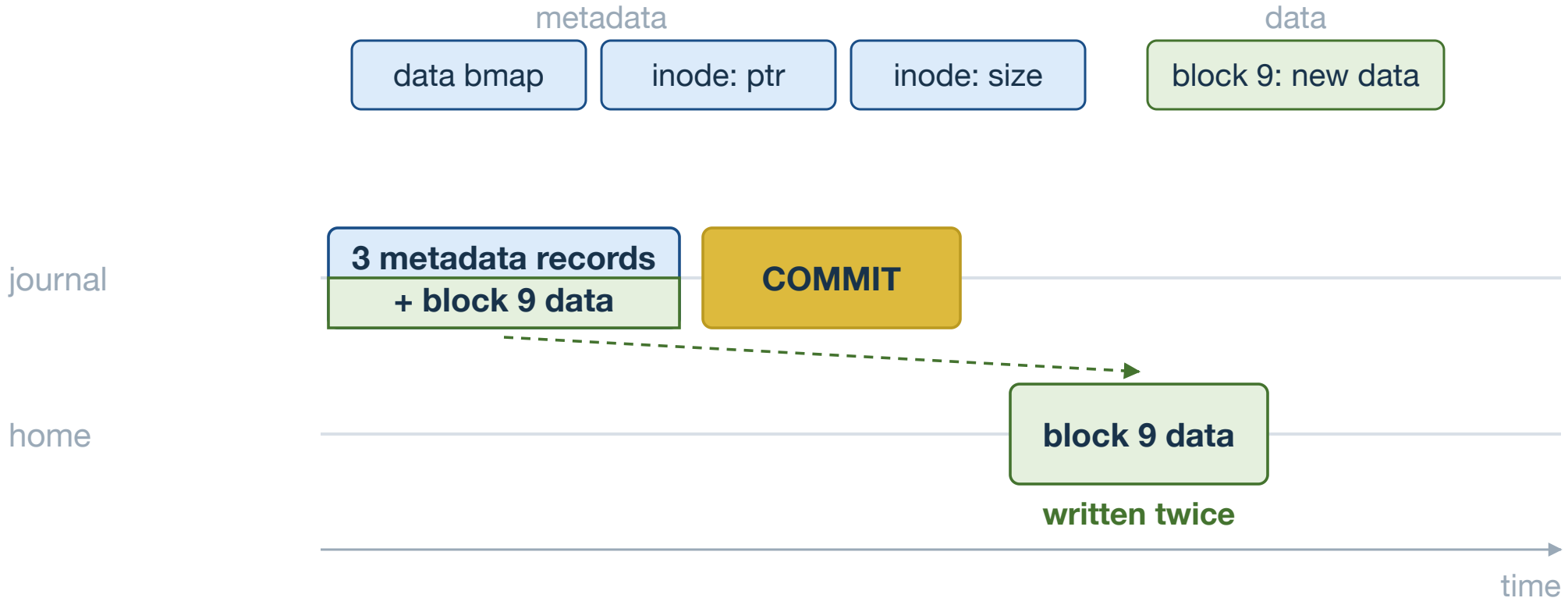
data=journal: the data goes through the journal too

The data goes into the journal as well: written twice, and now the data is all-or-nothing too



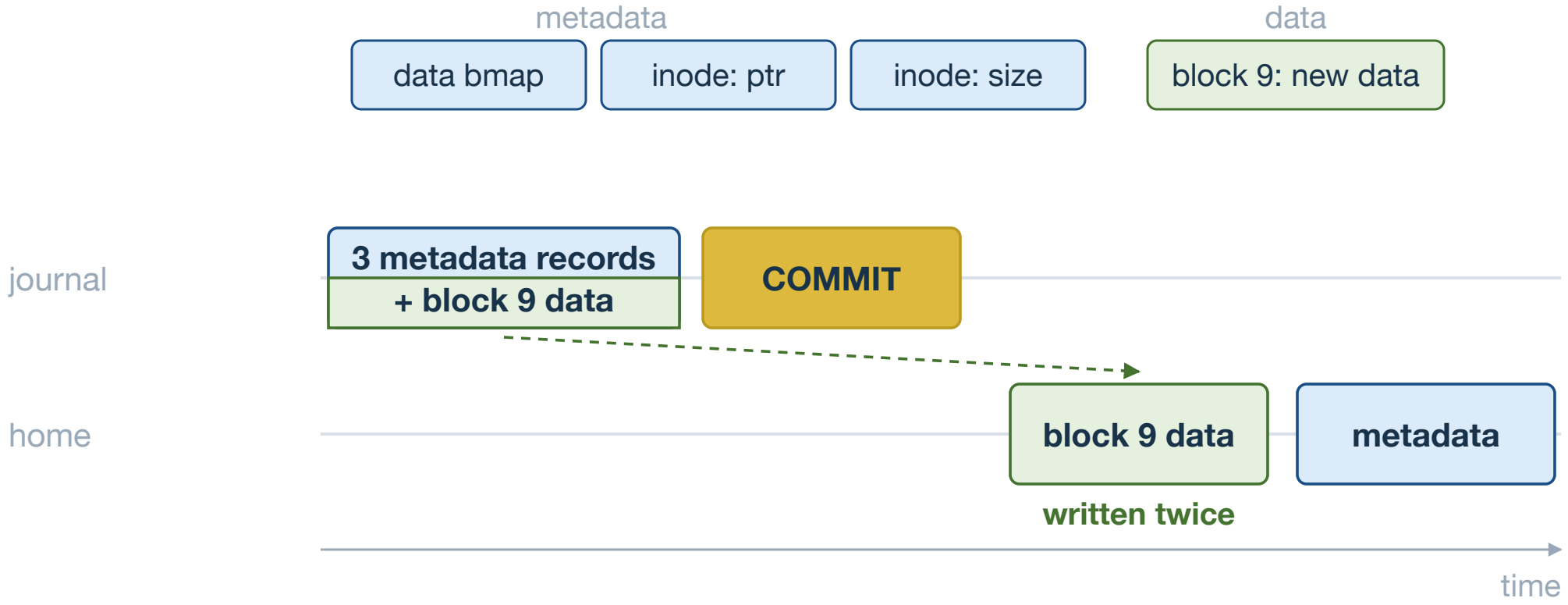
data=journal: the data goes through the journal too

The data goes into the journal as well: written twice, and now the data is all-or-nothing too



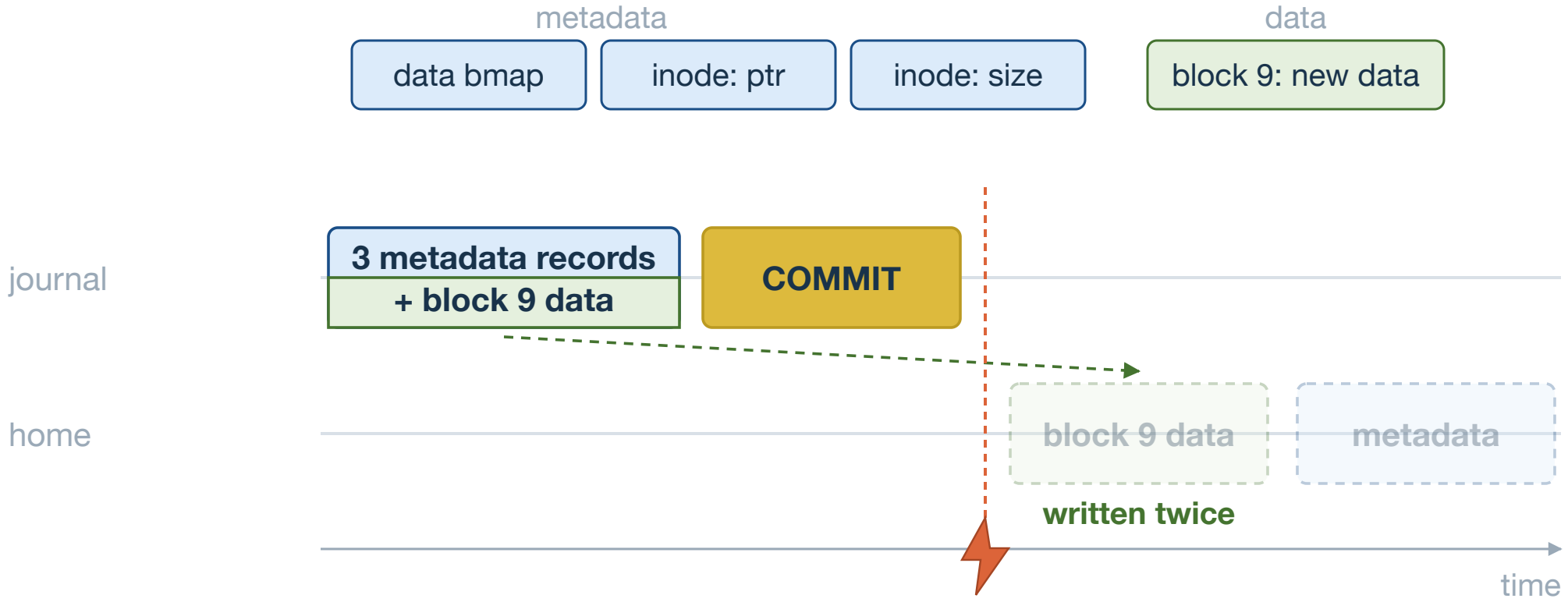
data=journal: the data goes through the journal too

The data goes into the journal as well: written twice, and now the data is all-or-nothing too



data=journal: the data goes through the journal too

The data goes into the journal as well: written twice, and now the data is all-or-nothing too



replay the journal: metadata and data are both in it

the append comes back whole, block 9 included

Question: what if we crash while writing the commit?

Journaling hinges on the commit write — is the commit itself atomic?

Usual assumption: writing one sector is all-or-nothing

- The disk saves enough energy (a small capacitor) to complete the sector in hand
- A write that never started is simply “nothing” — still all-or-nothing
- A sector is sufficient to record all metadata changes

But what if the record is larger than one sector?

- We can use checksum to check the integrity of the journal

Journaling makes rename atomic, so shadow copy is now complete.

The file system keeps itself consistent; the application keeps itself consistent.

But is shadow copy sufficient?

Question: two clients modify the same file

Two clients run the same code, only with different accounts

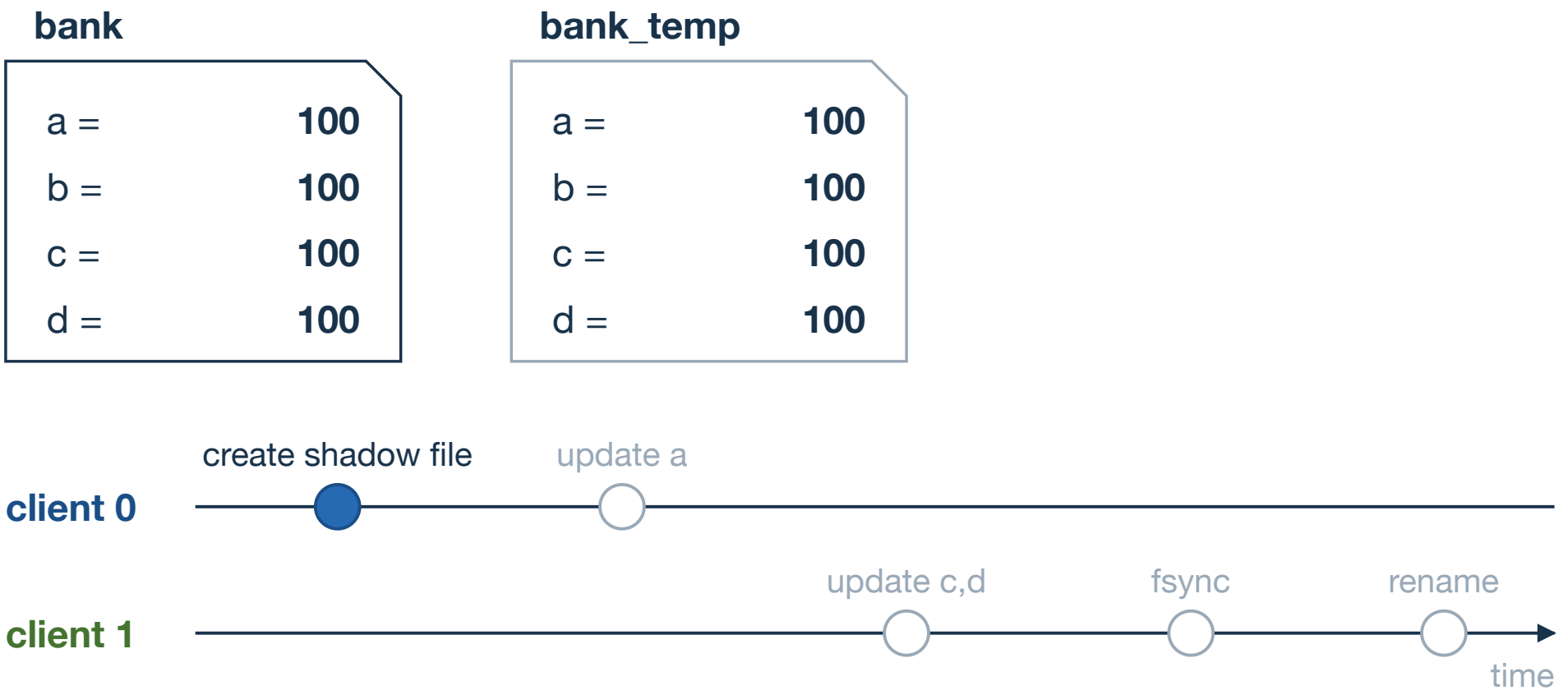
- client 0: `transfer(bank, a, b, 100)`
- client 1: `transfer(bank, c, d, 100)`

```
transfer(bank, a, b, amt):
    fcopy(bank, bank_temp)
    records = mmap(bank_temp, ...)
    records[a] = records[a] - amt
    records[b] = records[b] + amt
    fsync(bank_temp, ...)
    rename(bank_temp, bank)
```

The four accounts do not overlap, so the two transfers are logically independent

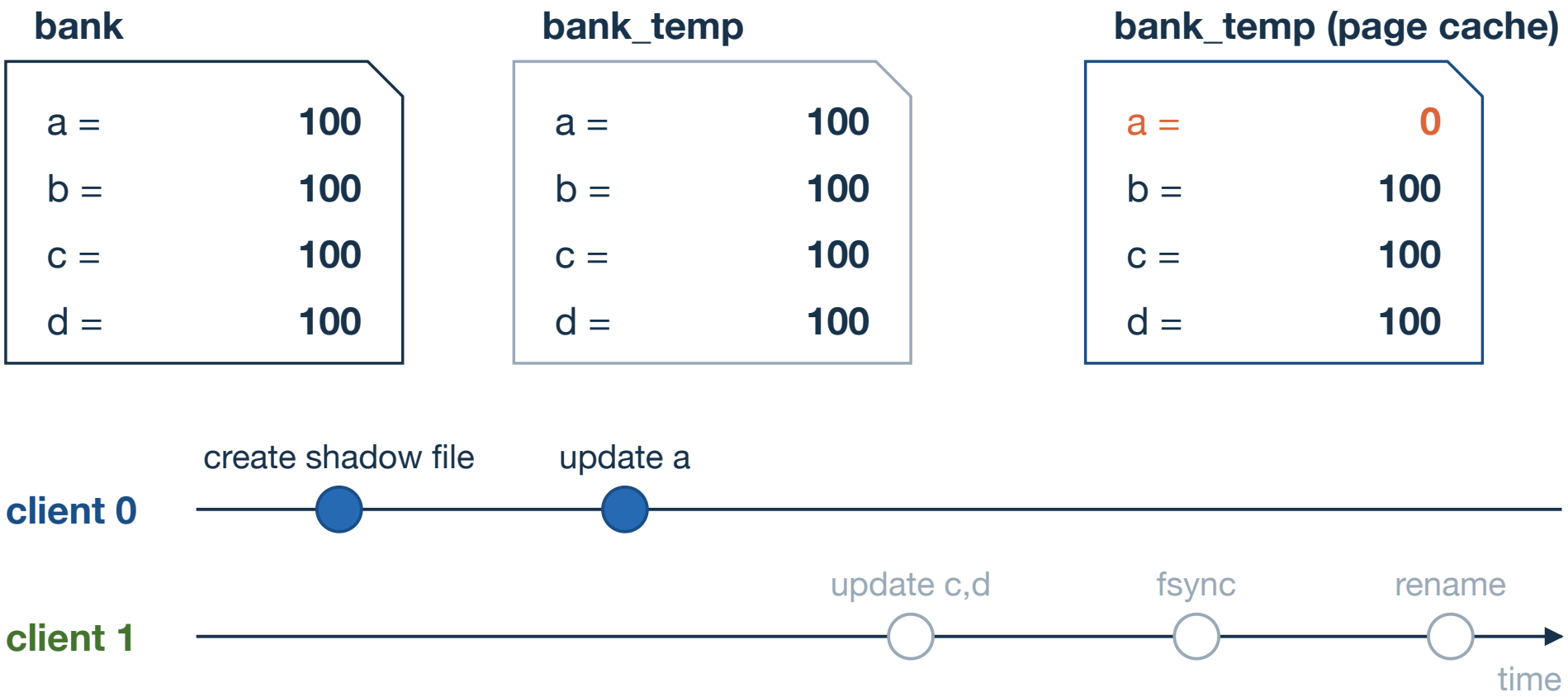
A case study of two operations using shadow copy

The two transfers touch disjoint accounts, but both must copy the same `bank` file



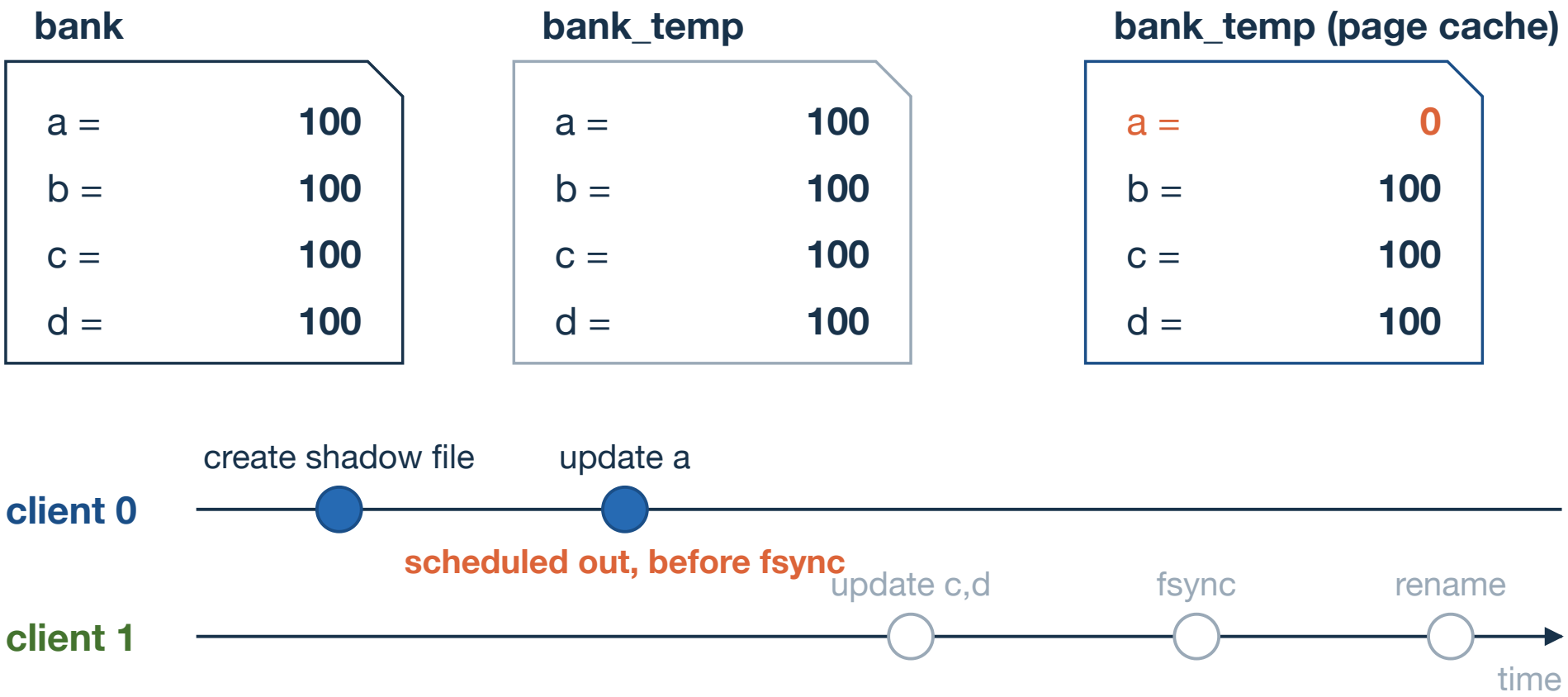
A case study of two operations using shadow copy

The two transfers touch disjoint accounts, but both must copy the same `bank` file



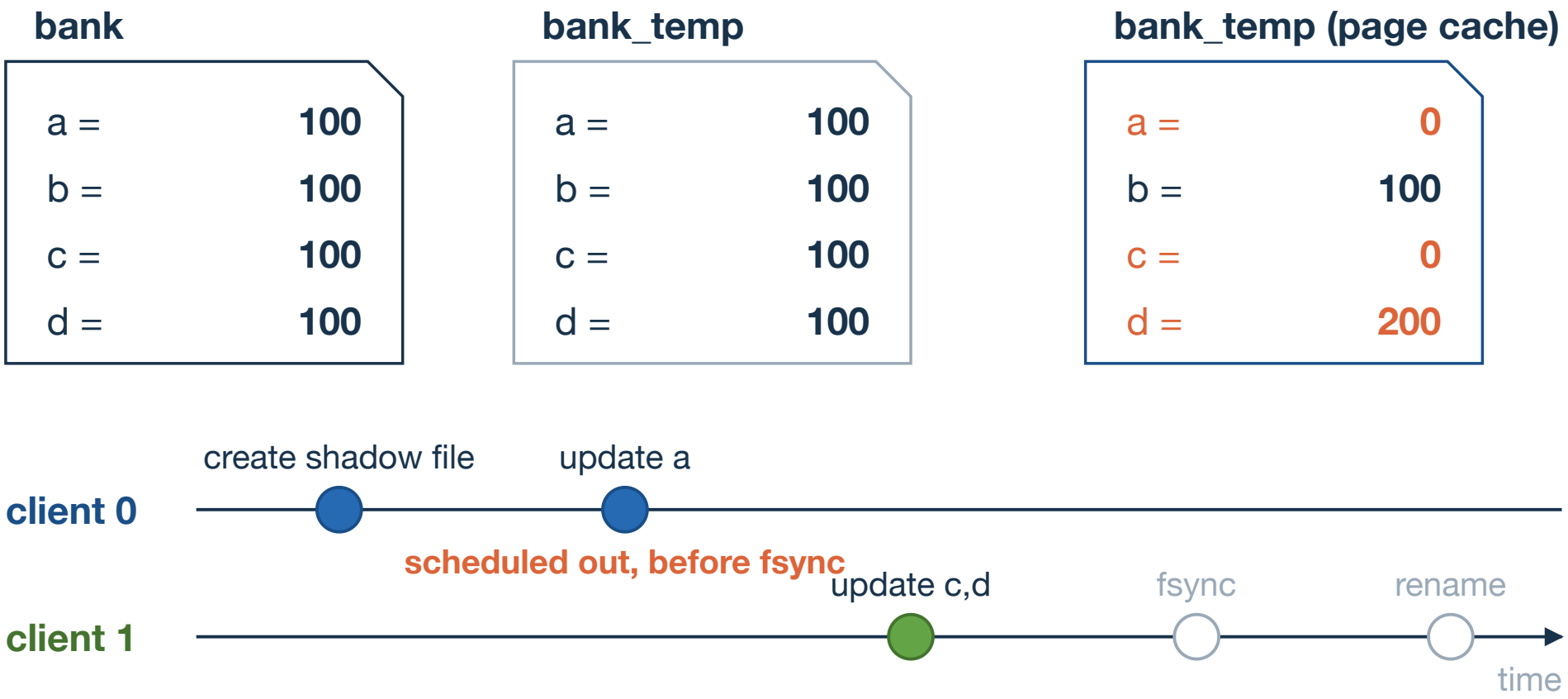
A case study of two operations using shadow copy

The two transfers touch disjoint accounts, but both must copy the same `bank` file



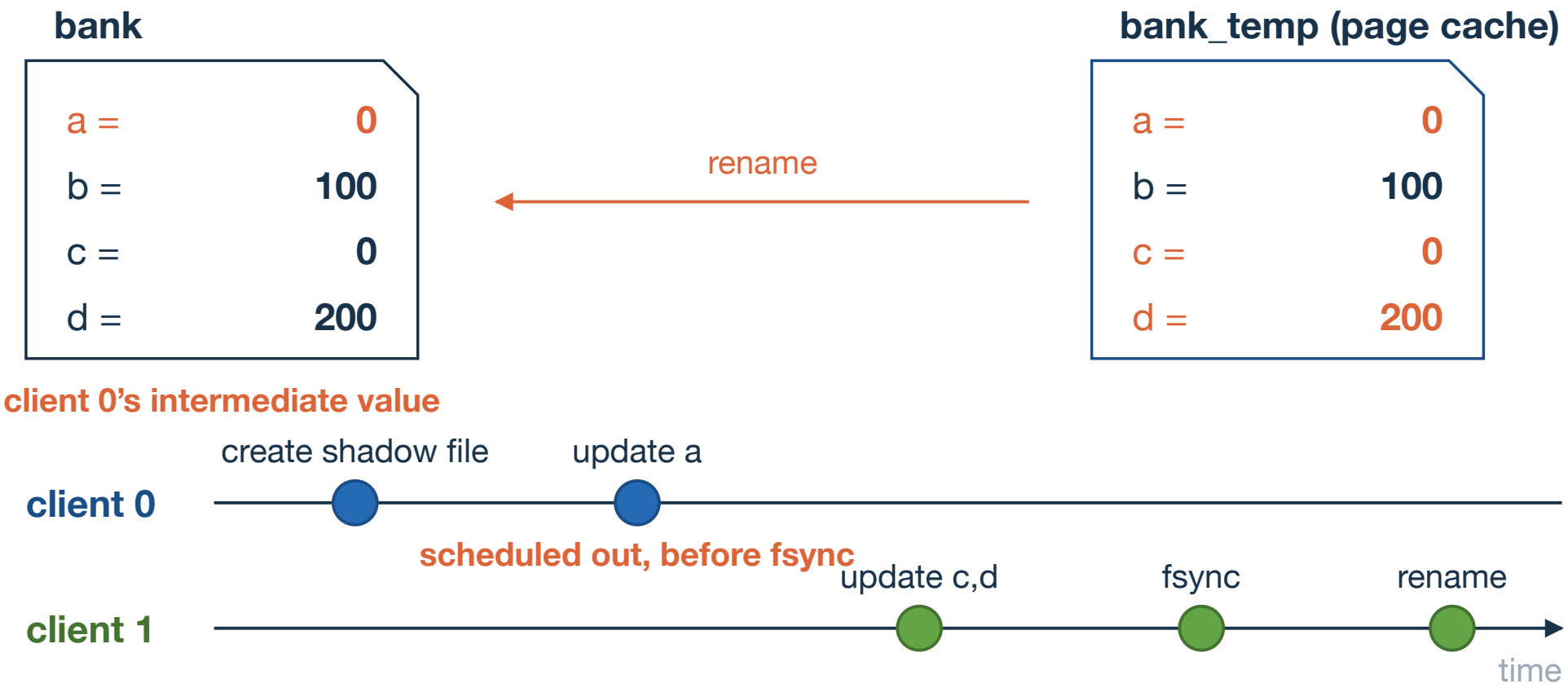
A case study of two operations using shadow copy

The two transfers touch disjoint accounts, but both must copy the same `bank` file



A case study of two operations using shadow copy

The two transfers touch disjoint accounts, but both must copy the same `bank` file



Three limitations of shadow copy

Cons #1: only one operation (e.g., transfer) can happen at a time, unless we add extra concurrency control

- Even transfers that could run in parallel

Cons #2: hard to generalize to multiple files or directories

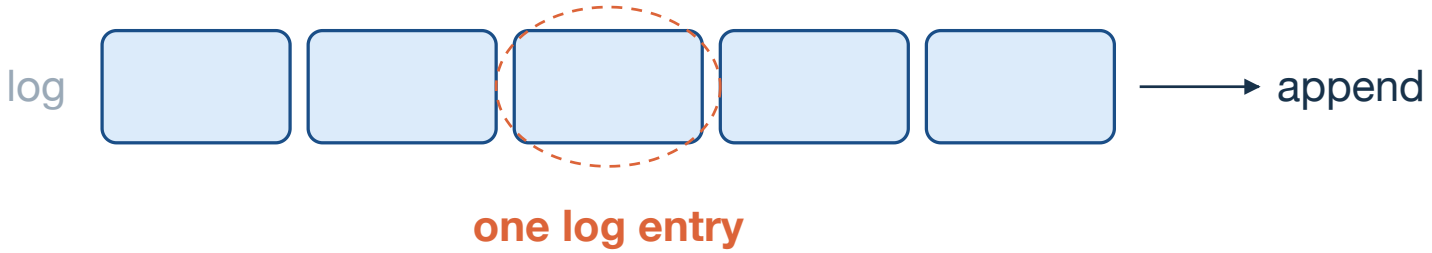
- Have to place all files in a single directory and rename the directory

Cons #3: any (small) change may require copying the entire file

Logging: generalizing journaling to any data modifications

Journaling and shadow copy follow the same principle: buffer the updates elsewhere, apply them to the home location once complete

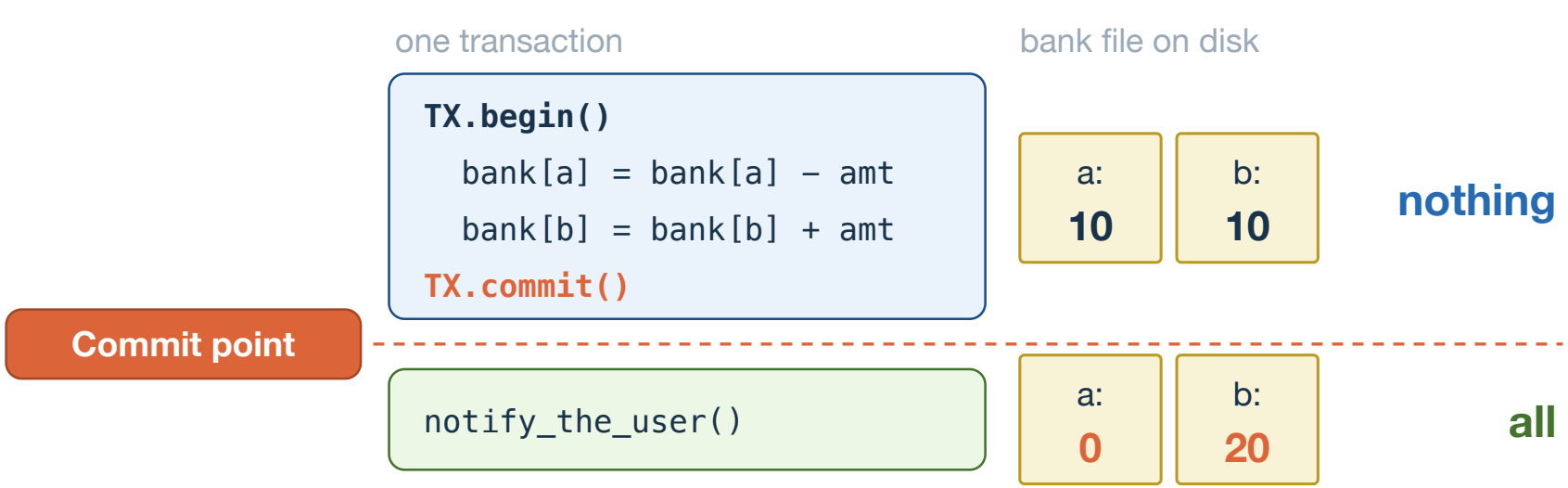
- shadow copy buffers the updates in a copy of the original file
- journaling buffers the updates to the file system in a log



Transaction and commit point

Before moving on, we need to identify the operations that must be executed as an atomic unit

- A set of operations that must be all-or-nothing is called a transaction
- Its boundary is typically marked by something like `TX.begin()` and `TX.commit()`



The commit point: from here the operations are guaranteed to be "all"

- Important in practice: e.g., we can send the application the ack after this point

[1] Terminology: Saltzer & Kaashoek, *Principles of Computer System Design*, §9.1.5

Redo-only logging #1: buffer the updates in memory, log at commit

Suppose we have a log file that stores all the application's updates

```
transfer(bank, a, b, amt, log):
    records = mmap(bank, ...)
    new_a = records[a] - amt
    new_b = records[b] + amt
    entry = "a: " + new_a + ", b: " + new_b
    log.append(entry).sync()
    records[a] = new_a
    records[b] = new_b
```

`new_a` / `new_b` are just local variables; nothing on disk has changed

- Purpose: no intermediate value ever gets a chance to reach the disk

Redo-only logging #2: sync the log to disk first

```
transfer(bank, a, b, amt, log):
    records = mmap(bank, ...)
    new_a = records[a] - amt
    new_b = records[b] + amt
    entry = "a: " + new_a + ", b: " + new_b
    log.append(entry).sync()
    records[a] = new_a
    records[b] = new_b
```

`log.append(entry).sync()`: once this line returns, the updates are already on disk

Question: what if the sync fails during the execution? (`sync` by itself is not atomic)

- This means that upon failure, the log entry may be incomplete

Question: how do we know a log entry is complete?

```
transfer(bank, a, b, amt, log):
    records = mmap(bank, ...)
    new_a = records[a] - amt
    new_b = records[b] + amt
    entry = "a: " + new_a + ", b: " + new_b
    log.append(entry).sync()
    records[a] = new_a
    records[b] = new_b
```

Solution: use a checksum

- Write the entry together with its checksum
- On recovery, recompute it: a mismatch means the entry is incomplete — so the transaction is nothing

Redo-only logging #3: then update the data on disk

```
transfer(bank, a, b, amt, log):
    records = mmap(bank, ...)
    new_a = records[a] - amt
    new_b = records[b] + amt
    entry = "a: " + new_a + ", b: " + new_b
    log.append(entry).sync()
    records[a] = new_a
    records[b] = new_b
```

Only after the log reaches the disk do we write the new values back to `records`

Log first, data second

Question: do we need an fsync on the bank file?

```
transfer(bank, a, b, amt, log):
    records = mmap(bank, ...)
    new_a = records[a] - amt
    new_b = records[b] + amt
    entry = "a: " + new_a + ", b: " + new_b
    log.append(entry).sync()
    records[a] = new_a
    records[b] = new_b
    fsync(bank)   # needed?
```

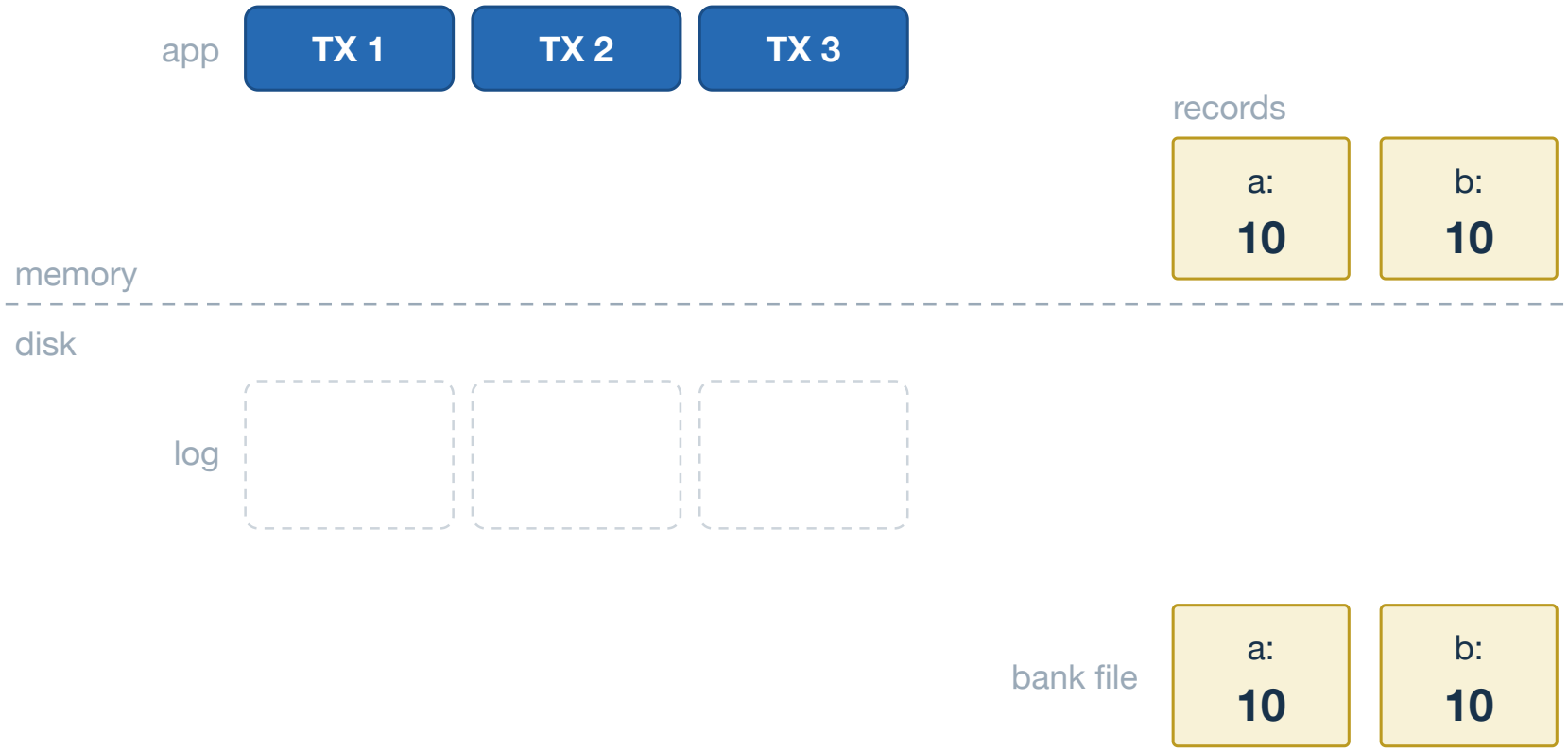
Not necessary

- Even if the changes to `records` are still in the page cache, not yet on disk
- replaying the log after reboot fills them in

Crash recovery: replay the log from the start

After reboot, use the log to repair the system into a consistent state

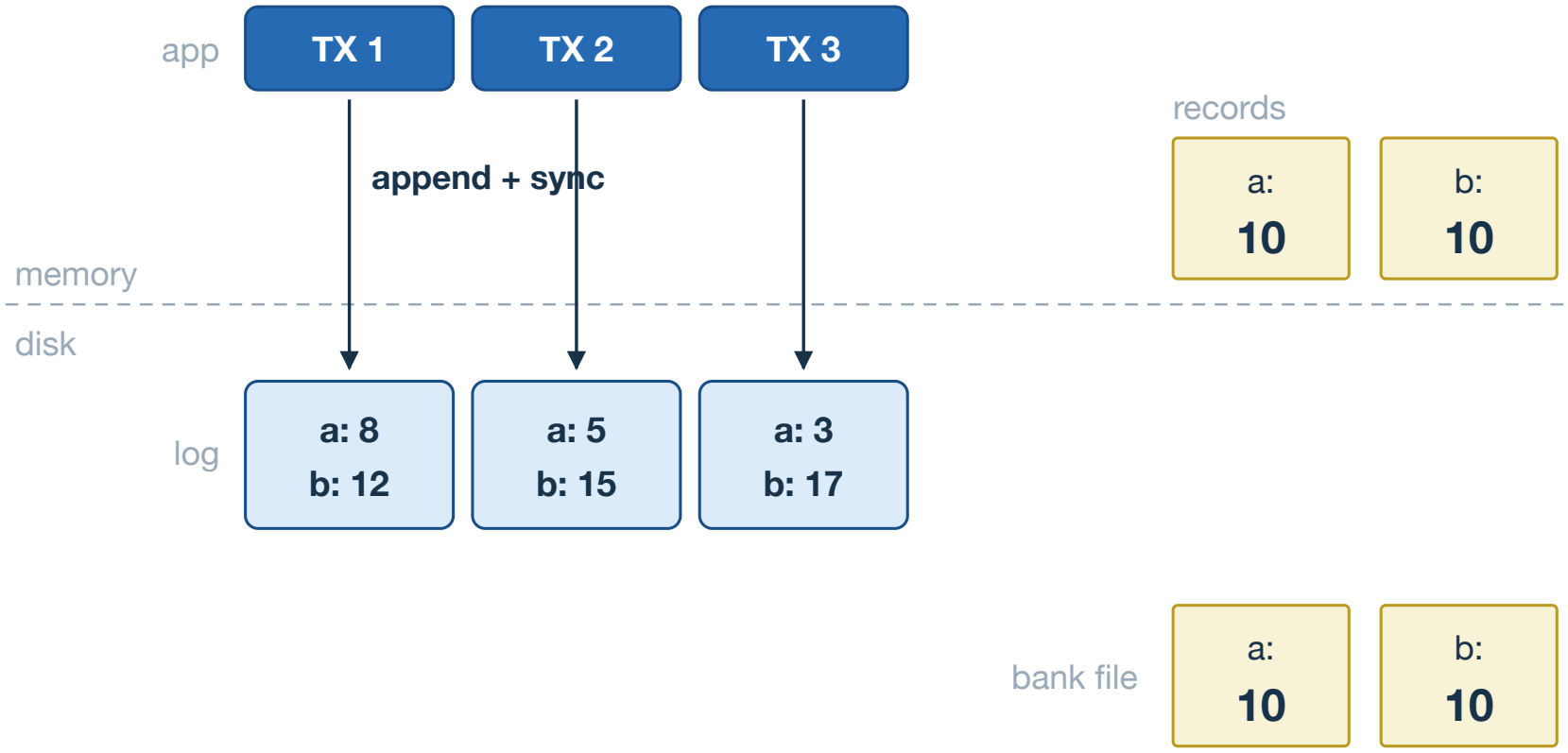
- Travel from start to end, re-apply every complete log entry; skip the one the crash cut in half



Crash recovery: replay the log from the start

After reboot, use the log to repair the system into a consistent state

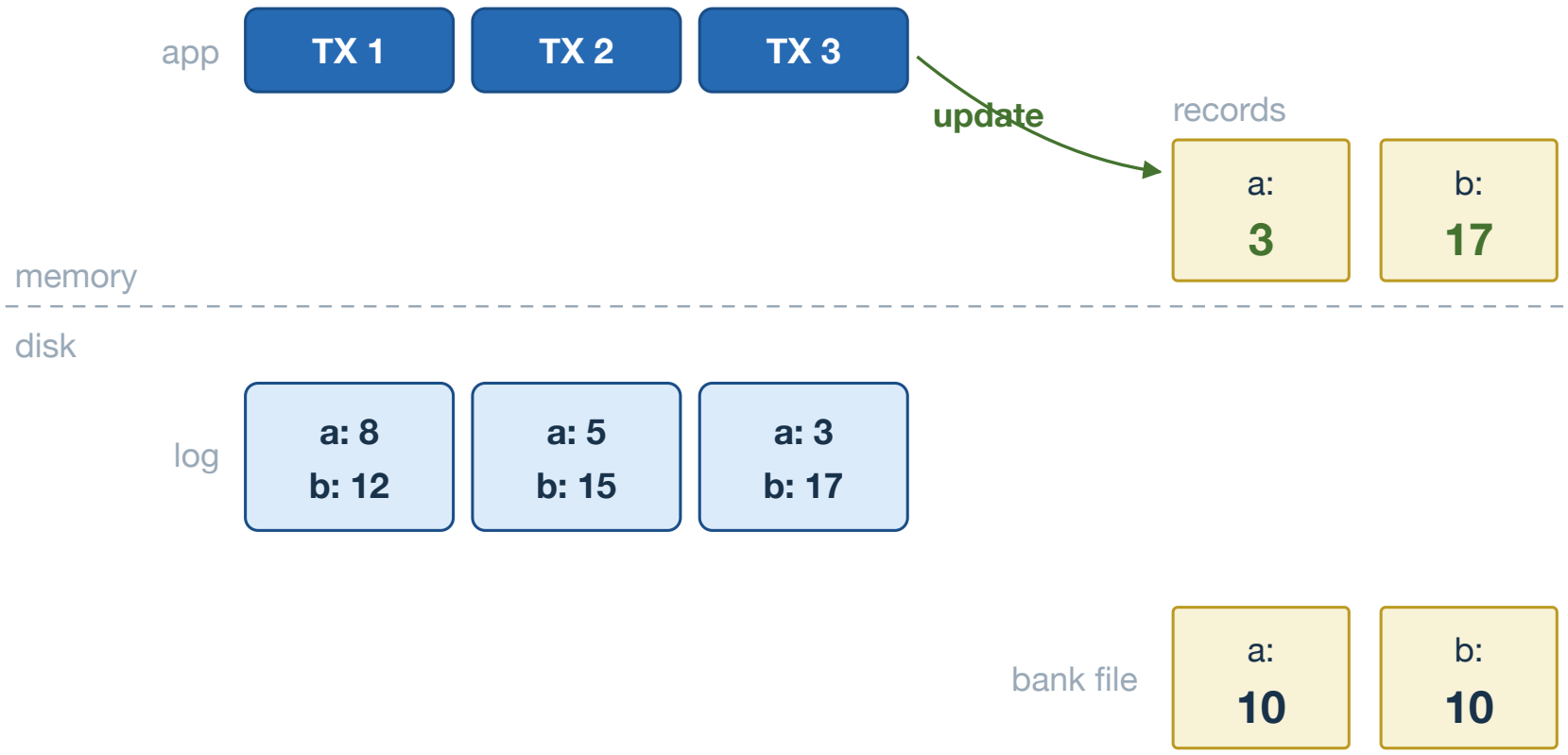
- Travel from start to end, re-apply every complete log entry; skip the one the crash cut in half



Crash recovery: replay the log from the start

After reboot, use the log to repair the system into a consistent state

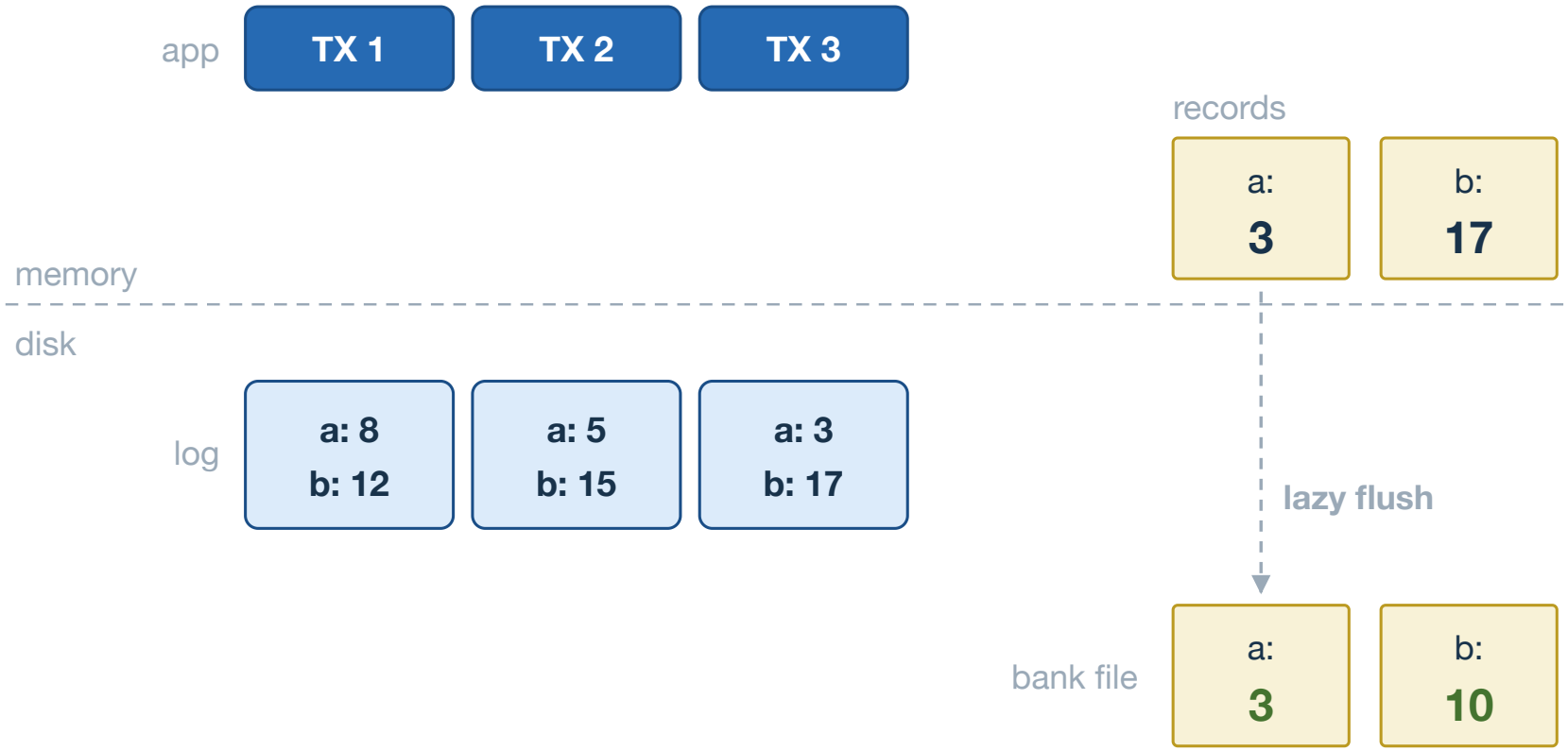
- Travel from start to end, re-apply every complete log entry; skip the one the crash cut in half



Crash recovery: replay the log from the start

After reboot, use the log to repair the system into a consistent state

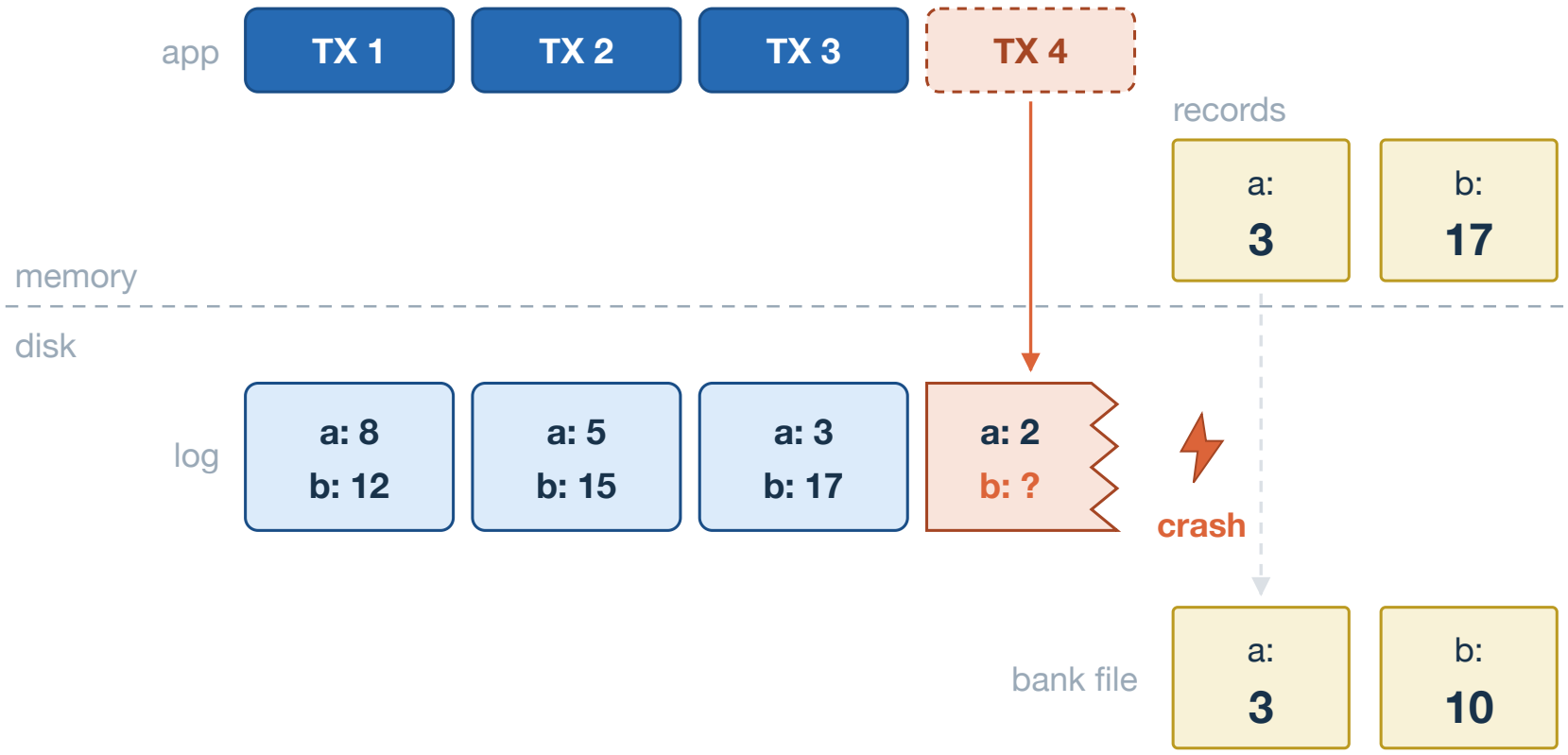
- Travel from start to end, re-apply every complete log entry; skip the one the crash cut in half



Crash recovery: replay the log from the start

After reboot, use the log to repair the system into a consistent state

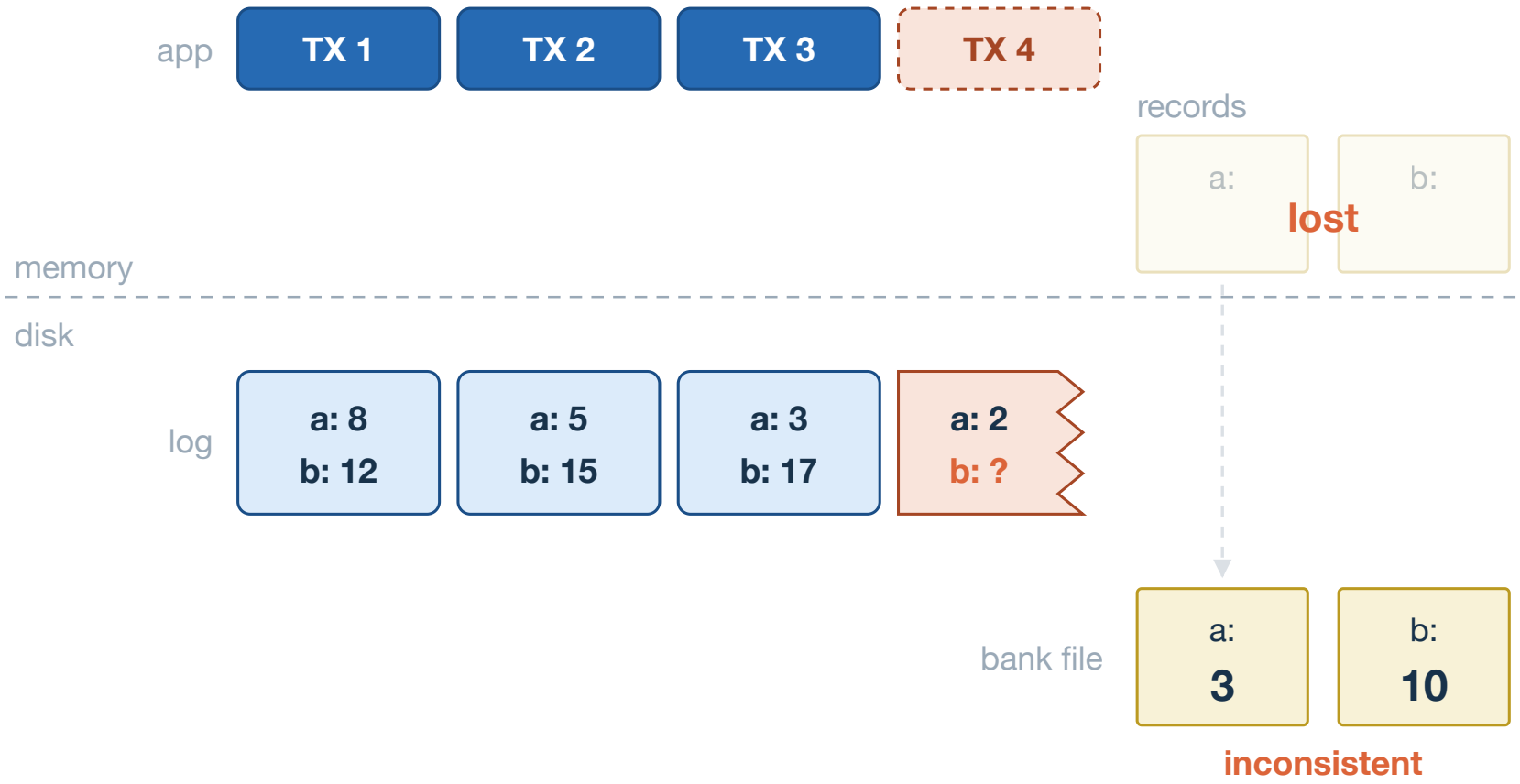
- Travel from start to end, re-apply every complete log entry; skip the one the crash cut in half



Crash recovery: replay the log from the start

After reboot, use the log to repair the system into a consistent state

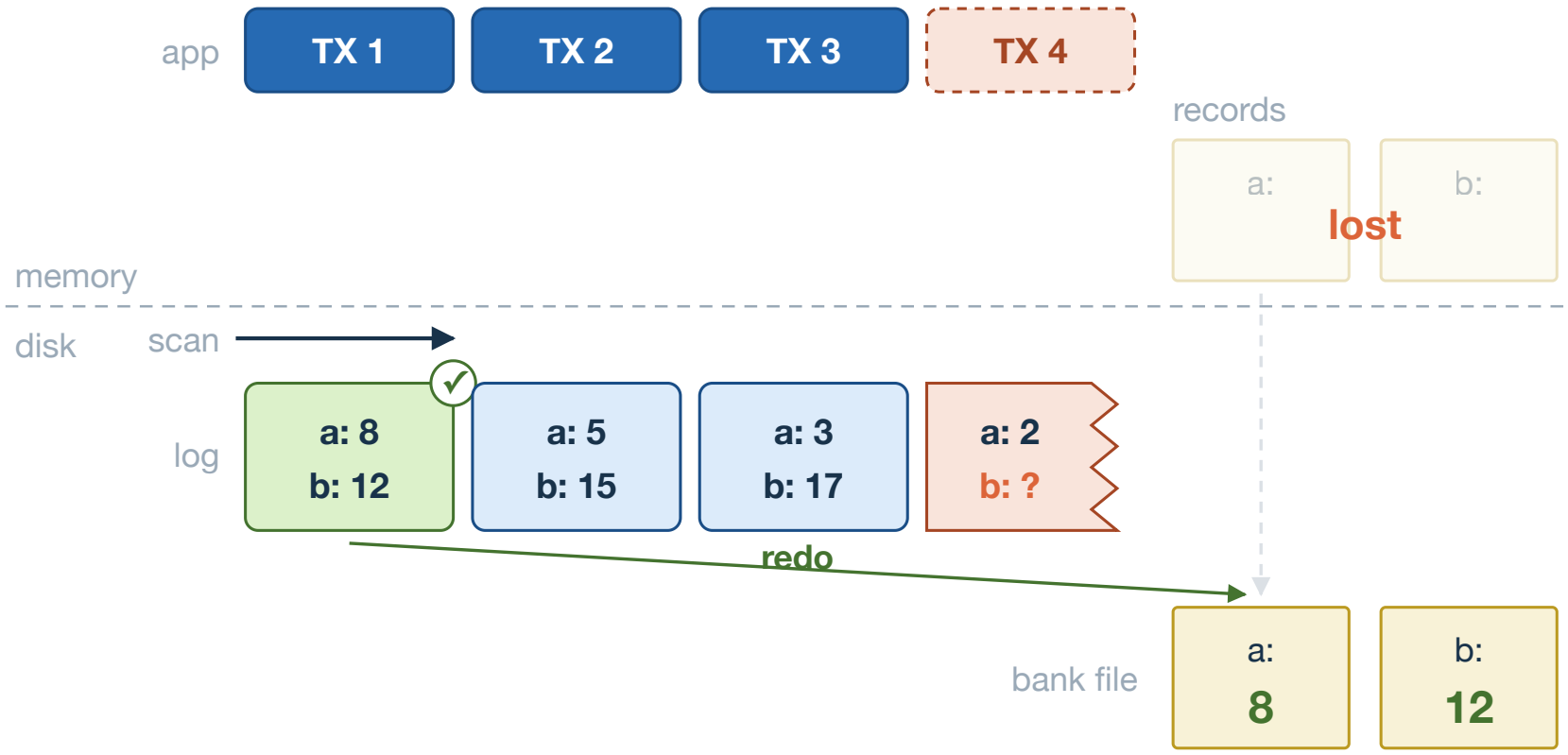
- Travel from start to end, re-apply every complete log entry; skip the one the crash cut in half



Crash recovery: replay the log from the start

After reboot, use the log to repair the system into a consistent state

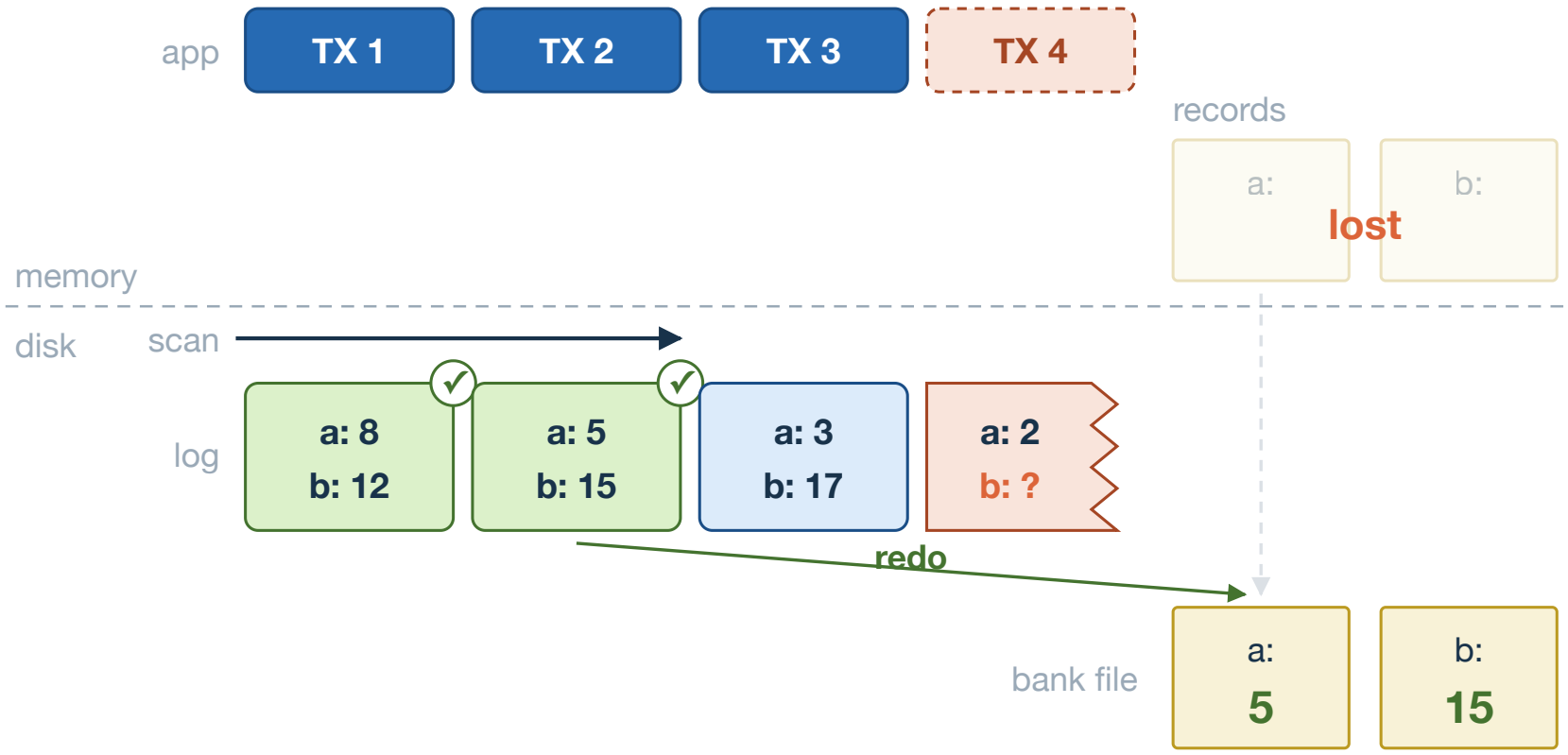
- Travel from start to end, re-apply every complete log entry; skip the one the crash cut in half



Crash recovery: replay the log from the start

After reboot, use the log to repair the system into a consistent state

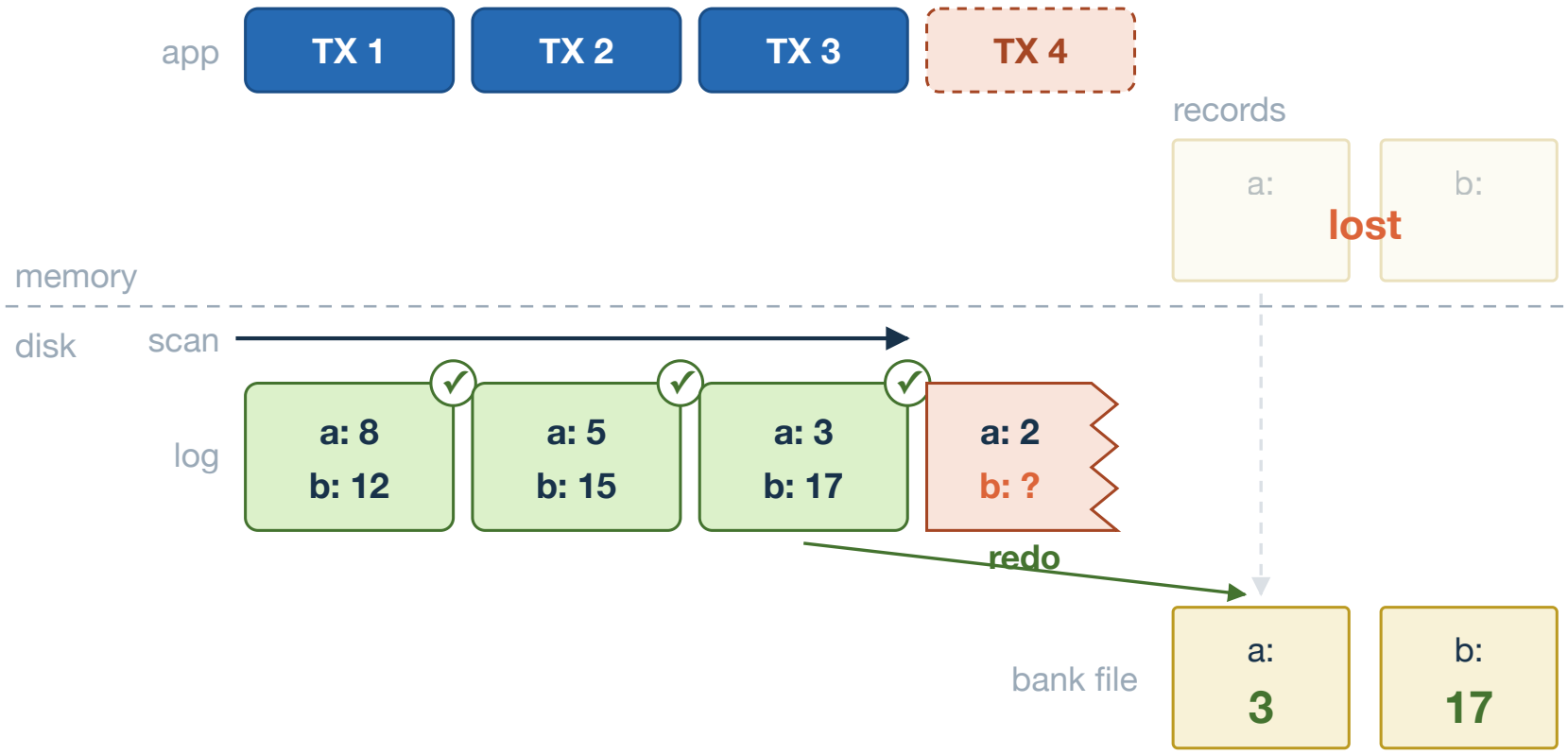
- Travel from start to end, re-apply every complete log entry; skip the one the crash cut in half



Crash recovery: replay the log from the start

After reboot, use the log to repair the system into a consistent state

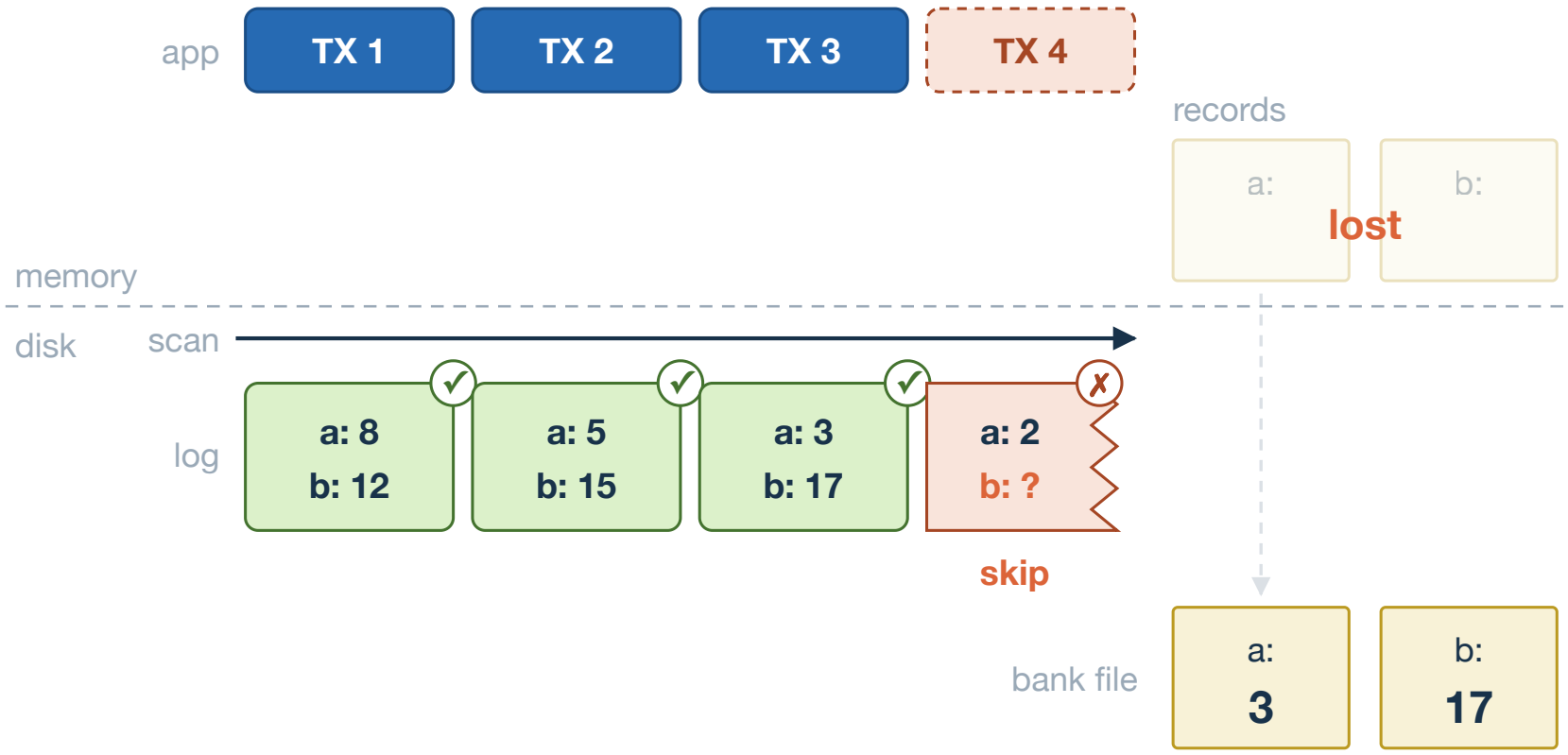
- Travel from start to end, re-apply every complete log entry; skip the one the crash cut in half



Crash recovery: replay the log from the start

After reboot, use the log to repair the system into a consistent state

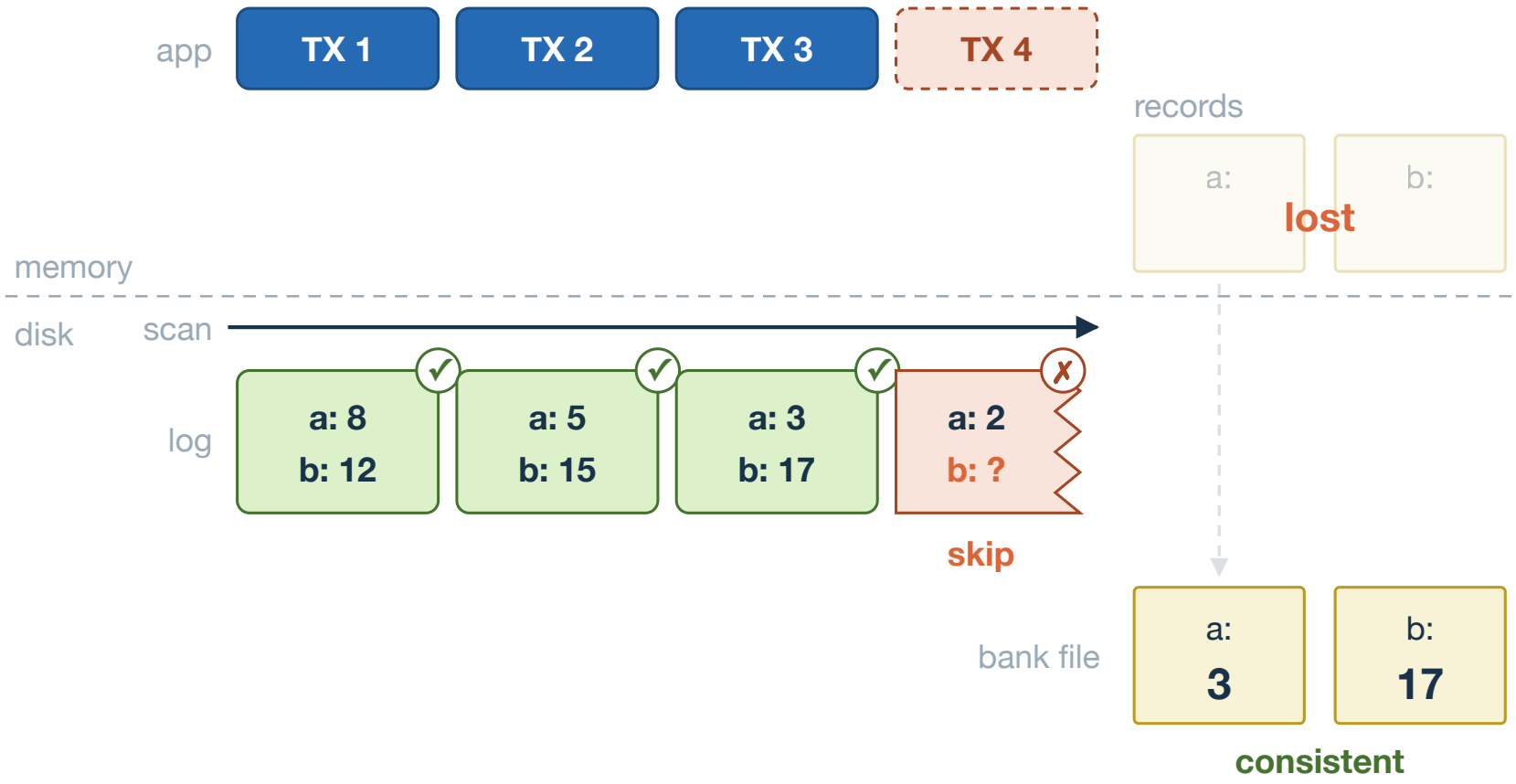
- Travel from start to end, re-apply every complete log entry; skip the one the crash cut in half



Crash recovery: replay the log from the start

After reboot, use the log to repair the system into a consistent state

- Travel from start to end, re-apply every complete log entry; skip the one the crash cut in half



Question: where is the commit point of this transaction?

```
transfer(bank, a, b, amt, log):
    records = mmap(bank, ...)
    new_a = records[a] - amt
    new_b = records[b] + amt
    entry = "a: " + new_a + ", b: " + new_b
    log.append(entry).sync()
    records[a] = new_a
    records[b] = new_b
```

The line right after `log.append(entry).sync()` returns

- Crash before it: the log is incomplete, recovery skips it — nothing happened
- Crash after it: the log is complete, recovery replays it — it will finish

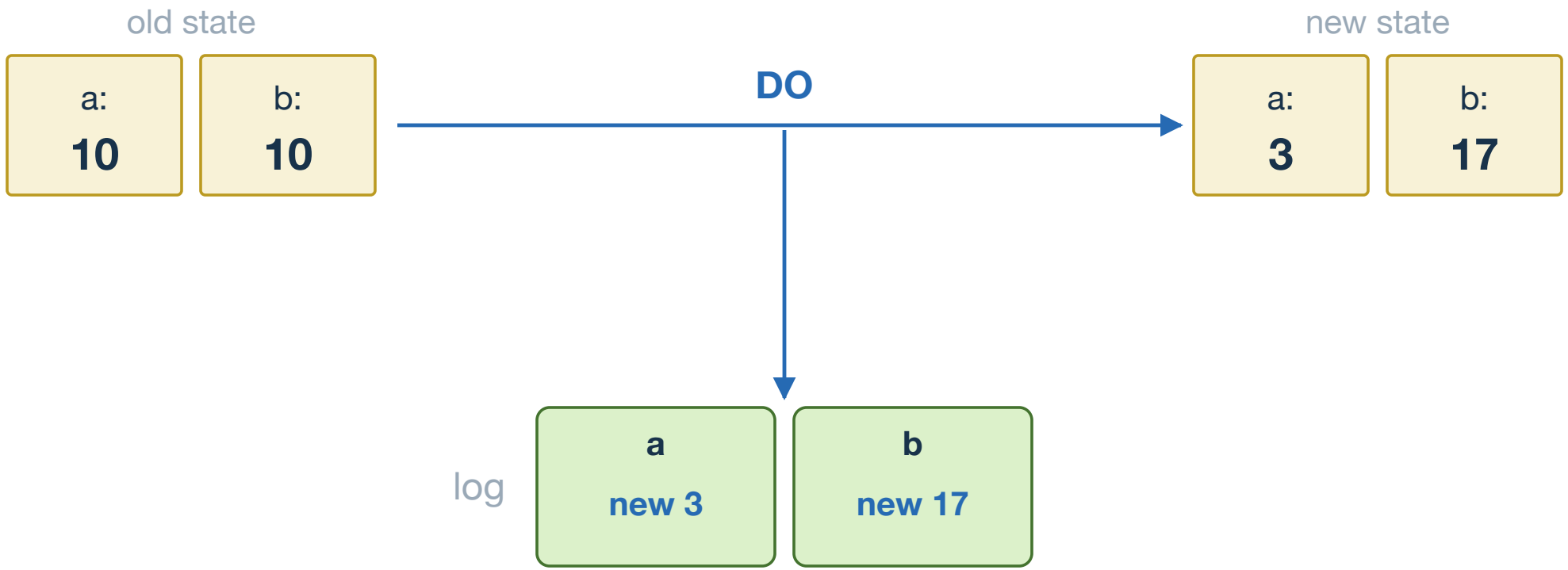
Quick summary: redo-only logging

The log records only new values, so recovery has a single move: redo



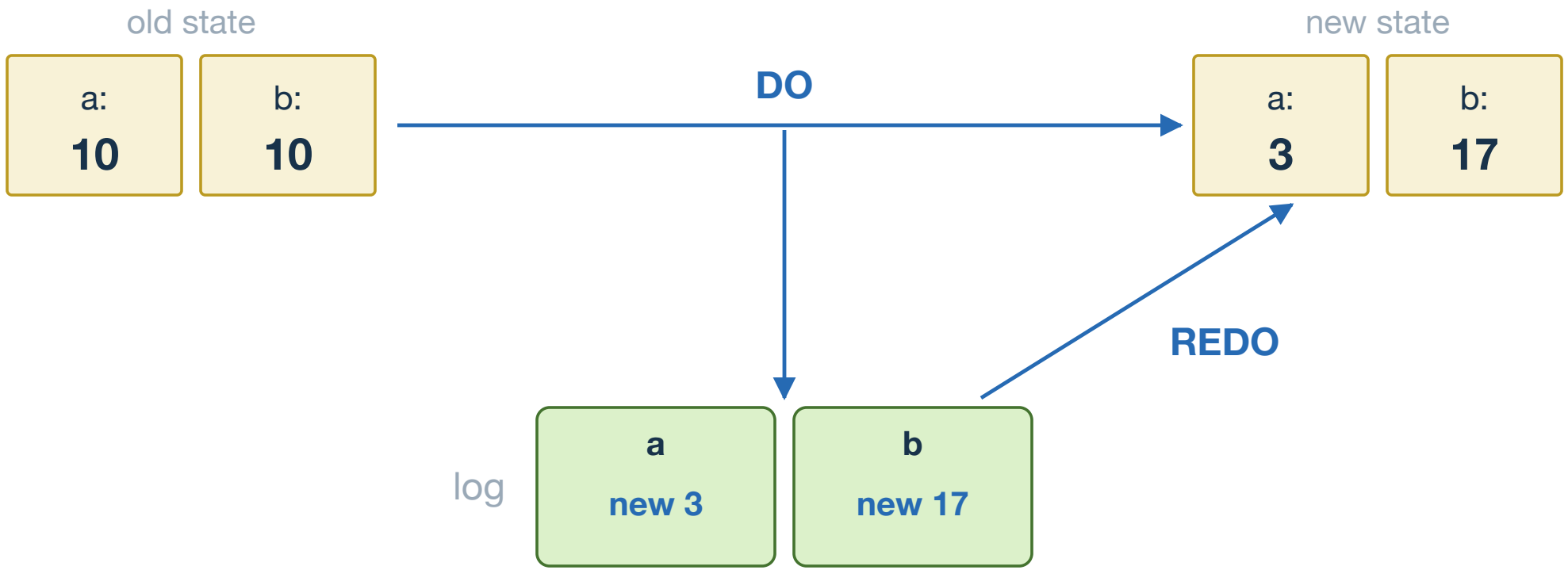
Quick summary: redo-only logging

The log records only new values, so recovery has a single move: redo



Quick summary: redo-only logging

The log records only new values, so recovery has a single move: redo



(Aside) Value log vs. operator log

So far the log stores values; it could store the operation instead

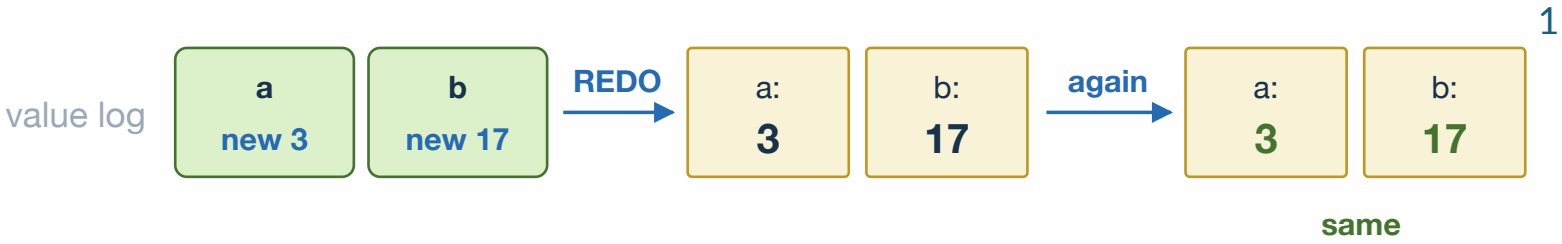
[1]



[1] Gray, *Notes on Data Base Operating Systems*, 1978: "an operation may be undone or redone more than once … This requires that the redo and undo operators be repeatable (idempotent)" (pdf)

(Aside) Value log vs. operator log

So far the log stores values; it could store the operation instead

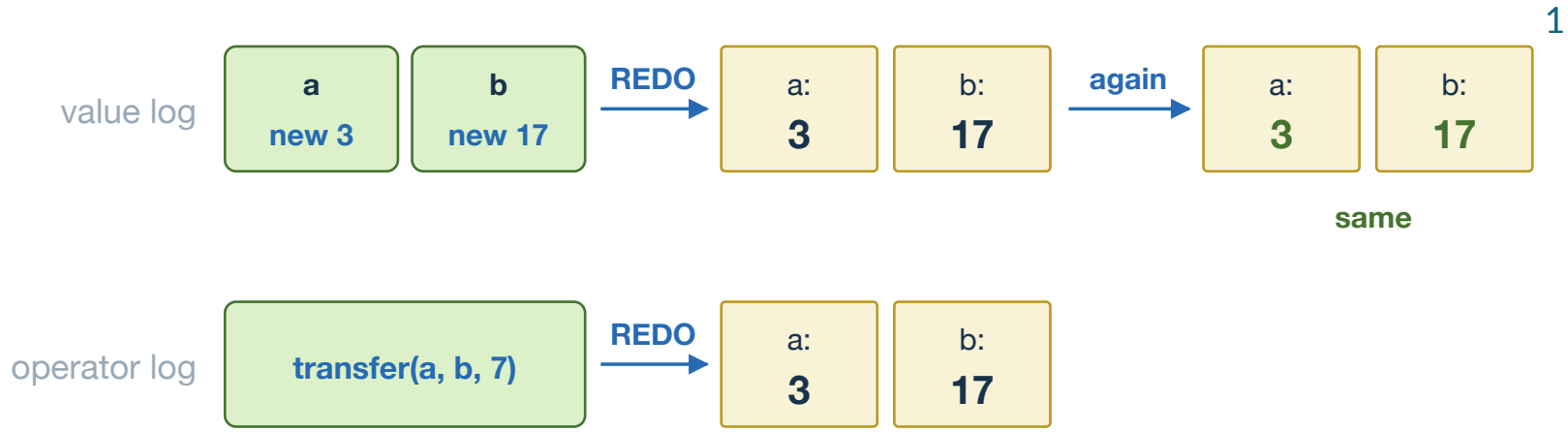


[1]

[1] Gray, *Notes on Data Base Operating Systems*, 1978: "an operation may be undone or redone more than once ... This requires that the redo and undo operators be repeatable (idempotent)" (pdf)

(Aside) Value log vs. operator log

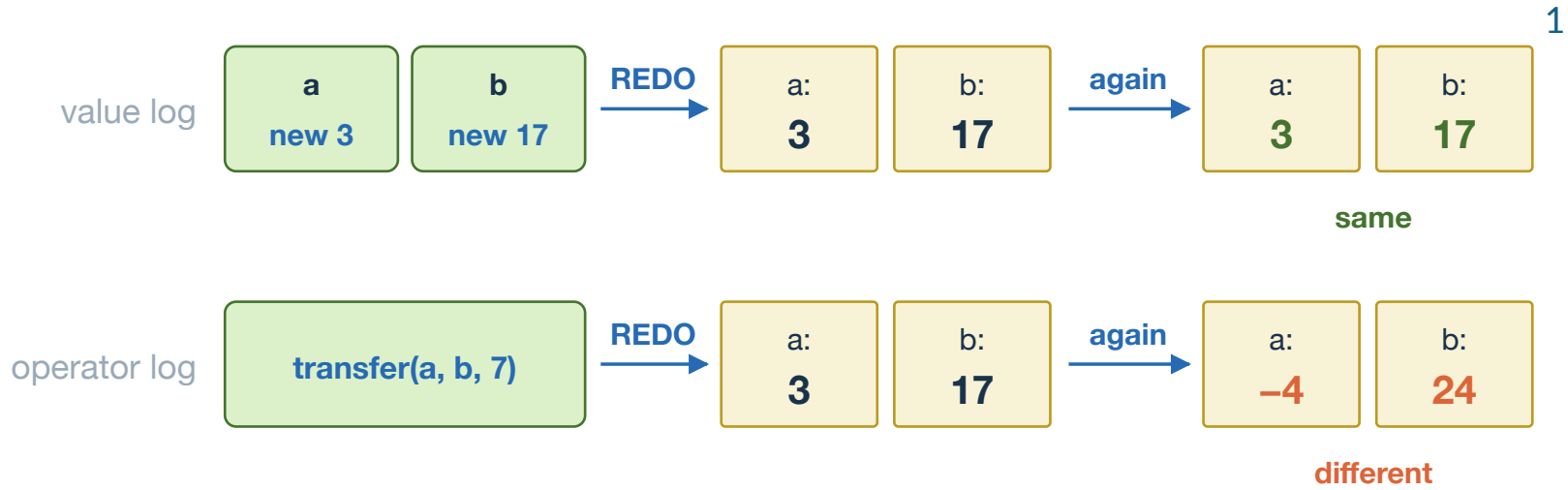
So far the log stores values; it could store the operation instead



[1] Gray, *Notes on Data Base Operating Systems*, 1978: "an operation may be undone or redone more than once ... This requires that the redo and undo operators be repeatable (idempotent)" (pdf)

(Aside) Value log vs. operator log

So far the log stores values; it could store the operation instead



[1]

Question: recovery replays from the start, and may itself crash midway. Is a still 3?

- Value redo is idempotent; operator redo must run exactly once
- MySQL replicates `UPDATE t SET id = UUID()` as rows, not as the statement[2]

The price: a value log is larger — ext3 journals the whole block even if one bit changed[3]

[1] Gray, *Notes on Data Base Operating Systems*, 1978: "an operation may be undone or redone more than once … This requires that the redo and undo operators be repeatable (idempotent)" (pdf)
[2] MySQL 8.4 manual, *Safe and Unsafe Statements in Binary Logging*: `RAND()`, `UUID()`, `SYSDATE()` … "may return a different value on the replica"; row-based logging is the default (link)
[3] Tweedie, *Journaling the Linux ext2fs Filesystem*, 1998: "write out entire blocks of metadata to the journal even if only a single bit in the block has been modified" (pdf)

Pros & cons of redo-only logging

Pro: the commit is extremely fast

- One transaction takes one sequential append, which is fast on nearly every storage device

Con: before the commit, all updates must be buffered in memory

- What if we have insufficient memory?

Con: the log keeps growing, while most of its updates are already flushed to the disk

Coping with insufficient memory: write uncommitted updates to disk

```
transfer(bank, a, b, amt, log):
    records = mmap(bank, ...)
    records[a] = records[a] - amt
    records[b] = records[b] + amt
```

Write `records` directly, and let the OS flush pages whenever it wants

- If there is insufficient memory, the OS can flush them to the disk

New problem: after a crash, the disk may hold half of an uncommitted transaction

Solution: the log can redo to a consistent state, and it can also undo an uncommitted state!

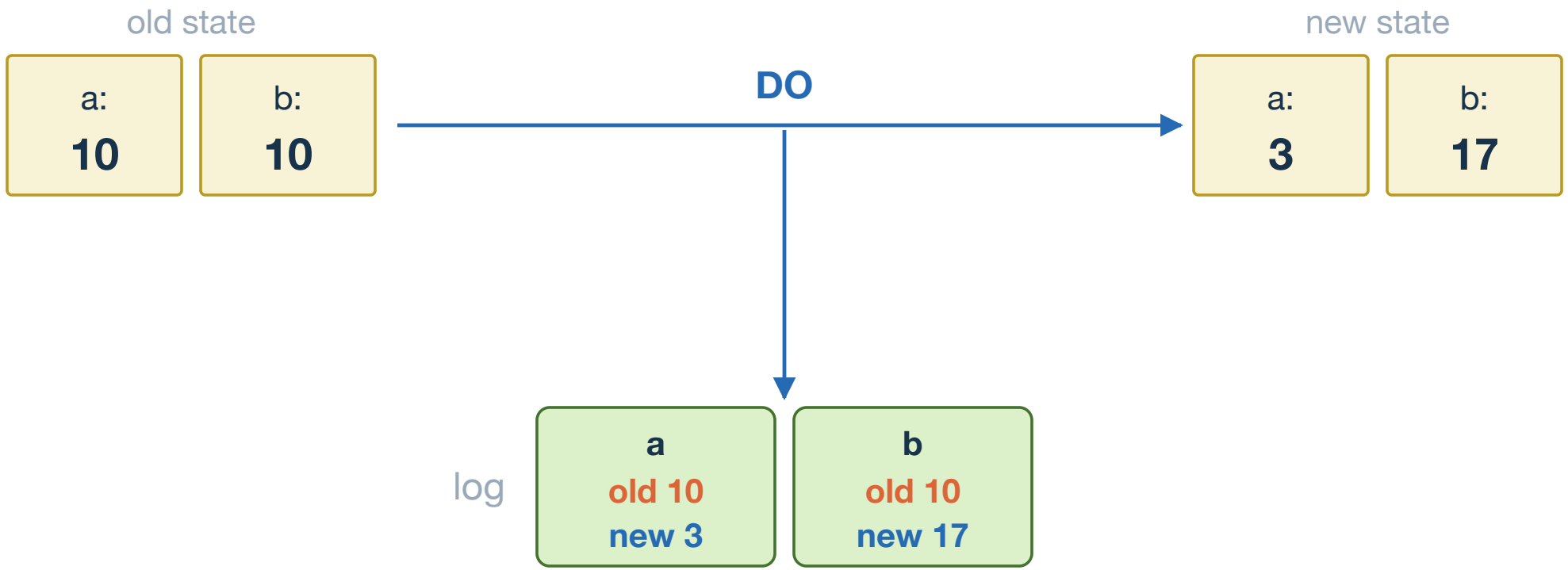
Undo-redo logging: the log can also roll back

Record old values in the log too, and recovery gains a second move: undo



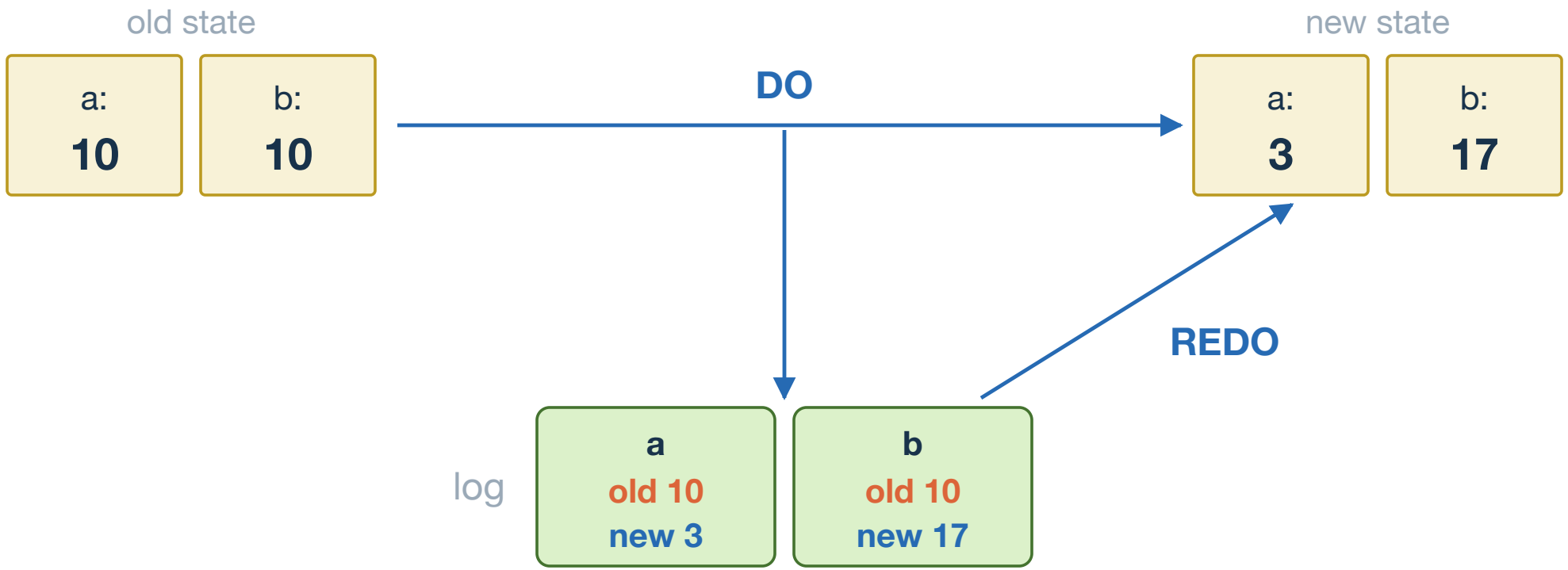
Undo-redo logging: the log can also roll back

Record old values in the log too, and recovery gains a second move: undo



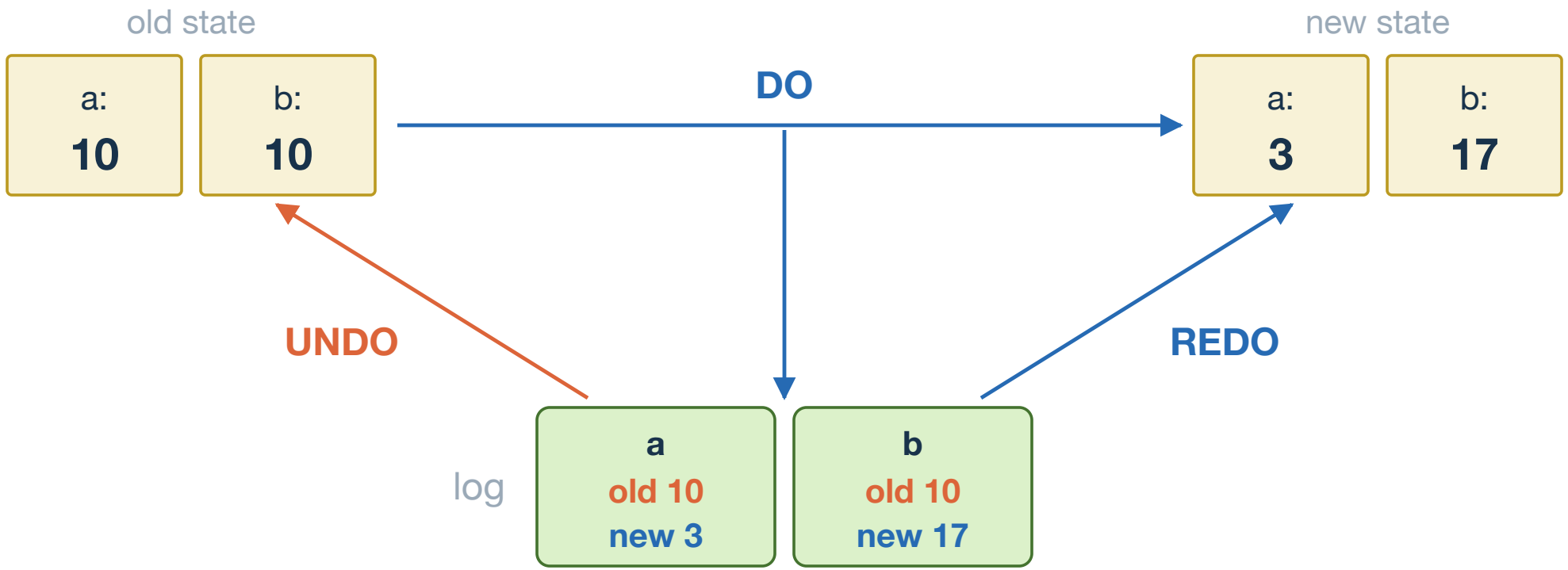
Undo-redo logging: the log can also roll back

Record old values in the log too, and recovery gains a second move: undo



Undo-redo logging: the log can also roll back

Record old values in the log too, and recovery gains a second move: undo



Implementation: write an undo record before each update

```
transfer(bank, a, b, amt, log):
    records = mmap(bank, ...)
    log.append(<a, old: records[a]>).sync()
    records[a] = records[a] - amt
    log.append(<b, old: records[b]>).sync()
    records[b] = records[b] + amt
```

Before each update, record “which file, which offset, what old value” into the log and sync it

Question: do we still need the new values (redo)?

Undo-redo logging: record redo entries

```
transfer(bank, a, b, amt, log):
    records = mmap(bank, ...)
    log.append(<a, old: records[a], new: records[a] - amt>).sync()
    records[a] = records[a] - amt
    log.append(<b, old: records[b], new: records[b] + amt>).sync()
    records[b] = records[b] + amt
    log.append(<TX 1 commit>).sync()
```

Whether to add redo entries depends on whether we are willing to wait for the data to reach the disk before commit

Typically, yes

- With old values only, the data must be fsync'ed before commit — waiting for two disk syncs is slow (random accesses)
- The log is a sequential write, which is fast

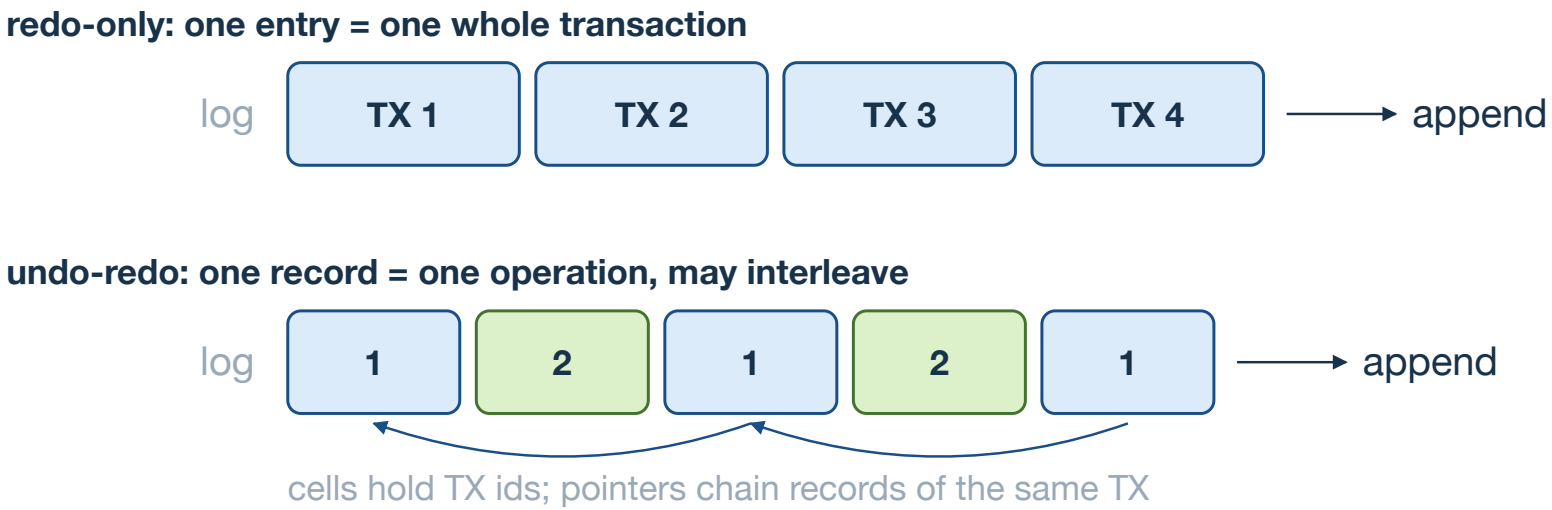
Recap: log entry vs. log record in redo-only vs. undo-redo logging

Redo-only appends log entries

- One entry holds all the updates of a whole transaction

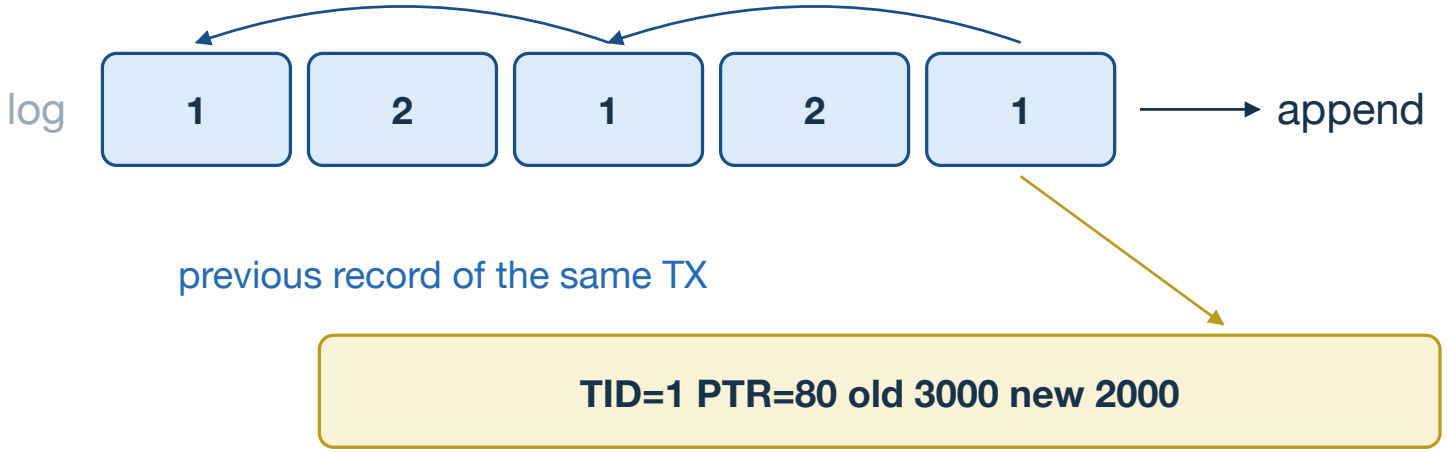
Undo-redo appends log records

- One record holds a single operation; records from different transactions may interleave



Inside a log record

- `Transaction ID`: which transaction this record belongs to
- `Pointer` to the previous record of the same transaction — chaining them into a list
- `The record itself`: file name, offset, old value, new value



Two logging rules

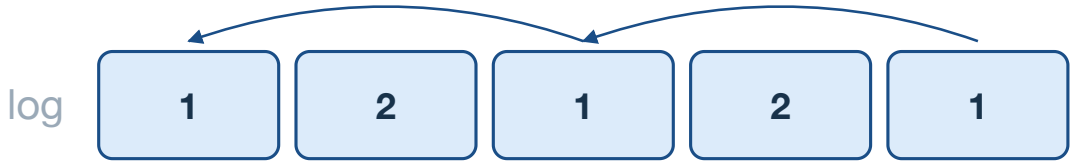
Rule #1: write the log record to disk before modifying the persistent state

- This rule is called write-ahead logging (WAL)

Rule #2: at the commit point, append a commit record to the log



log on disk

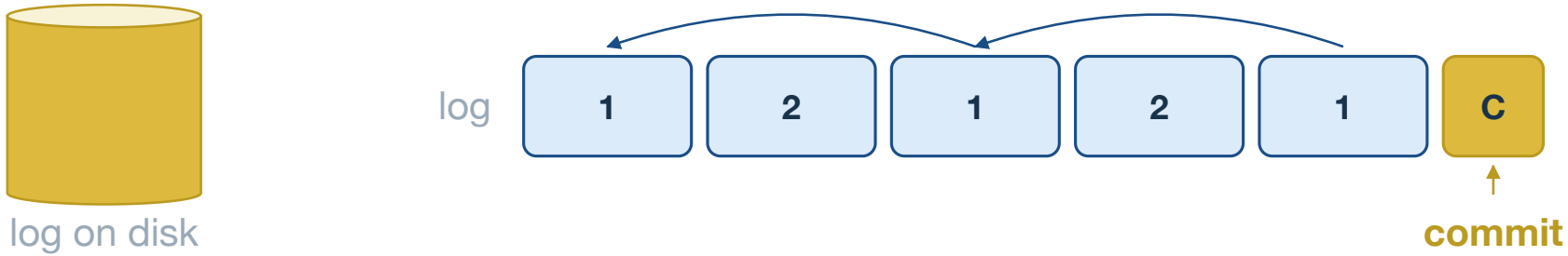


Two logging rules

Rule #1: write the log record to disk before modifying the persistent state

- This rule is called write-ahead logging (WAL)

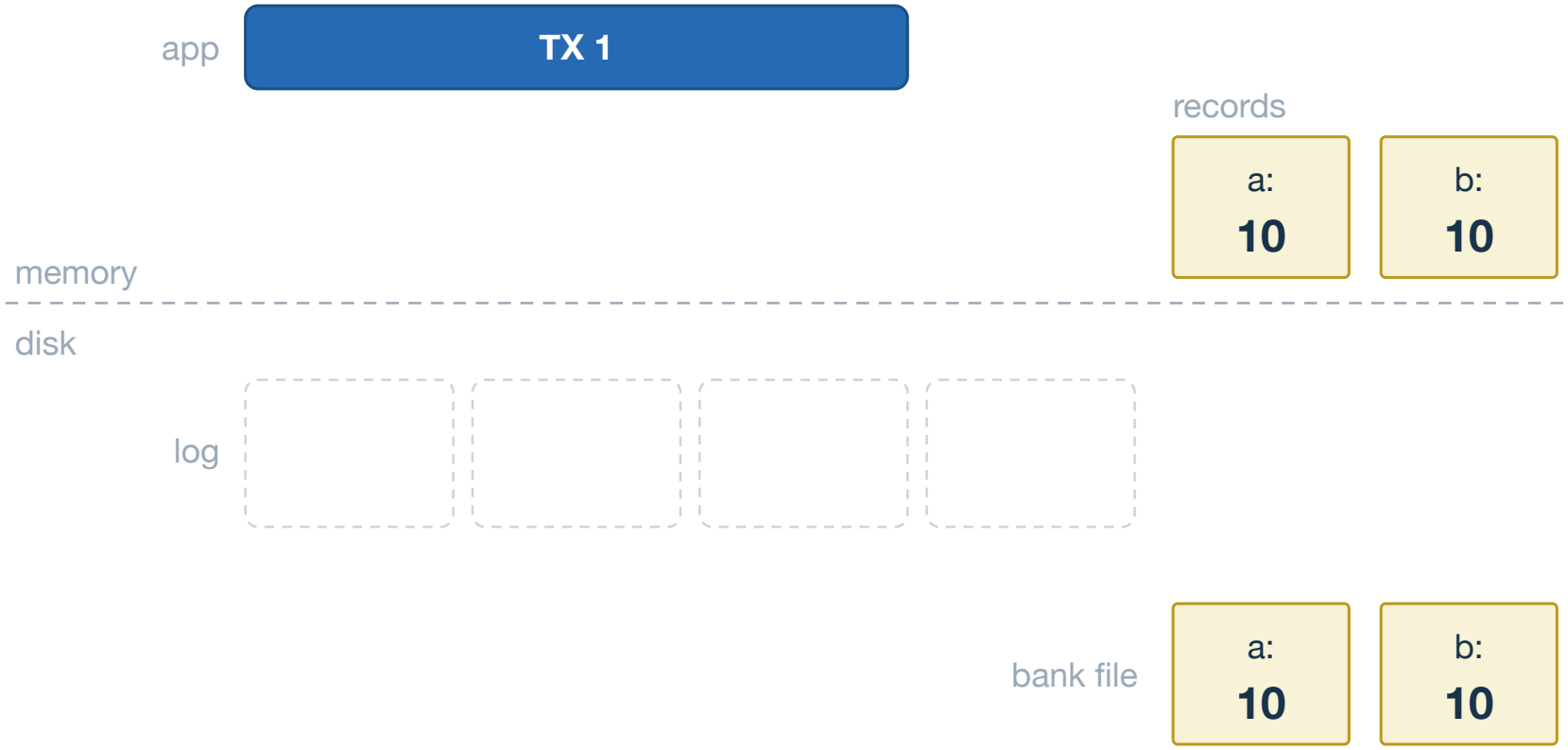
Rule #2: at the commit point, append a commit record to the log



Recovery rules: walk the log backwards

Scan from end to start: a transaction with no commit record is aborted

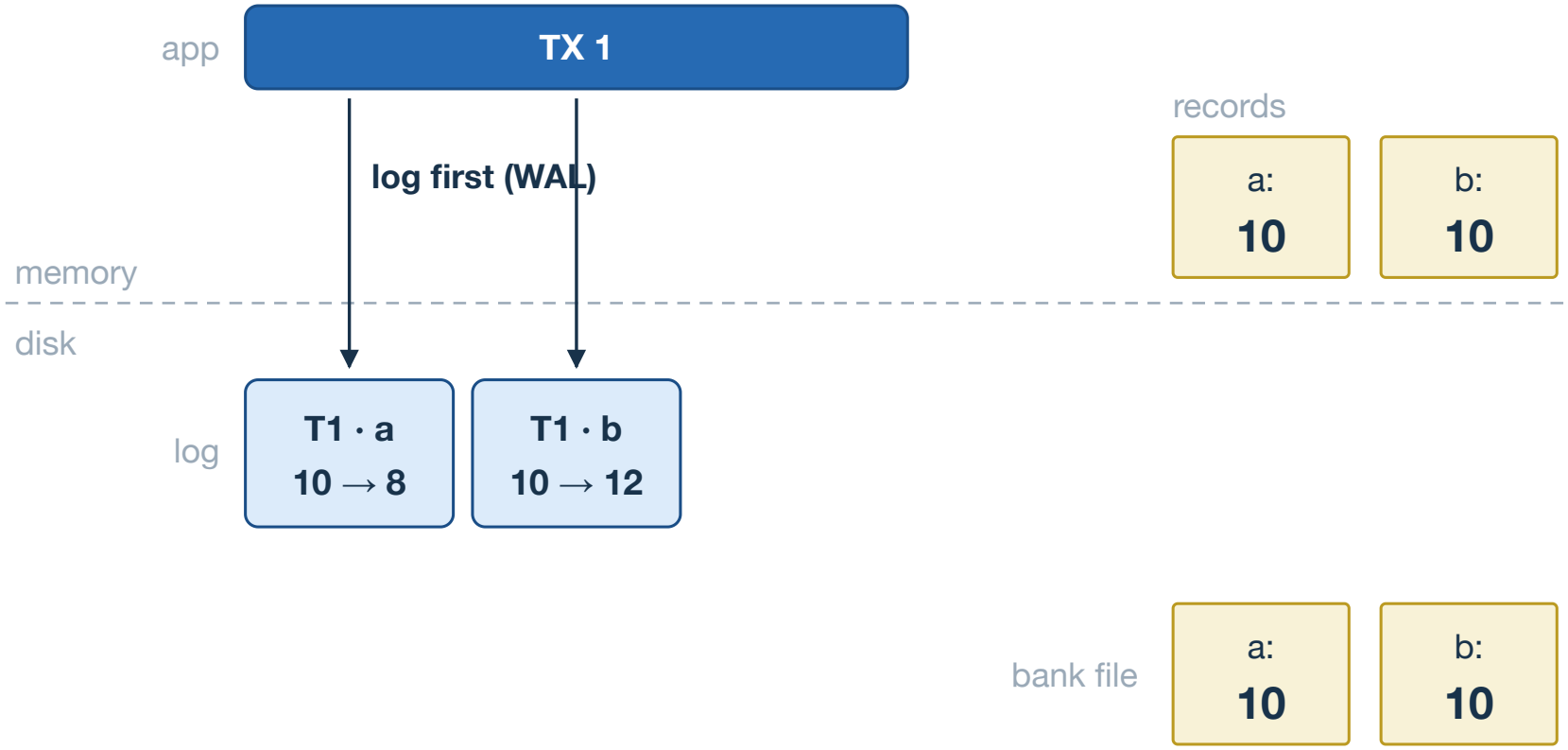
Undo the aborted records end to start, then redo the committed ones start to end



Recovery rules: walk the log backwards

Scan from end to start: a transaction with no commit record is aborted

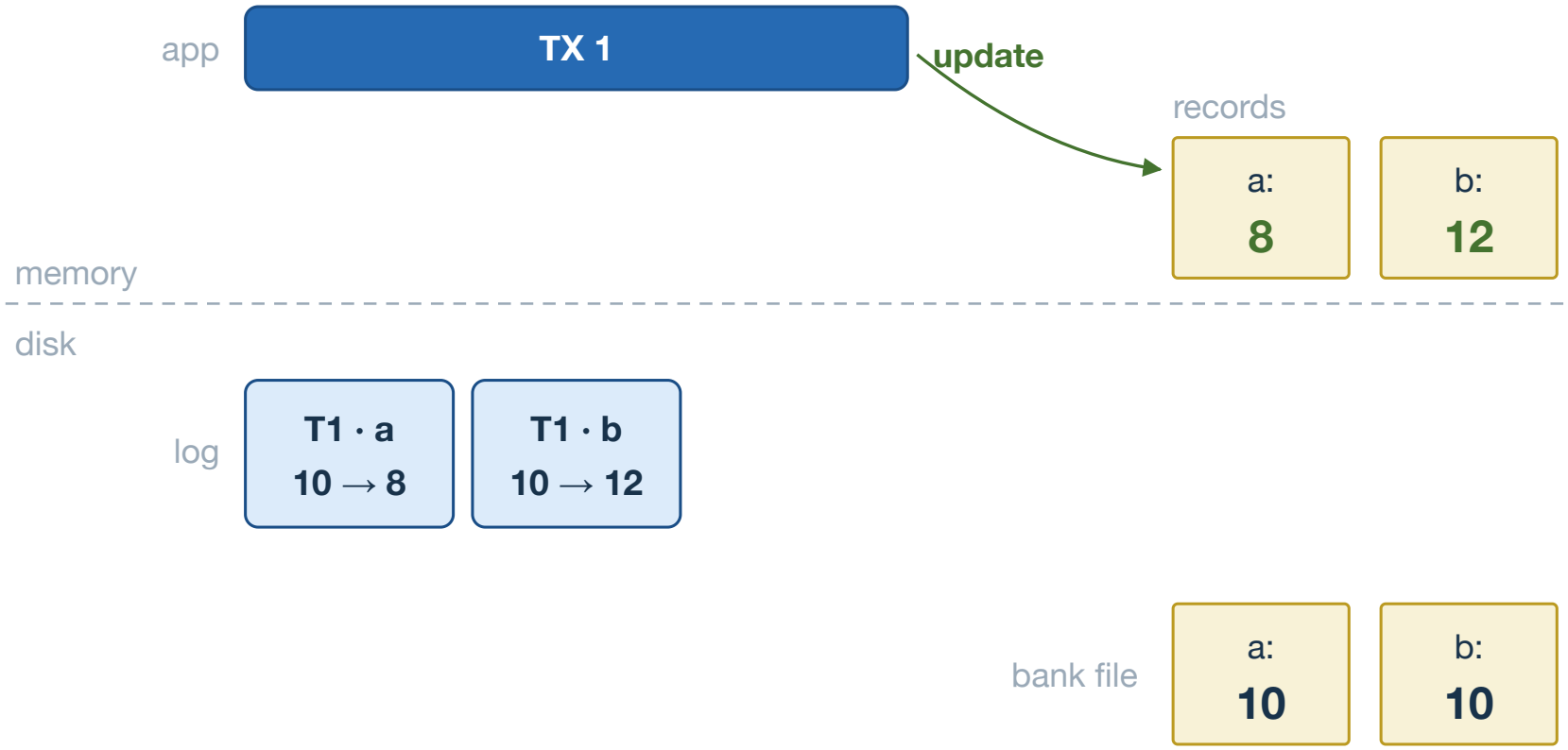
Undo the aborted records end to start, then redo the committed ones start to end



Recovery rules: walk the log backwards

Scan from end to start: a transaction with no commit record is aborted

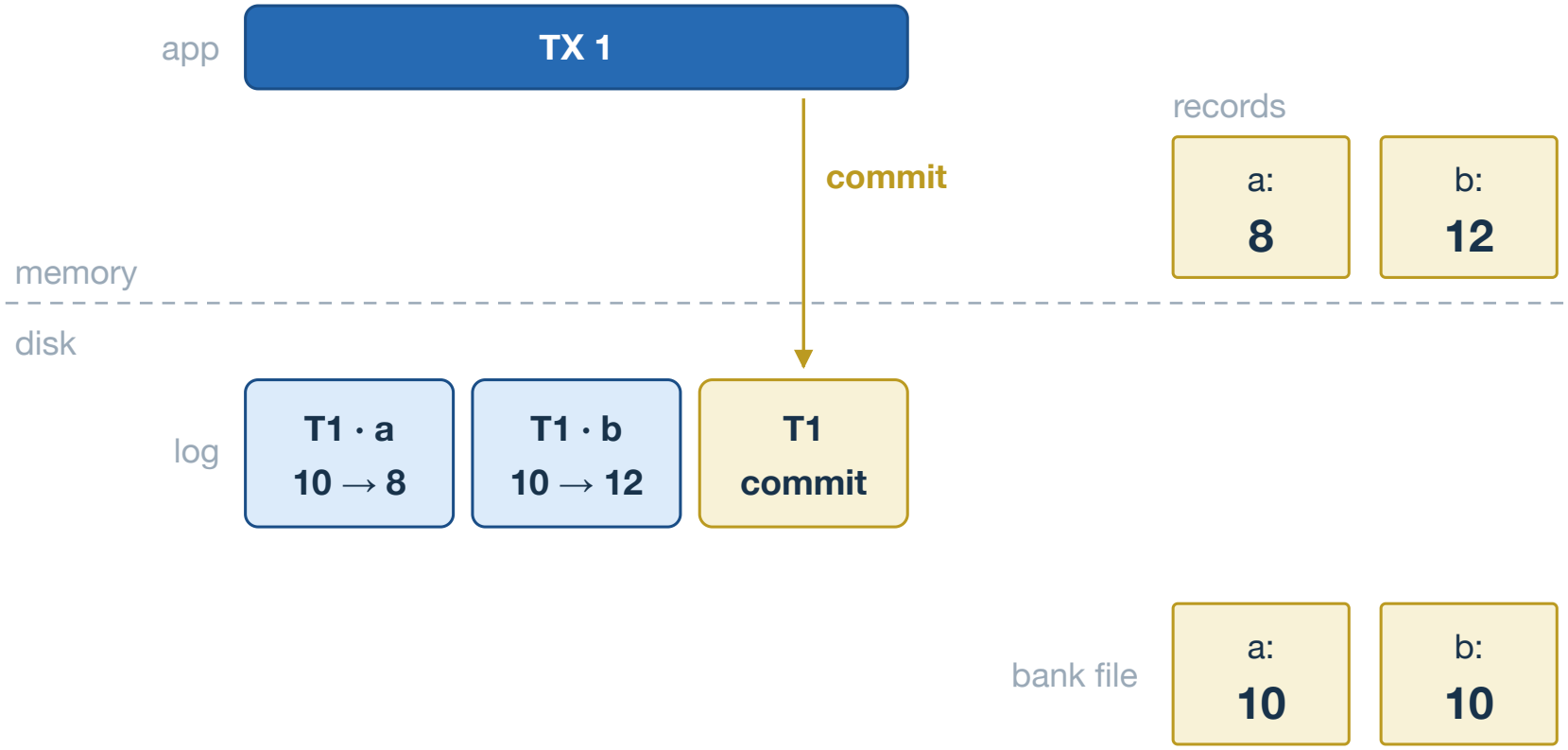
Undo the aborted records end to start, then redo the committed ones start to end



Recovery rules: walk the log backwards

Scan from end to start: a transaction with no commit record is aborted

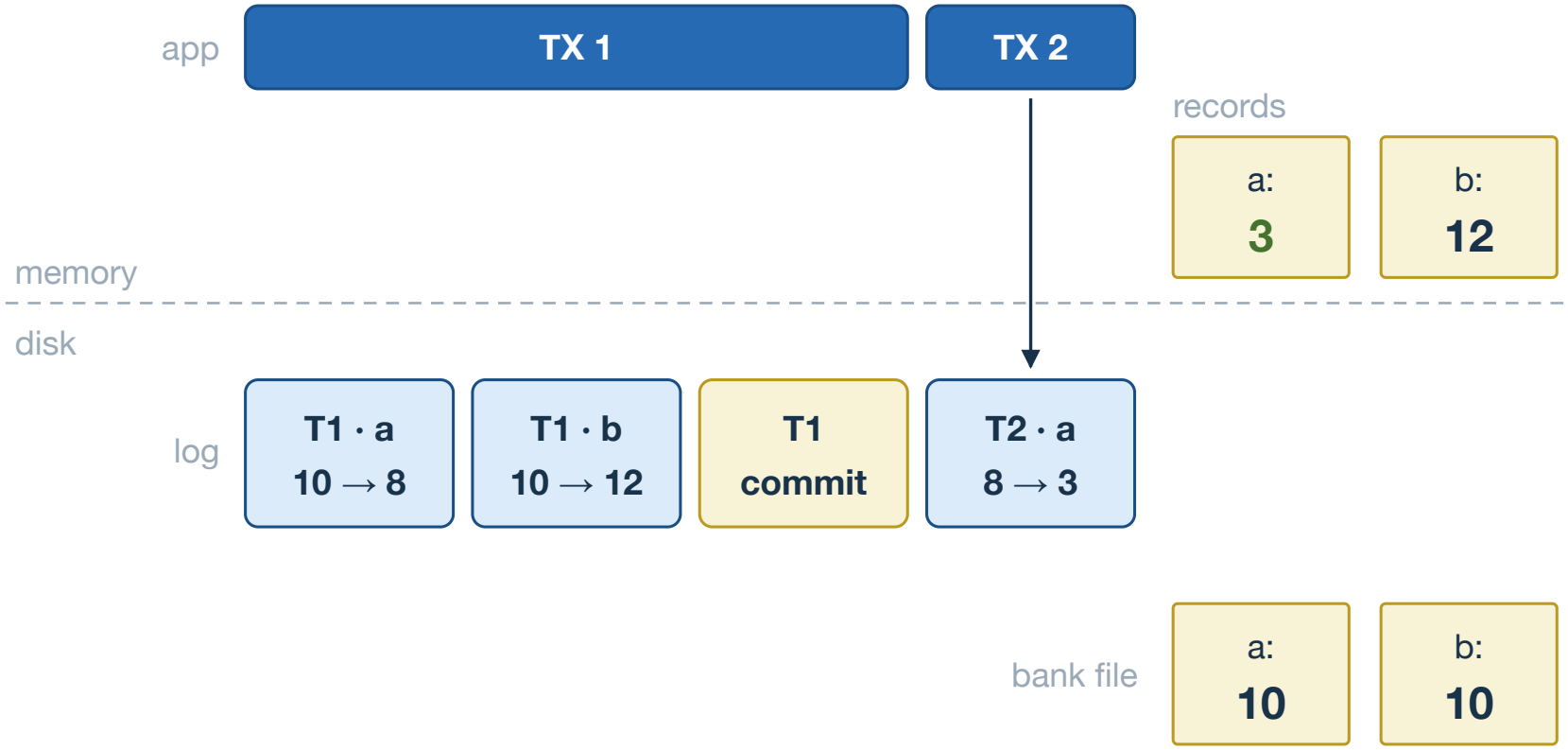
Undo the aborted records end to start, then redo the committed ones start to end



Recovery rules: walk the log backwards

Scan from end to start: a transaction with no commit record is aborted

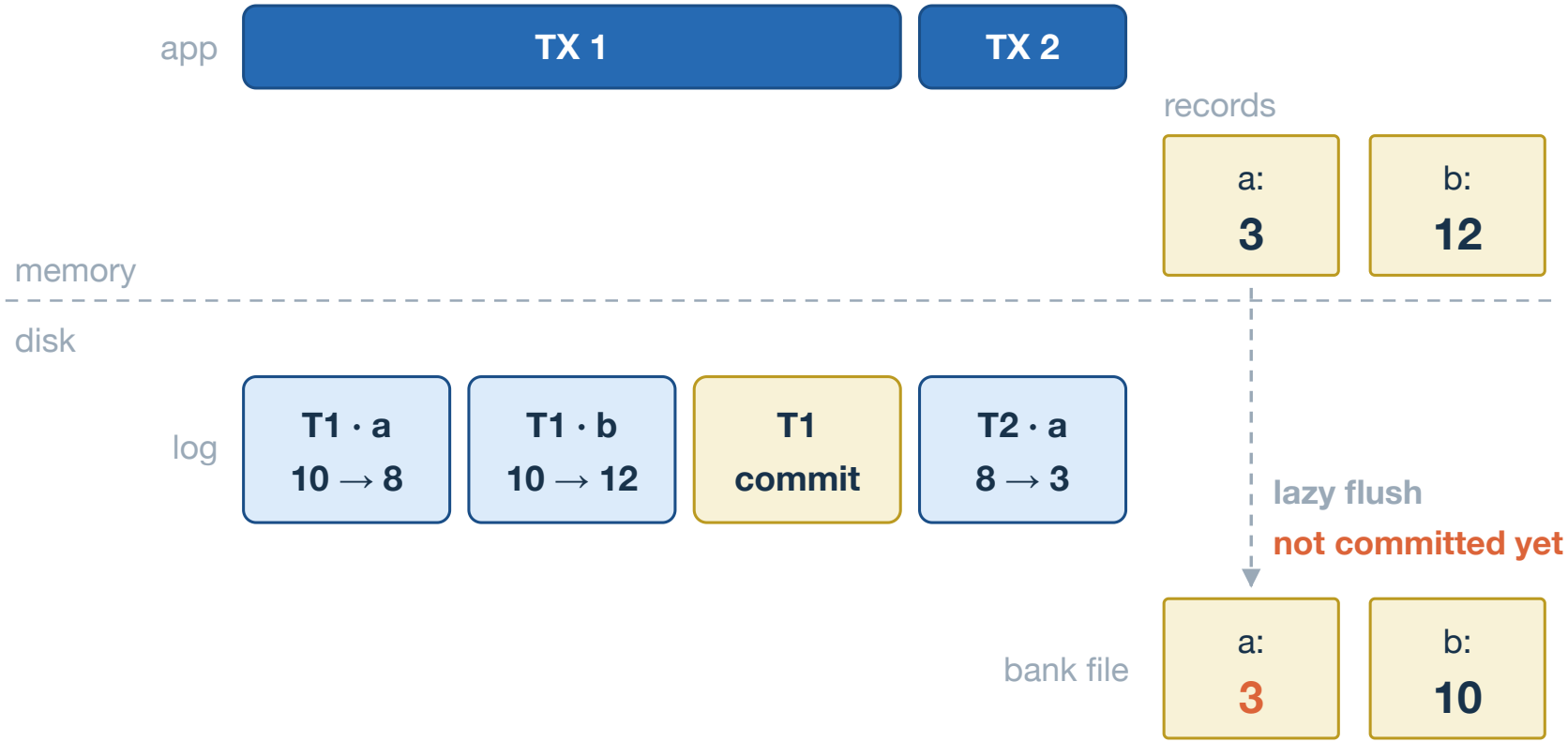
Undo the aborted records end to start, then redo the committed ones start to end



Recovery rules: walk the log backwards

Scan from end to start: a transaction with no commit record is aborted

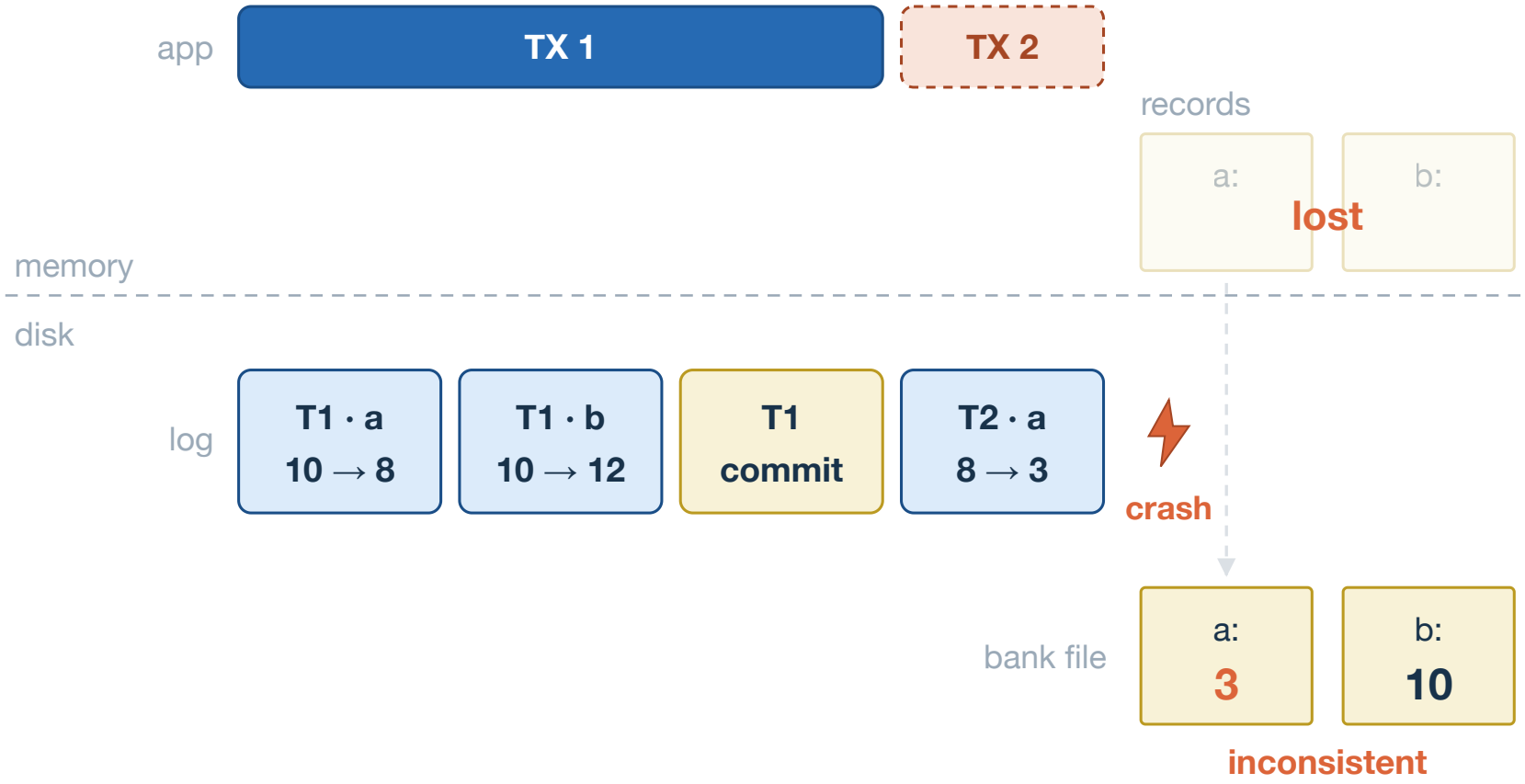
Undo the aborted records end to start, then redo the committed ones start to end



Recovery rules: walk the log backwards

Scan from end to start: a transaction with no commit record is aborted

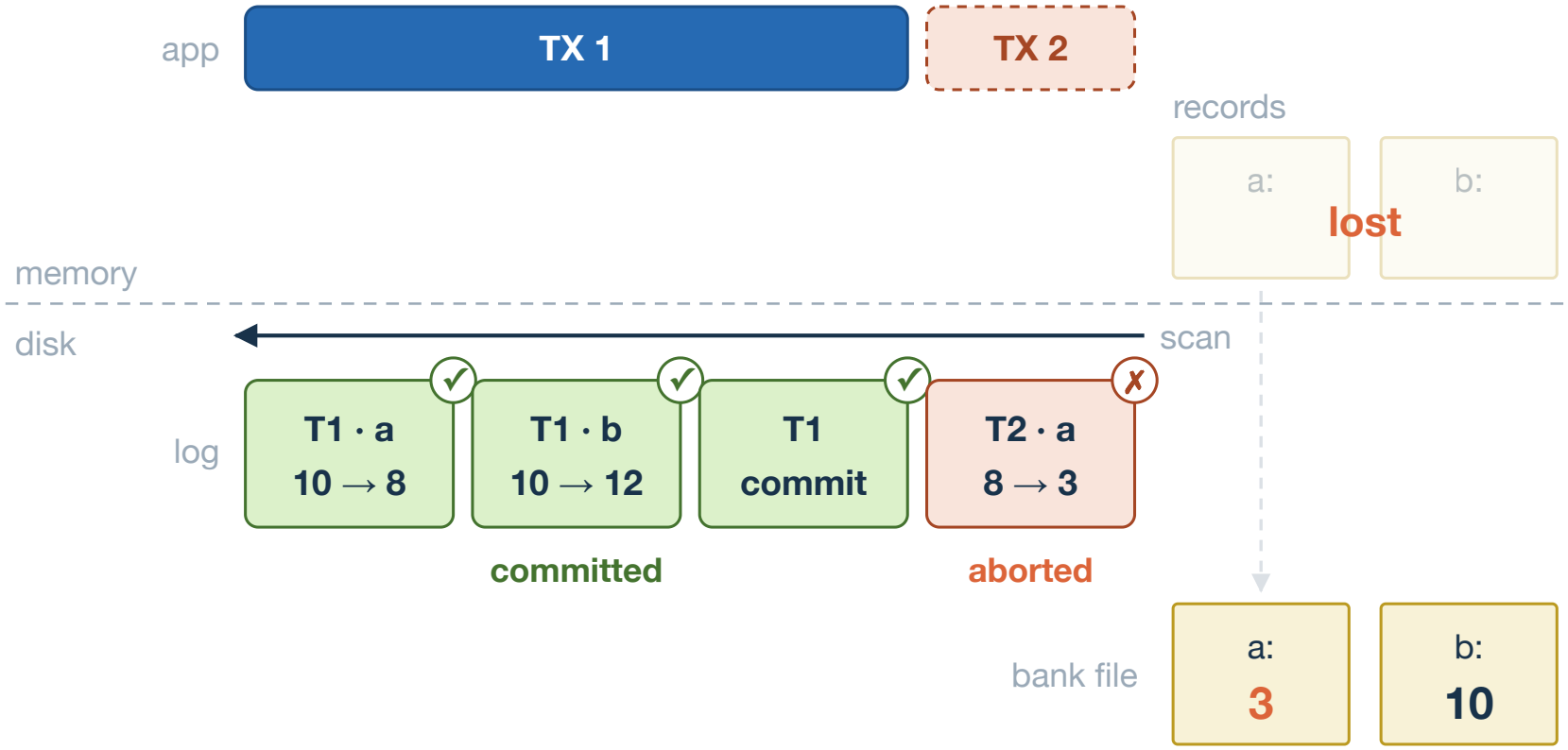
Undo the aborted records end to start, then redo the committed ones start to end



Recovery rules: walk the log backwards

Scan from end to start: a transaction with no commit record is aborted

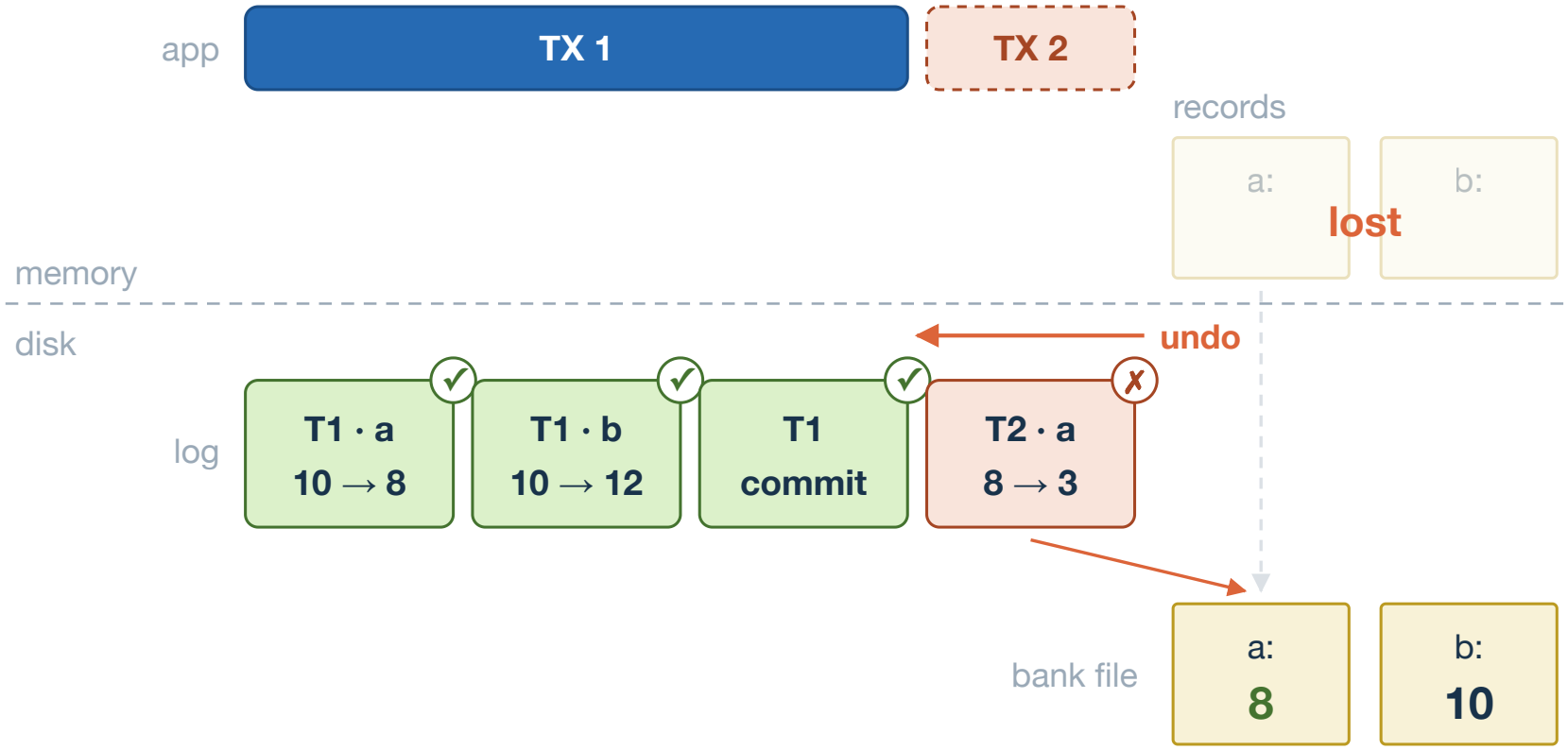
Undo the aborted records end to start, then redo the committed ones start to end



Recovery rules: walk the log backwards

Scan from end to start: a transaction with no commit record is aborted

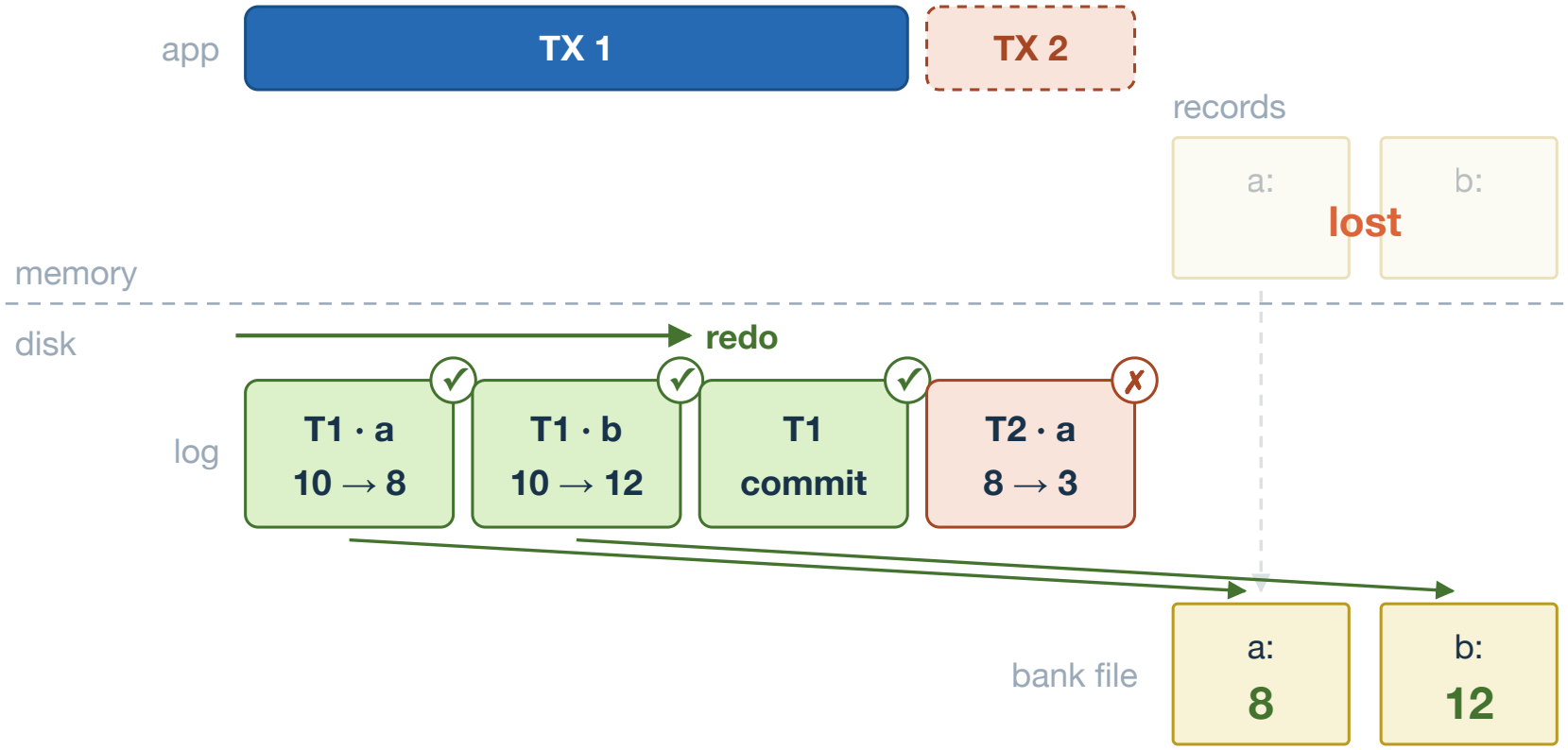
Undo the aborted records end to start, then redo the committed ones start to end



Recovery rules: walk the log backwards

Scan from end to start: a transaction with no commit record is aborted

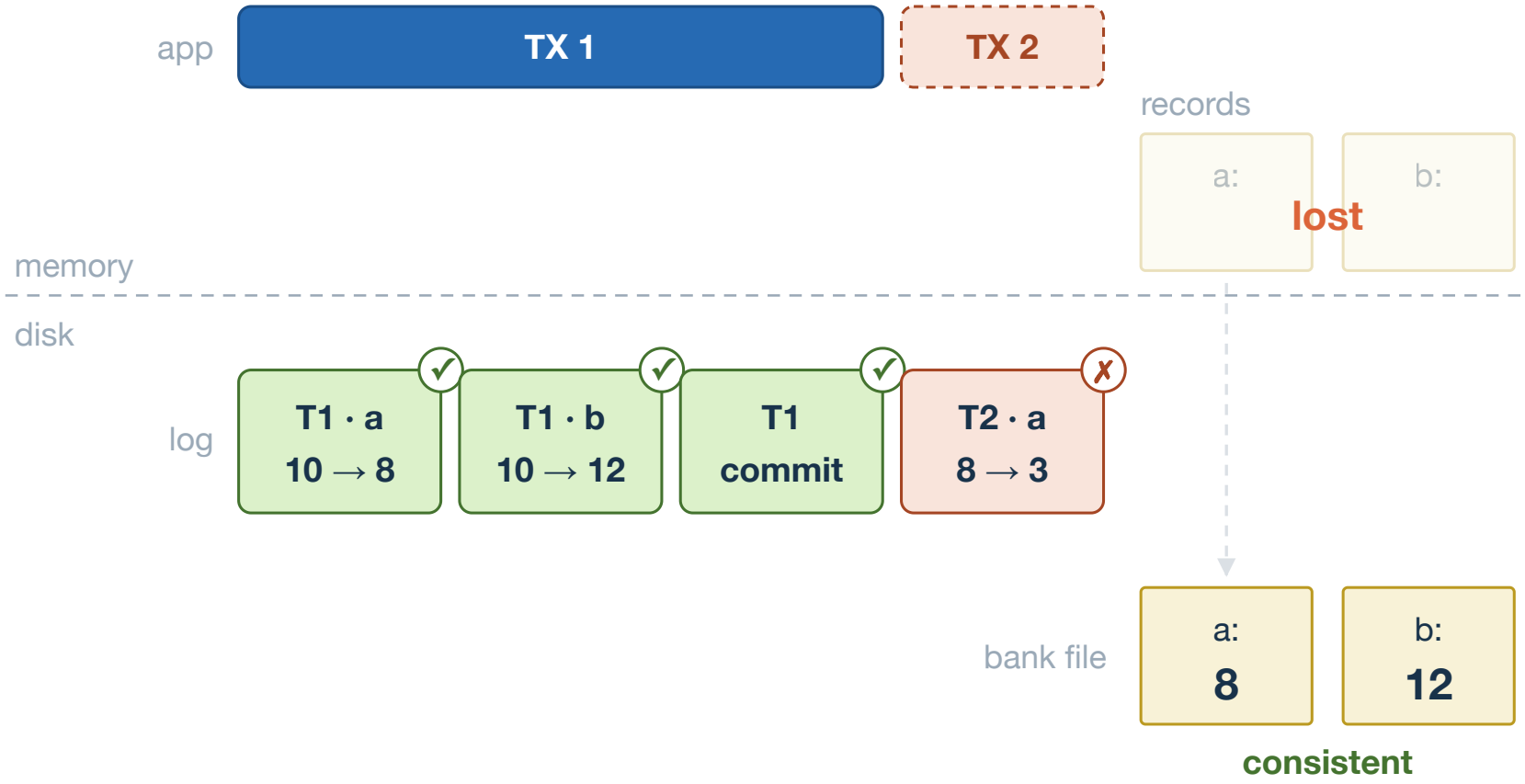
Undo the aborted records end to start, then redo the committed ones start to end



Recovery rules: walk the log backwards

Scan from end to start: a transaction with no commit record is aborted

Undo the aborted records end to start, then redo the committed ones start to end



The remaining problem: the log file keeps growing

Both redo-only and undo-redo only append to the log, never delete

This is unnecessary

- Most of its updates are already flushed to the disk; keeping them is useless
- The larger the log, the slower the recovery

Moreover, machines fail infrequently — maybe once in days

- We carry an ever-growing burden for something that rarely happens

We need a way to determine which part of the log can be discarded, then discard it

What is a checkpoint

Save the system state in a compact form, such that re-execution (restore) is faster

Not just for logs

- files after recovering the logs
- trained parameters of an LLM
- a process's memory (including CPUs and GPUs)
- the log used for reordering in an eventually consistent system

The key methodology is the same; the optimizations are system-dependent — today we focus on our bank application

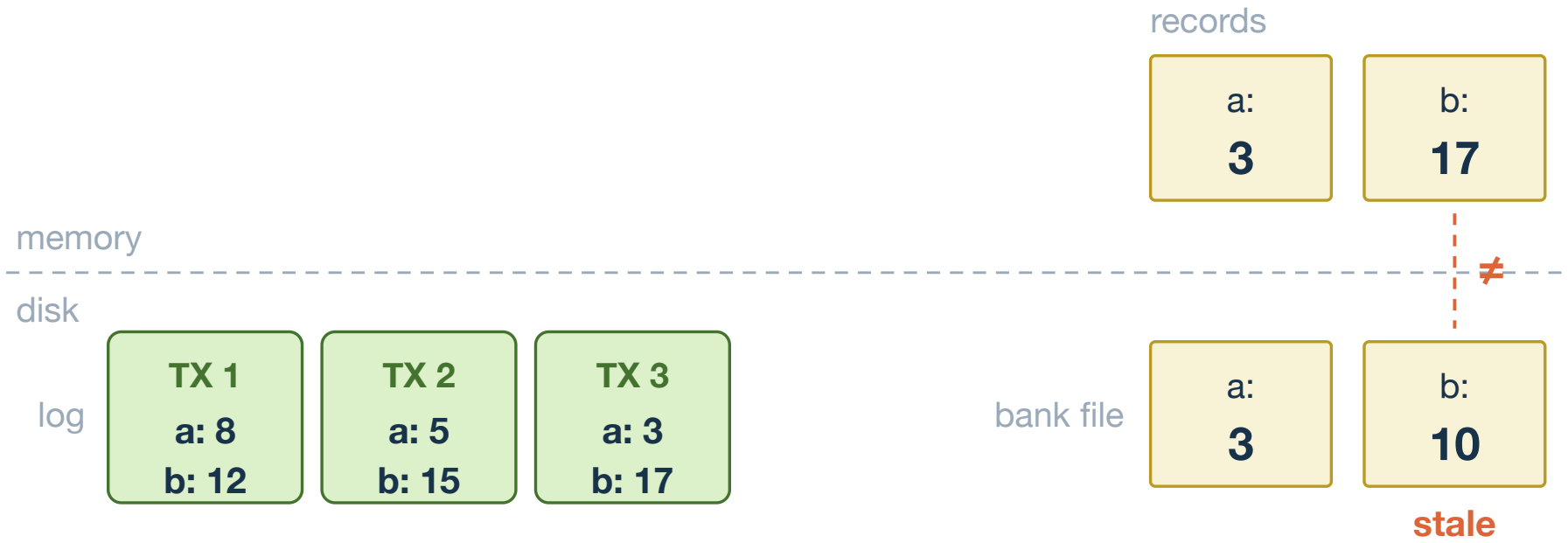
Checkpoint to truncate the log

Naive checkpoint: run the recovery process to bring the system to a consistent state; once it is done, discard the entire log

- Issue: too slow — and concurrent updates must also be handled

Observation: the un-flushed updates only live in the page cache

- Flush the whole page cache, and the committed transactions' log entries become useless



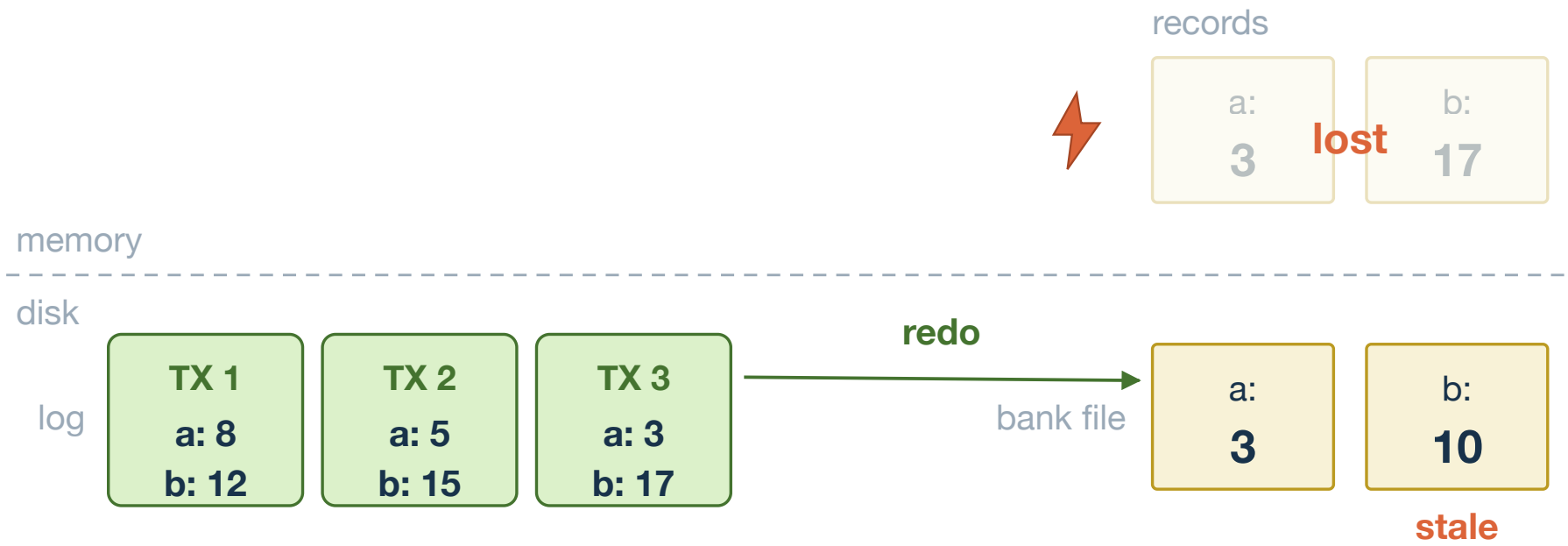
Checkpoint to truncate the log

Naive checkpoint: run the recovery process to bring the system to a consistent state; once it is done, discard the entire log

- Issue: too slow — and concurrent updates must also be handled

Observation: the un-flushed updates only live in the page cache

- Flush the whole page cache, and the committed transactions' log entries become useless



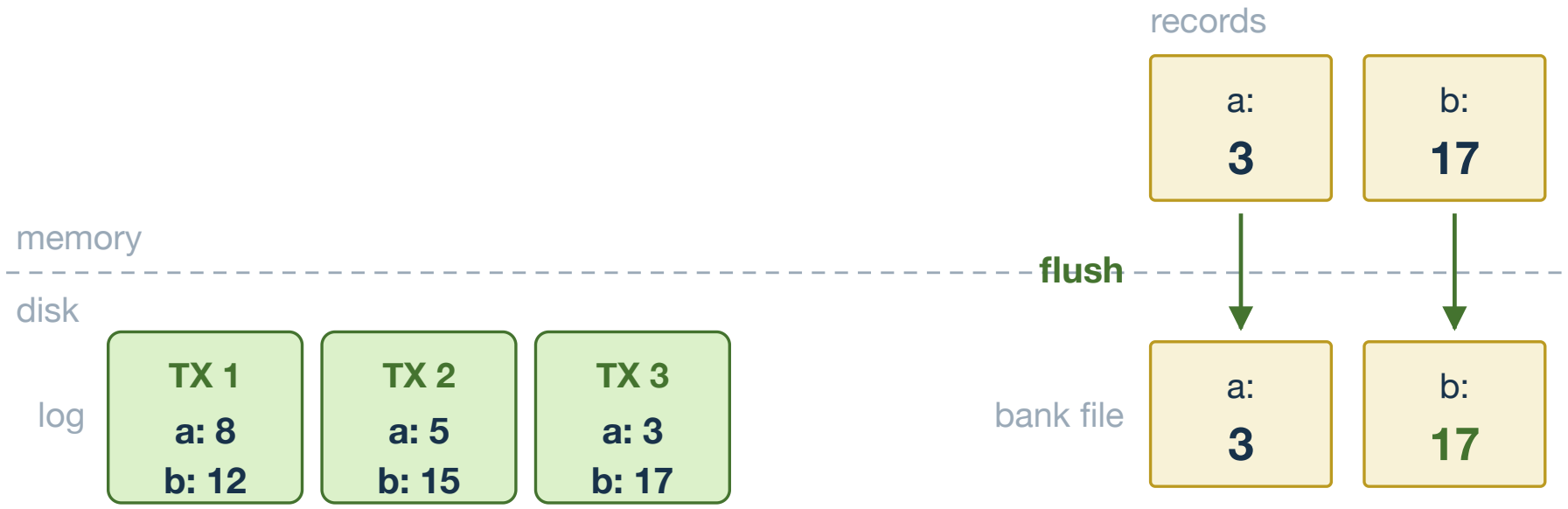
Checkpoint to truncate the log

Naive checkpoint: run the recovery process to bring the system to a consistent state; once it is done, discard the entire log

- Issue: too slow — and concurrent updates must also be handled

Observation: the un-flushed updates only live in the page cache

- Flush the whole page cache, and the committed transactions' log entries become useless



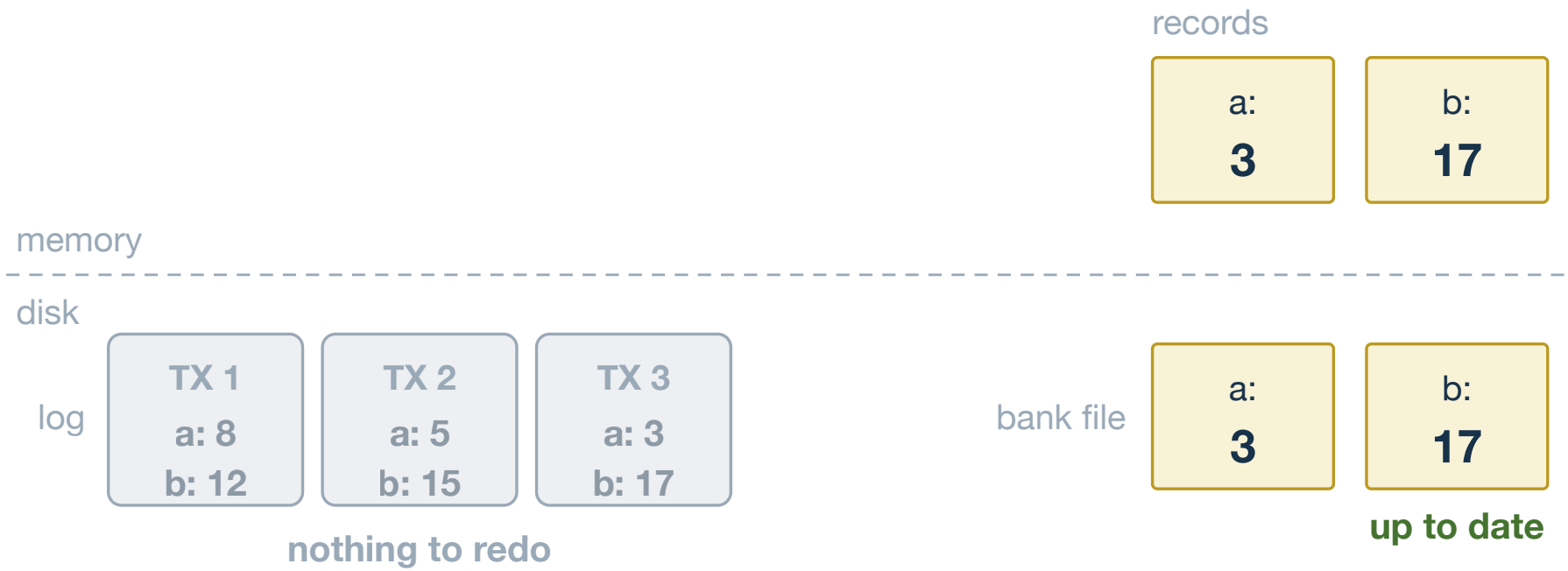
Checkpoint to truncate the log

Naive checkpoint: run the recovery process to bring the system to a consistent state; once it is done, discard the entire log

- Issue: too slow — and concurrent updates must also be handled

Observation: the un-flushed updates only live in the page cache

- Flush the whole page cache, and the committed transactions' log entries become useless



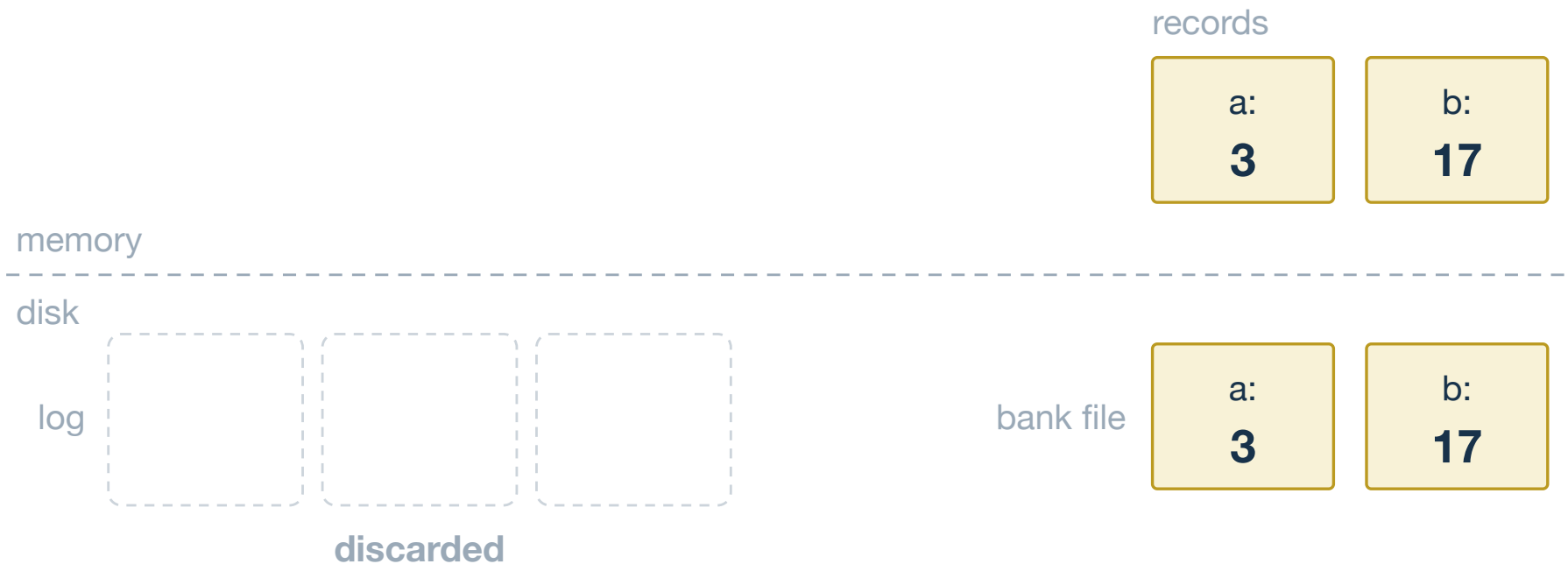
Checkpoint to truncate the log

Naive checkpoint: run the recovery process to bring the system to a consistent state; once it is done, discard the entire log

- Issue: too slow — and concurrent updates must also be handled

Observation: the un-flushed updates only live in the page cache

- Flush the whole page cache, and the committed transactions' log entries become useless

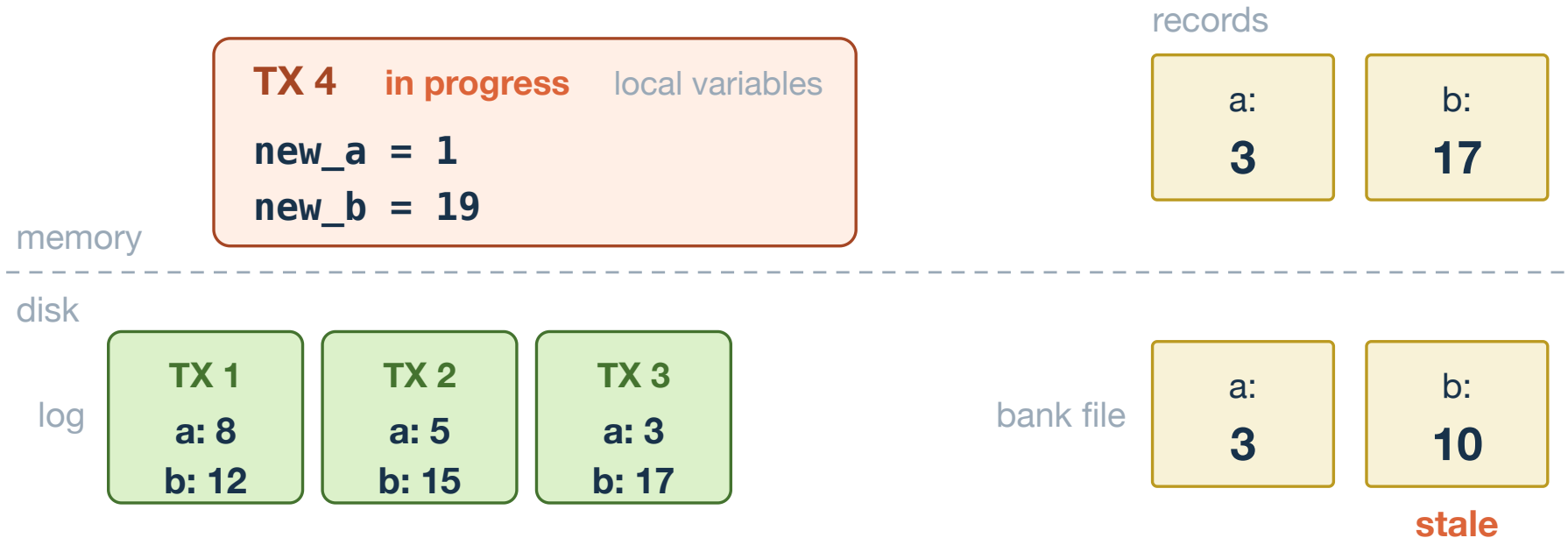


Question: what if a TX is ongoing during the checkpoint? (redo-only)

Before its commit point, TX 4 lives only in local variables

- Nothing in the log, nothing in the page cache — the flush and the discard never touch it

Question: what about undo-redo logging? Its uncommitted updates can reach the disk

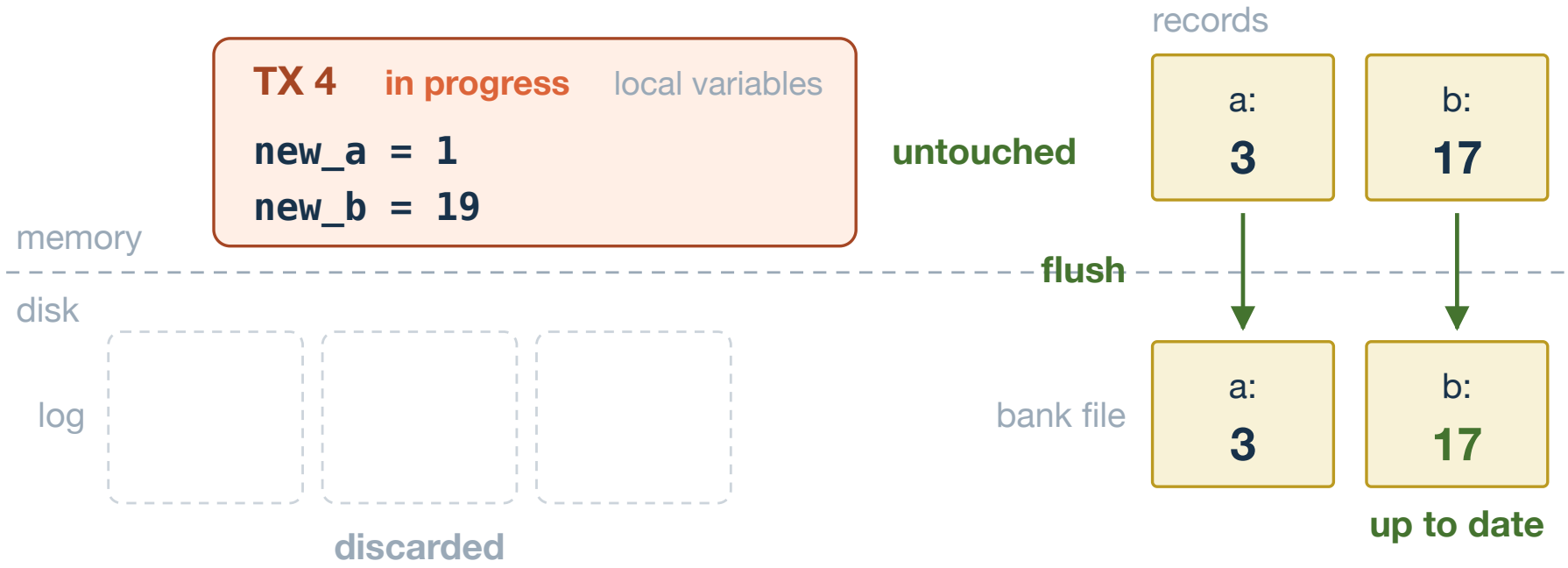


Question: what if a TX is ongoing during the checkpoint? (redo-only)

Before its commit point, TX 4 lives only in local variables

- Nothing in the log, nothing in the page cache — the flush and the discard never touch it

Question: what about undo-redo logging? Its uncommitted updates can reach the disk

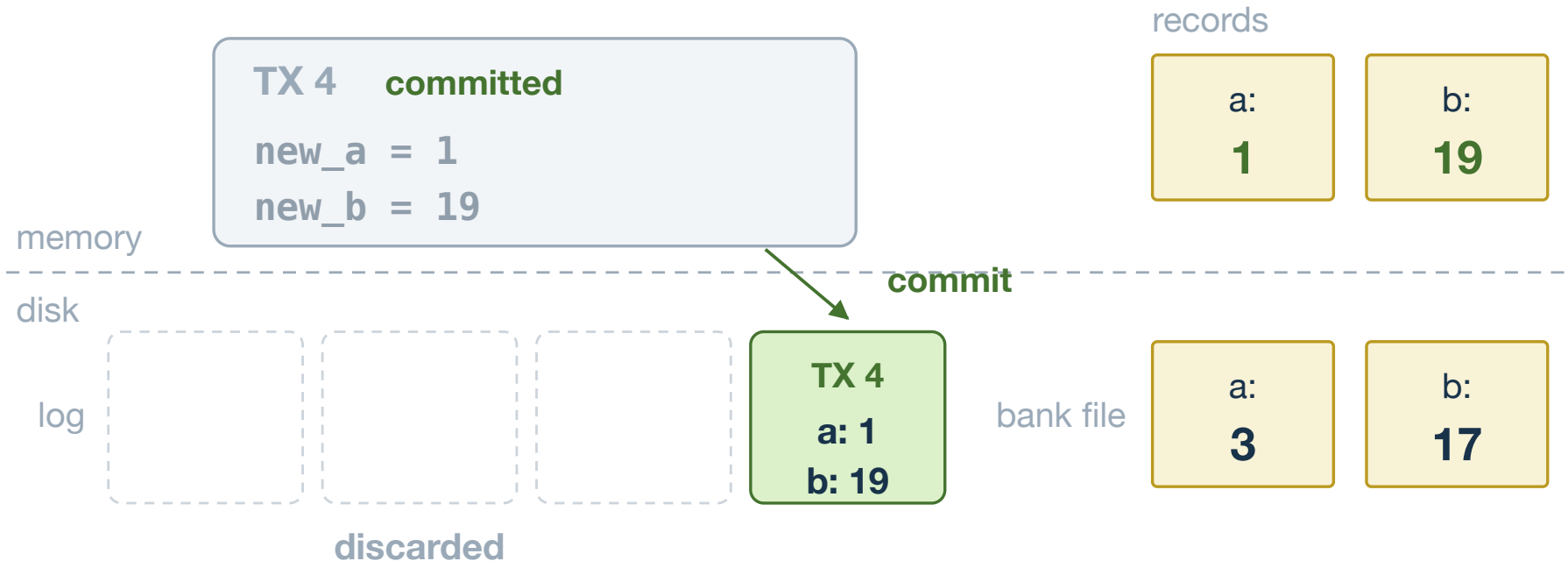


Question: what if a TX is ongoing during the checkpoint? (redo-only)

Before its commit point, TX 4 lives only in local variables

- Nothing in the log, nothing in the page cache — the flush and the discard never touch it

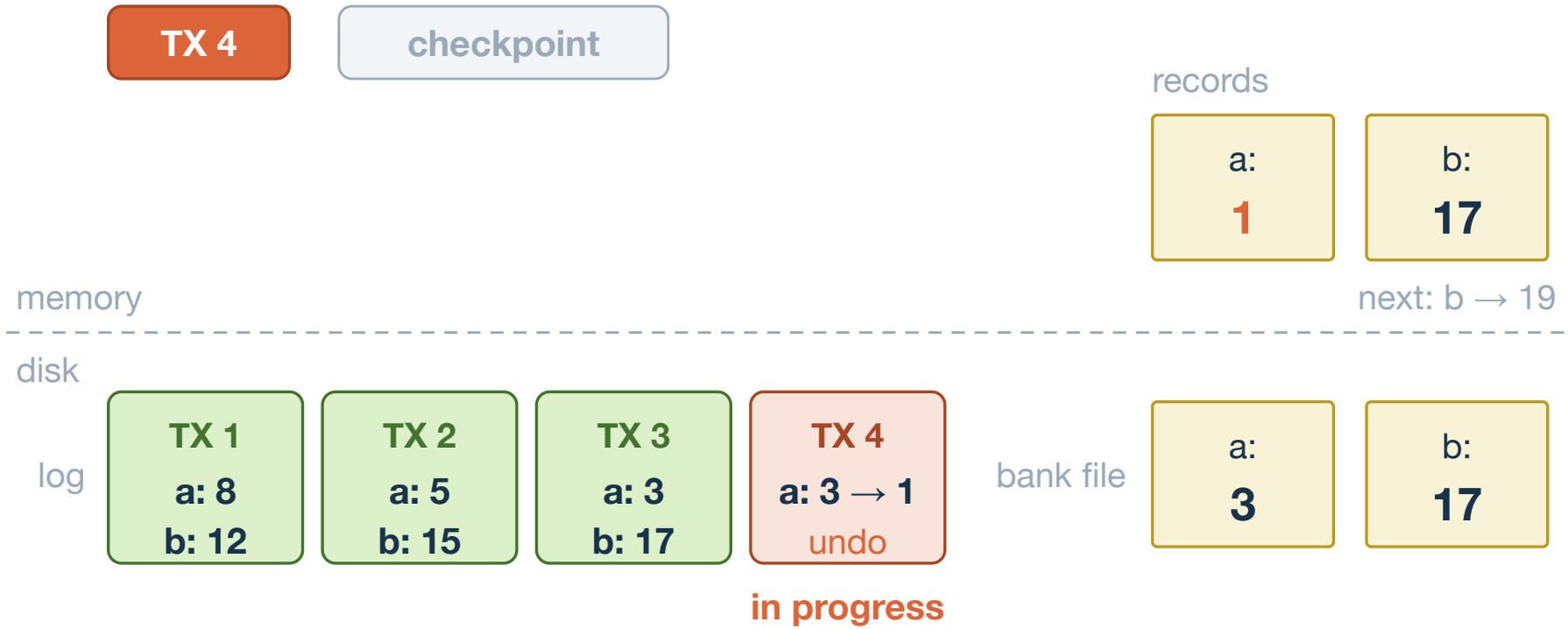
Question: what about undo-redo logging? Its uncommitted updates can reach the disk



Question: what if a TX is ongoing during the checkpoint? (undo-redo)

TX 4 has updated `a` but not `b` when the checkpoint flushes and discards the log

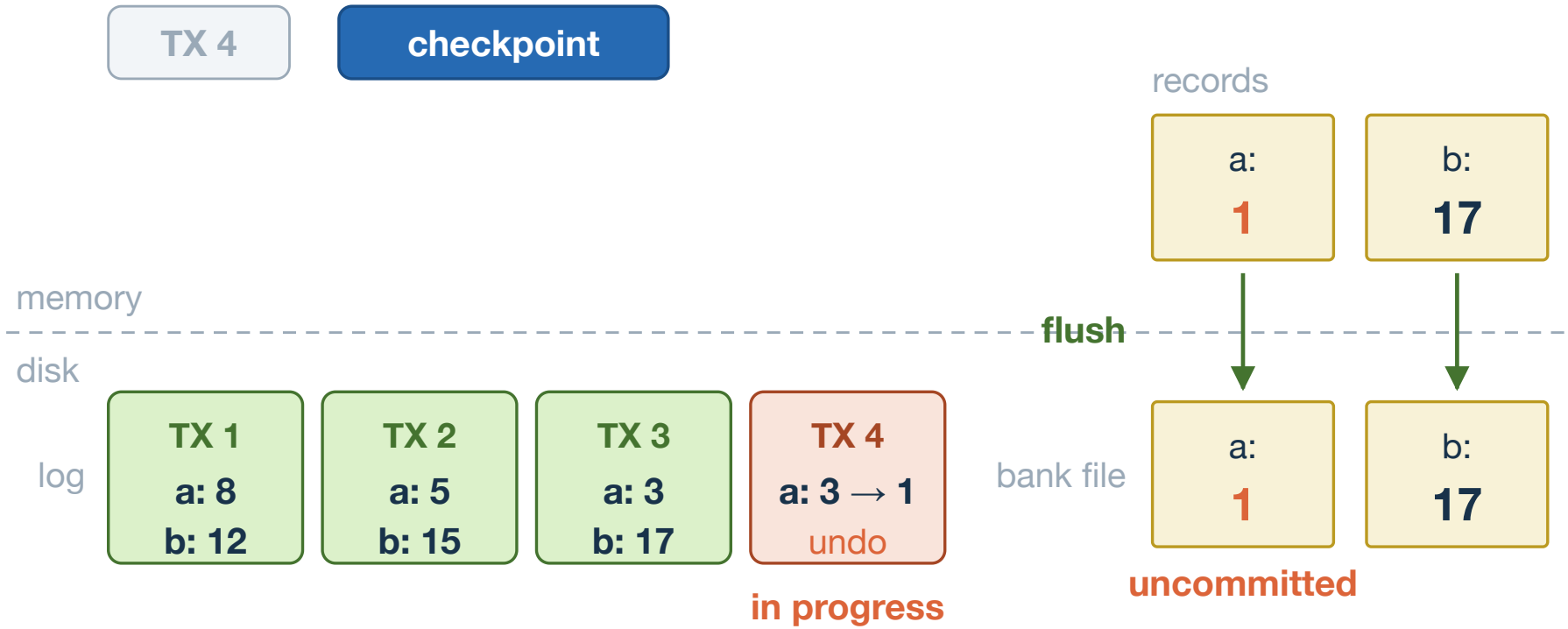
- Harmless if TX 4 commits; if it must be undone, its undo record is gone



Question: what if a TX is ongoing during the checkpoint? (undo-redo)

TX 4 has updated `a` but not `b` when the checkpoint flushes and discards the log

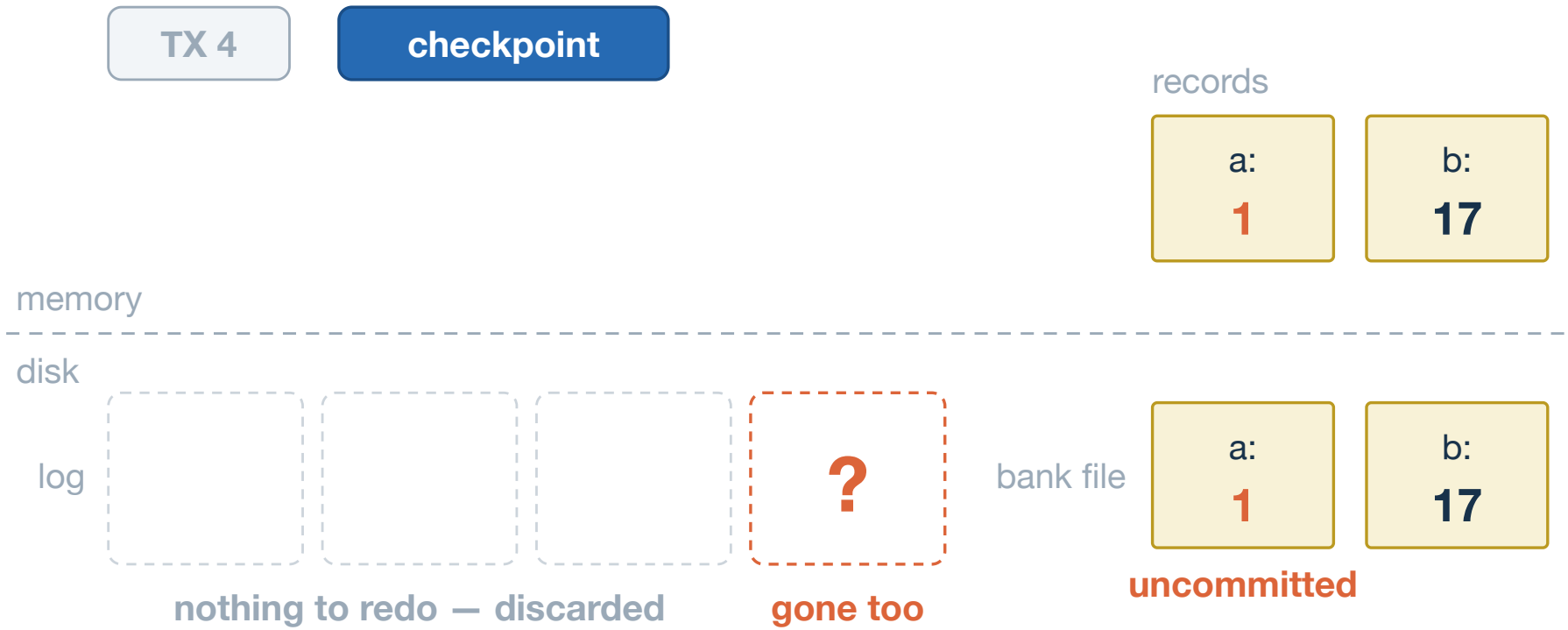
- Harmless if TX 4 commits; if it must be undone, its undo record is gone



Question: what if a TX is ongoing during the checkpoint? (undo-redo)

TX 4 has updated `a` but not `b` when the checkpoint flushes and discards the log

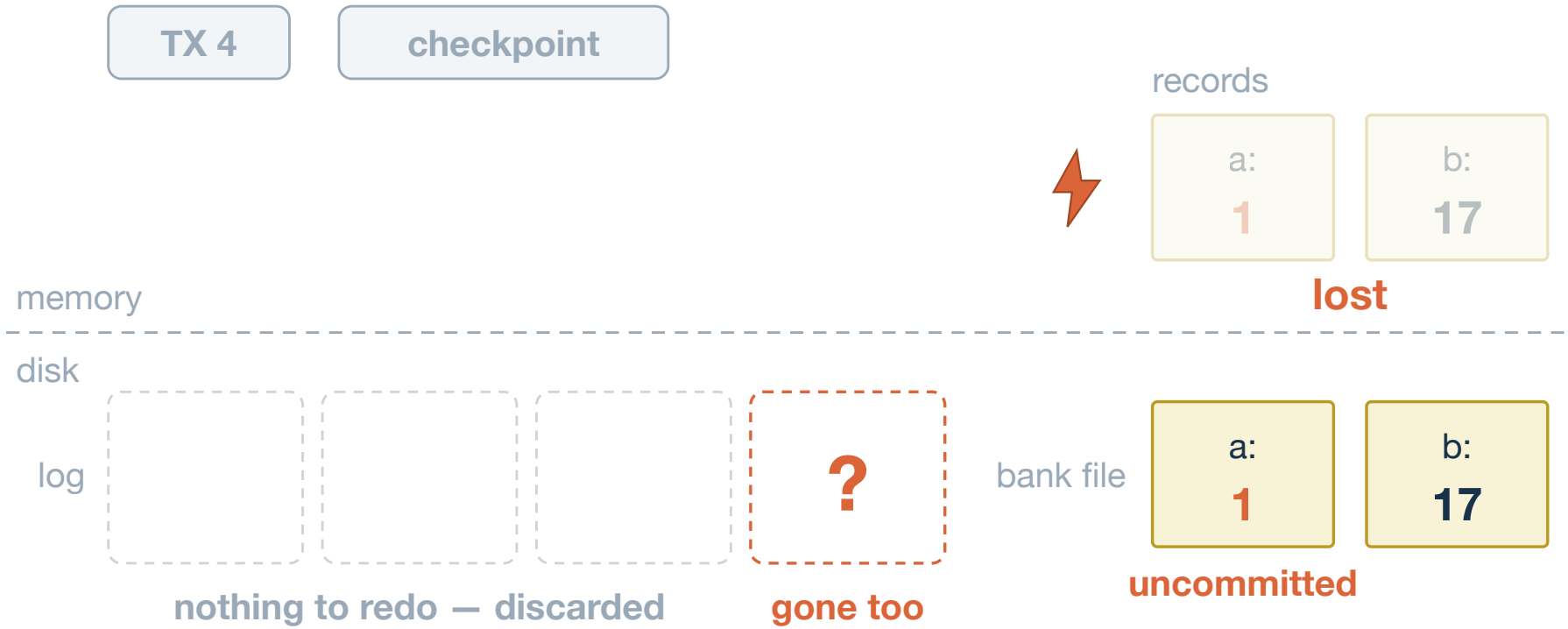
- Harmless if TX 4 commits; if it must be undone, its undo record is gone



Question: what if a TX is ongoing during the checkpoint? (undo-redo)

TX 4 has updated `a` but not `b` when the checkpoint flushes and discards the log

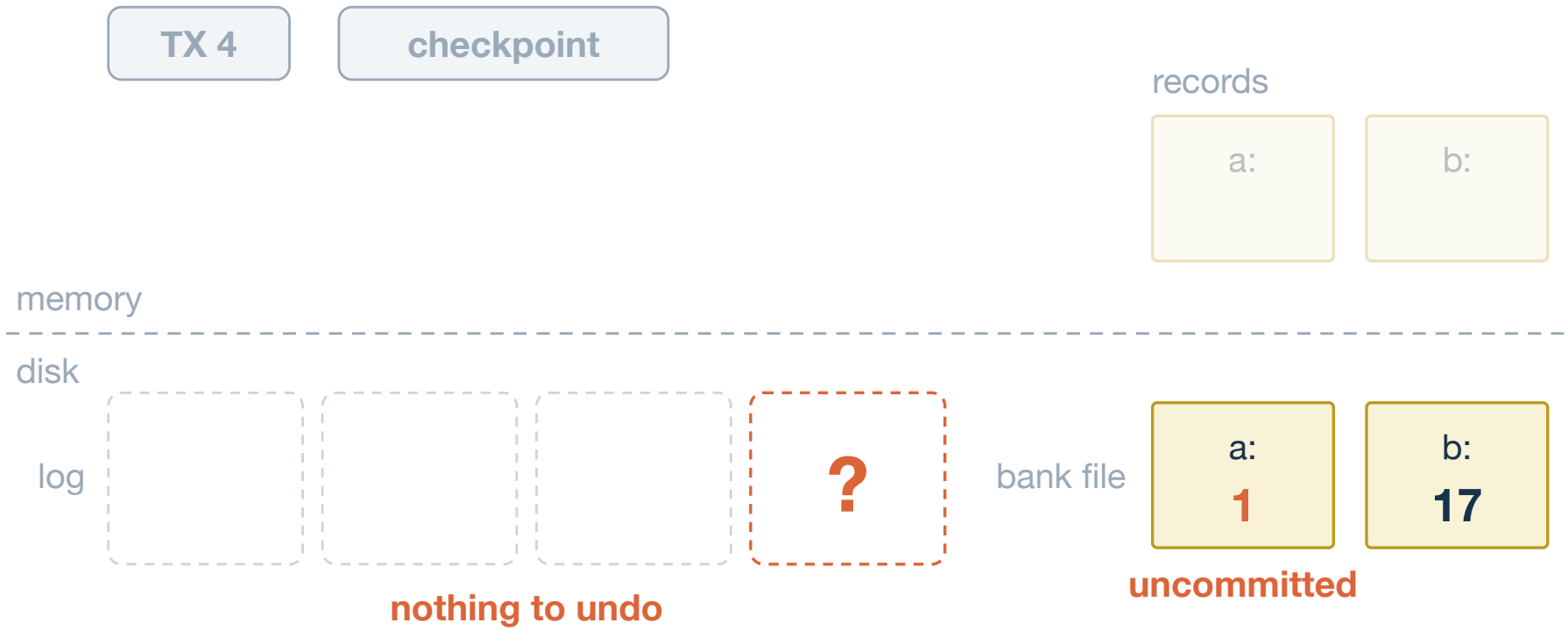
- Harmless if TX 4 commits; if it must be undone, its undo record is gone



Question: what if a TX is ongoing during the checkpoint? (undo-redo)

TX 4 has updated `a` but not `b` when the checkpoint flushes and discards the log

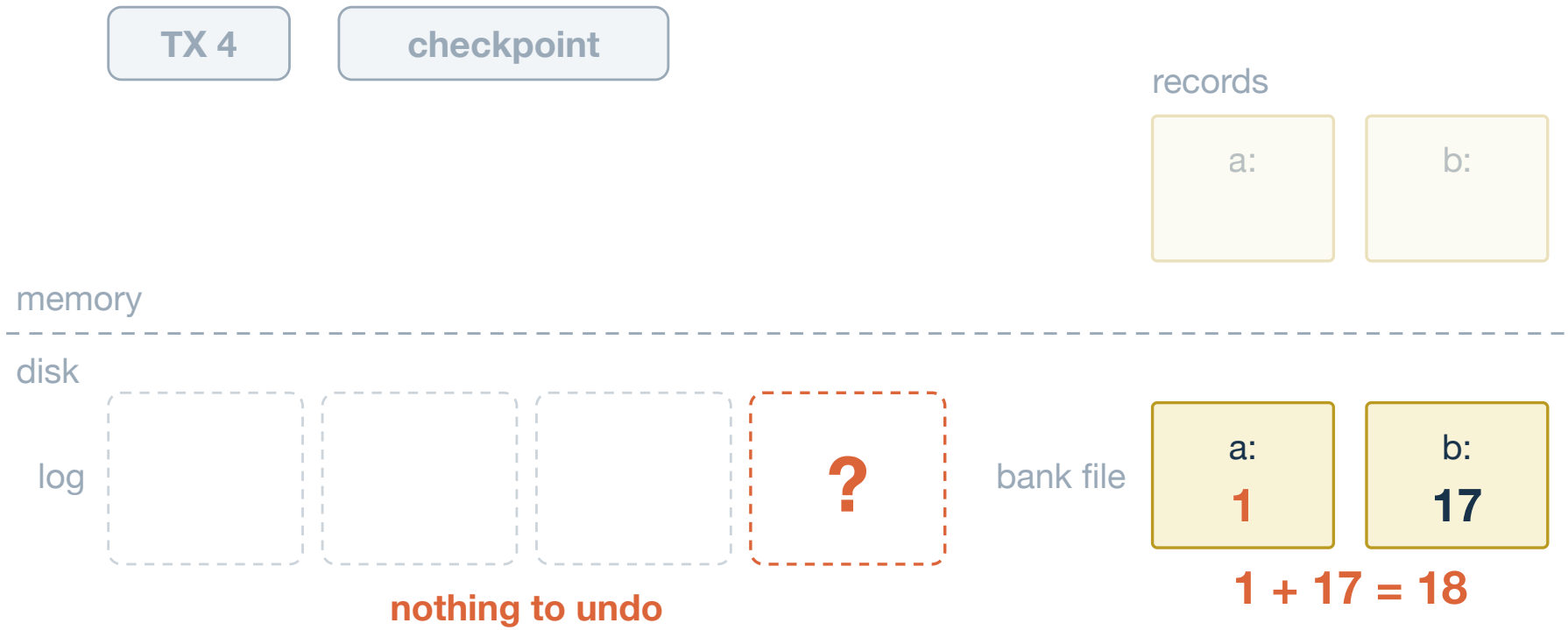
- Harmless if TX 4 commits; if it must be undone, its undo record is gone



Question: what if a TX is ongoing during the checkpoint? (undo-redo)

TX 4 has updated `a` but not `b` when the checkpoint flushes and discards the log

- Harmless if TX 4 commits; if it must be undone, its undo record is gone

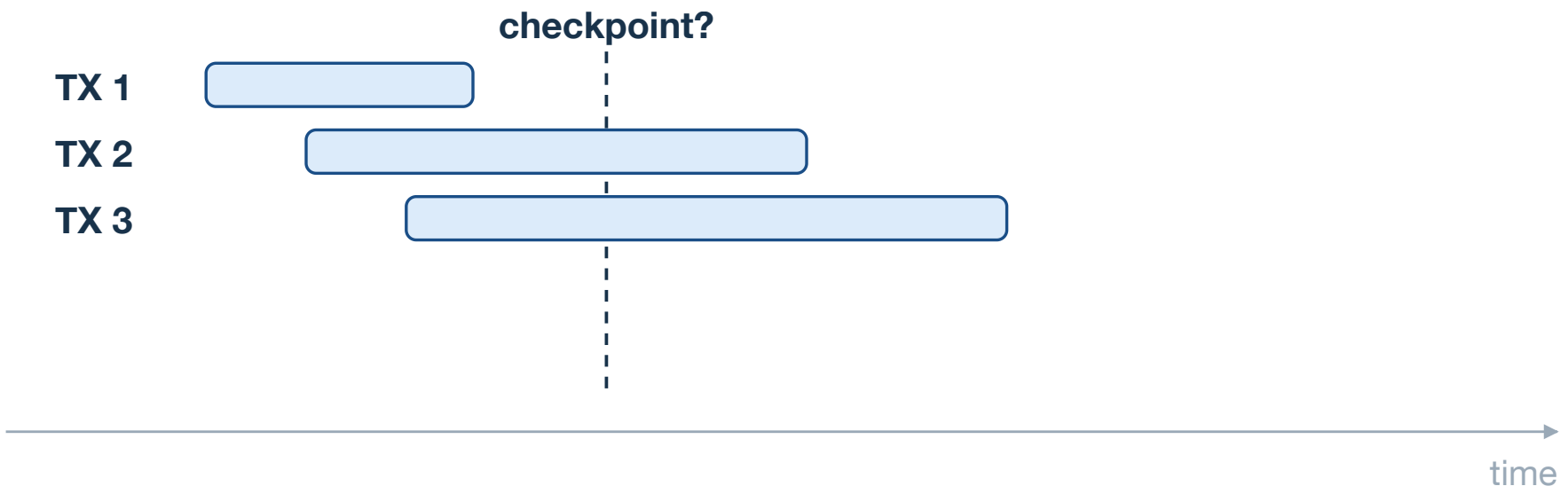


Naive solution: stop all transactions, then do the checkpoint

TX 4 went wrong because its partial state cannot be flushed

- Method: stop admitting new transactions; wait till the ongoing ones finish
- Then flush the page cache and discard the entire log

Problem: possibly long blocking time

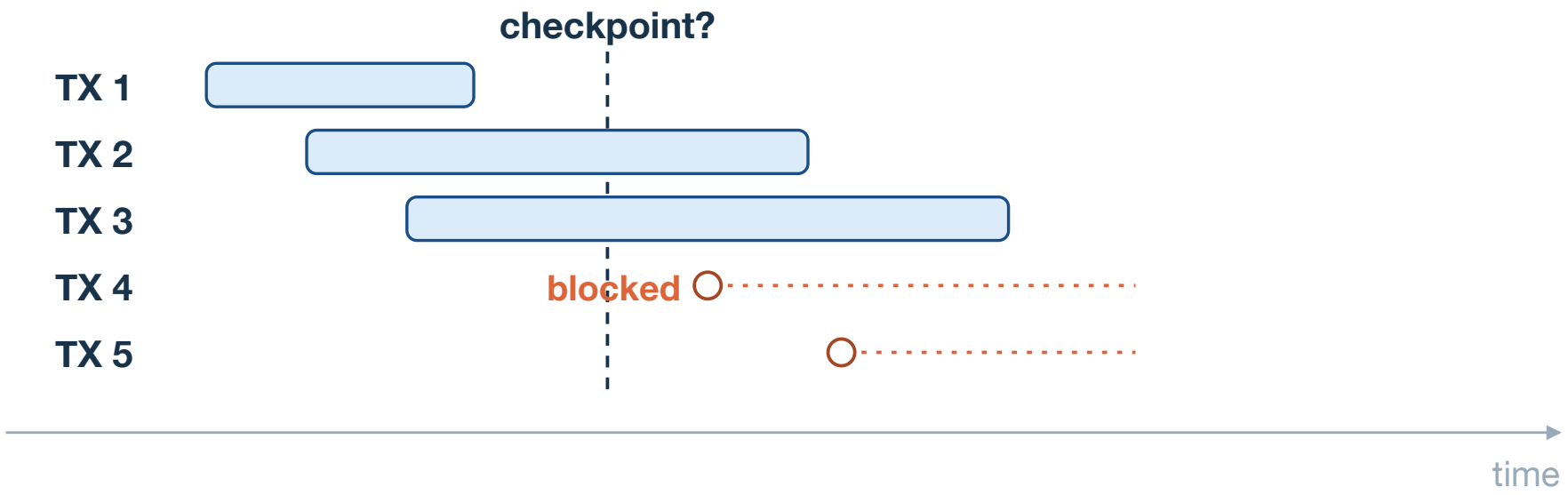


Naive solution: stop all transactions, then do the checkpoint

TX 4 went wrong because its partial state cannot be flushed

- Method: stop admitting new transactions; wait till the ongoing ones finish
- Then flush the page cache and discard the entire log

Problem: possibly long blocking time

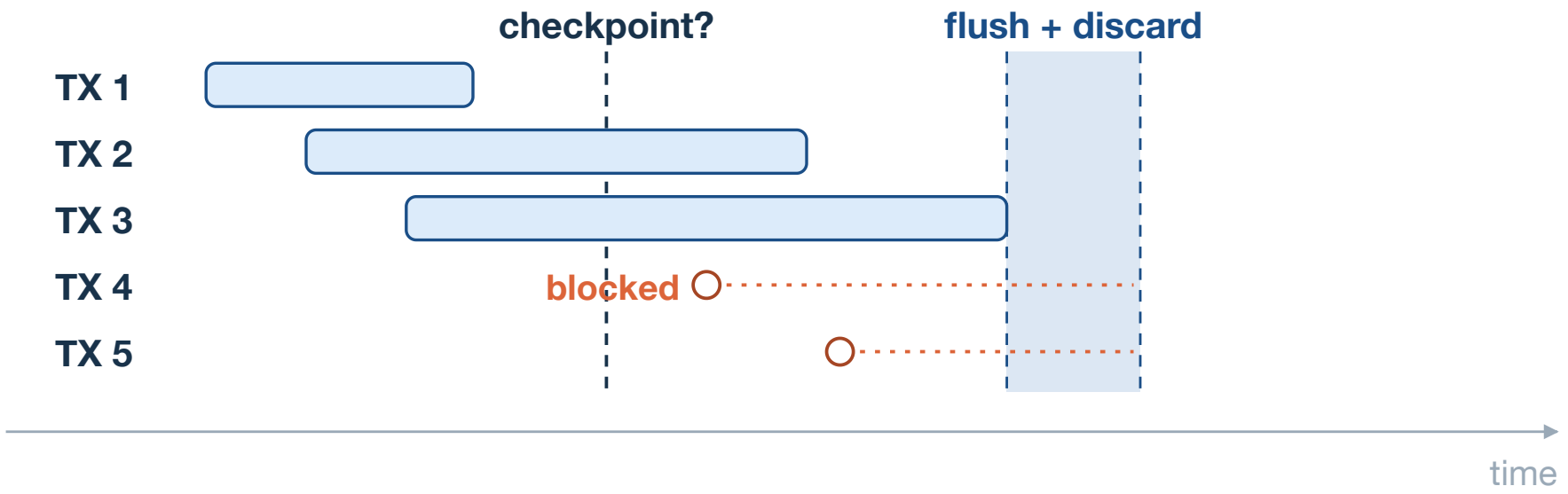


Naive solution: stop all transactions, then do the checkpoint

TX 4 went wrong because its partial state cannot be flushed

- Method: stop admitting new transactions; wait till the ongoing ones finish
- Then flush the page cache and discard the entire log

Problem: possibly long blocking time

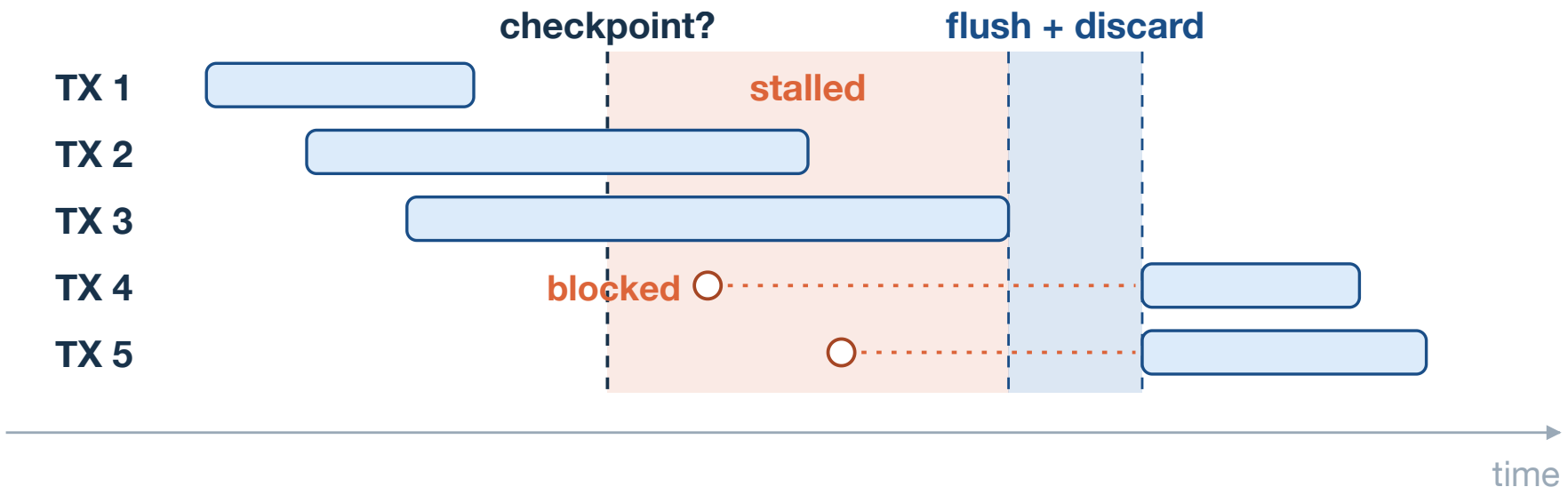


Naive solution: stop all transactions, then do the checkpoint

TX 4 went wrong because its partial state cannot be flushed

- Method: stop admitting new transactions; wait till the ongoing ones finish
- Then flush the page cache and discard the entire log

Problem: possibly long blocking time



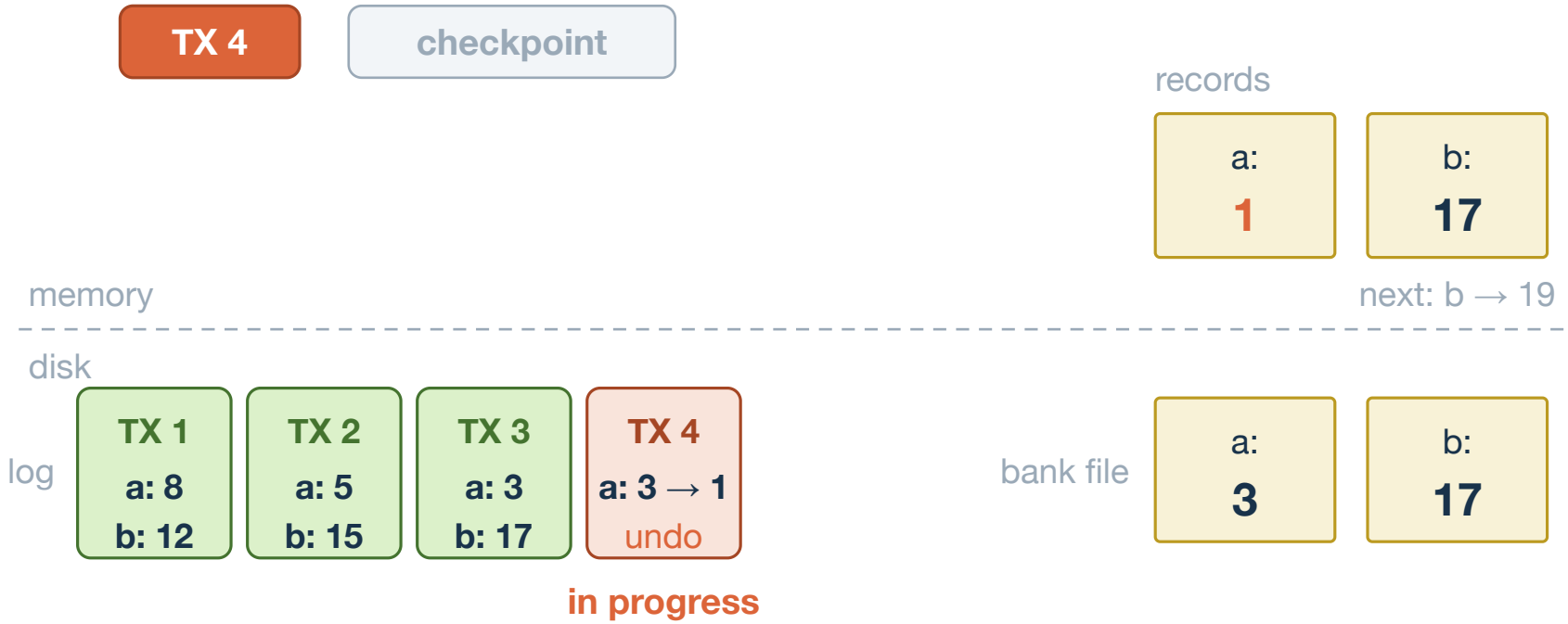
Solution: keep the in-progress TXs' log in a CKPT record

The logs are kept in a CKPT record to survive the log truncation during the checkpoint

Method: wait till no operation is in progress, to ensure the CKPT entry is complete

- An operation is short, so this wait always ends

Then we can do the checkpoint



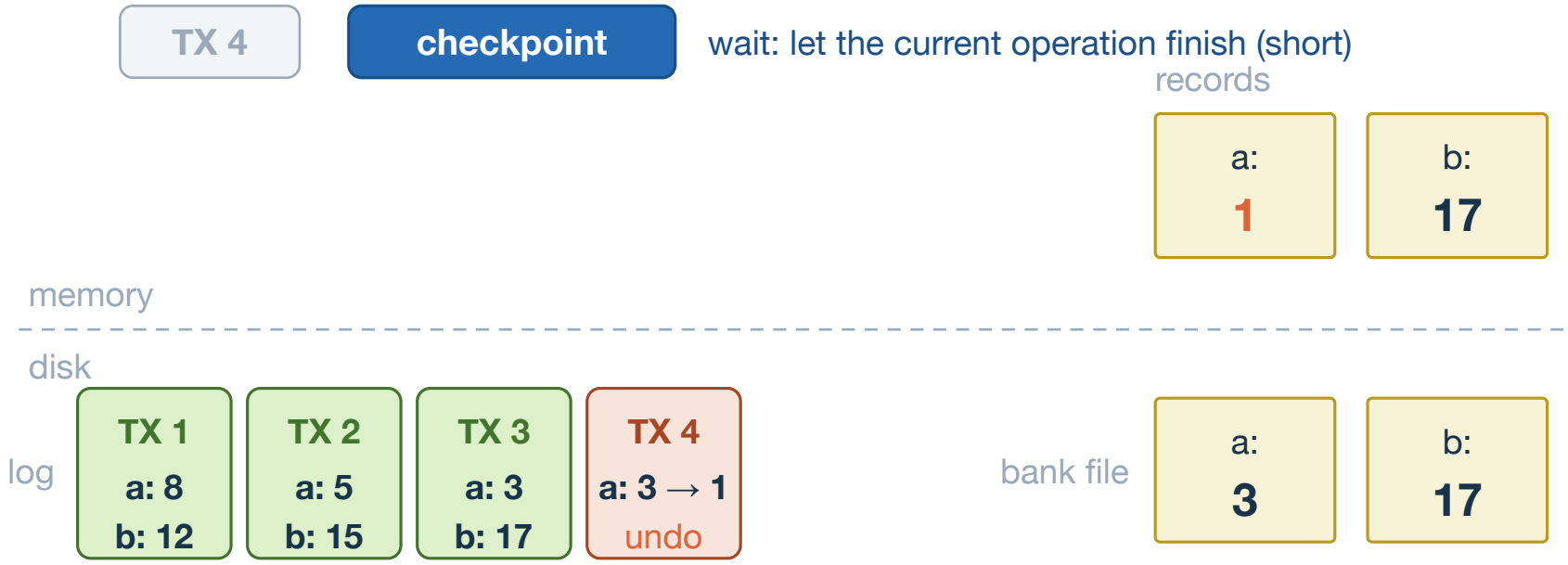
Solution: keep the in-progress TXs' log in a CKPT record

The logs are kept in a CKPT record to survive the log truncation during the checkpoint

Method: wait till no operation is in progress, to ensure the CKPT entry is complete

- An operation is short, so this wait always ends

Then we can do the checkpoint



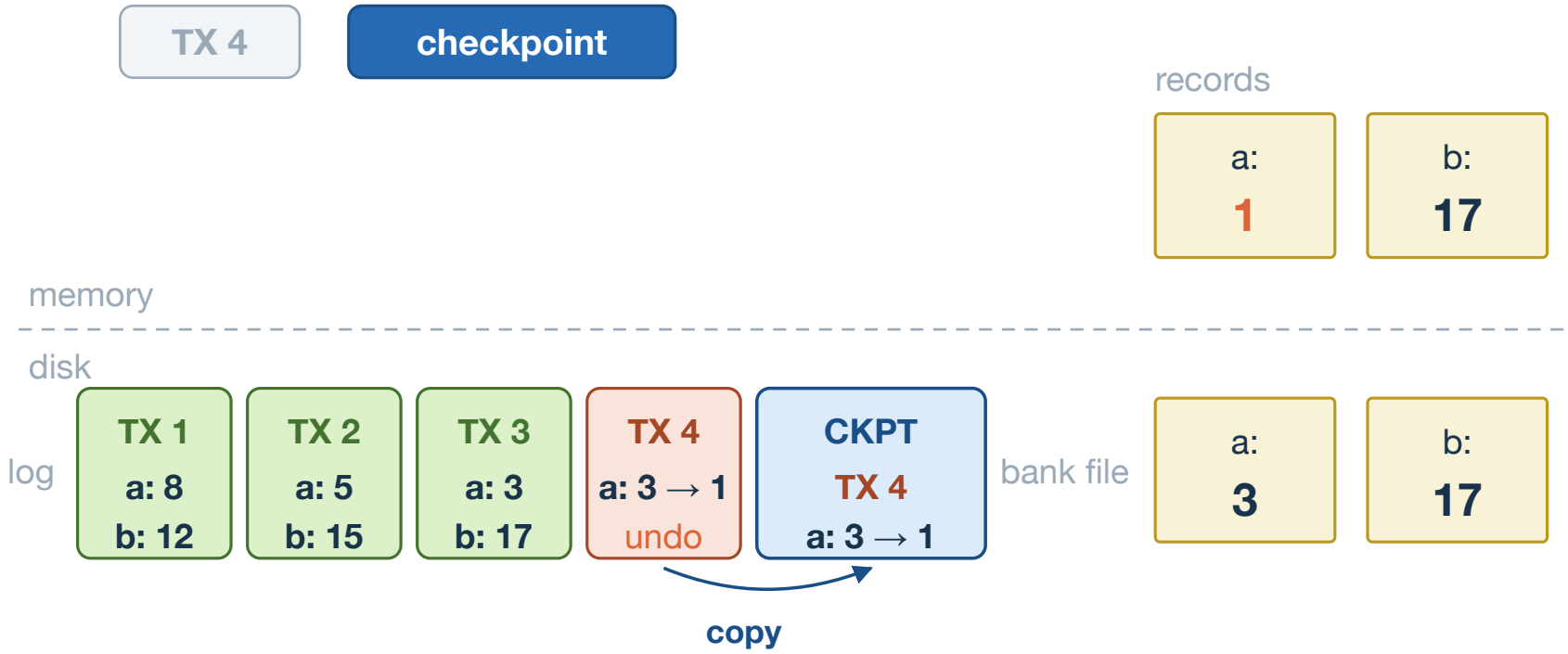
Solution: keep the in-progress TXs' log in a CKPT record

The logs are kept in a CKPT record to survive the log truncation during the checkpoint

Method: wait till no operation is in progress, to ensure the CKPT entry is complete

- An operation is short, so this wait always ends

Then we can do the checkpoint



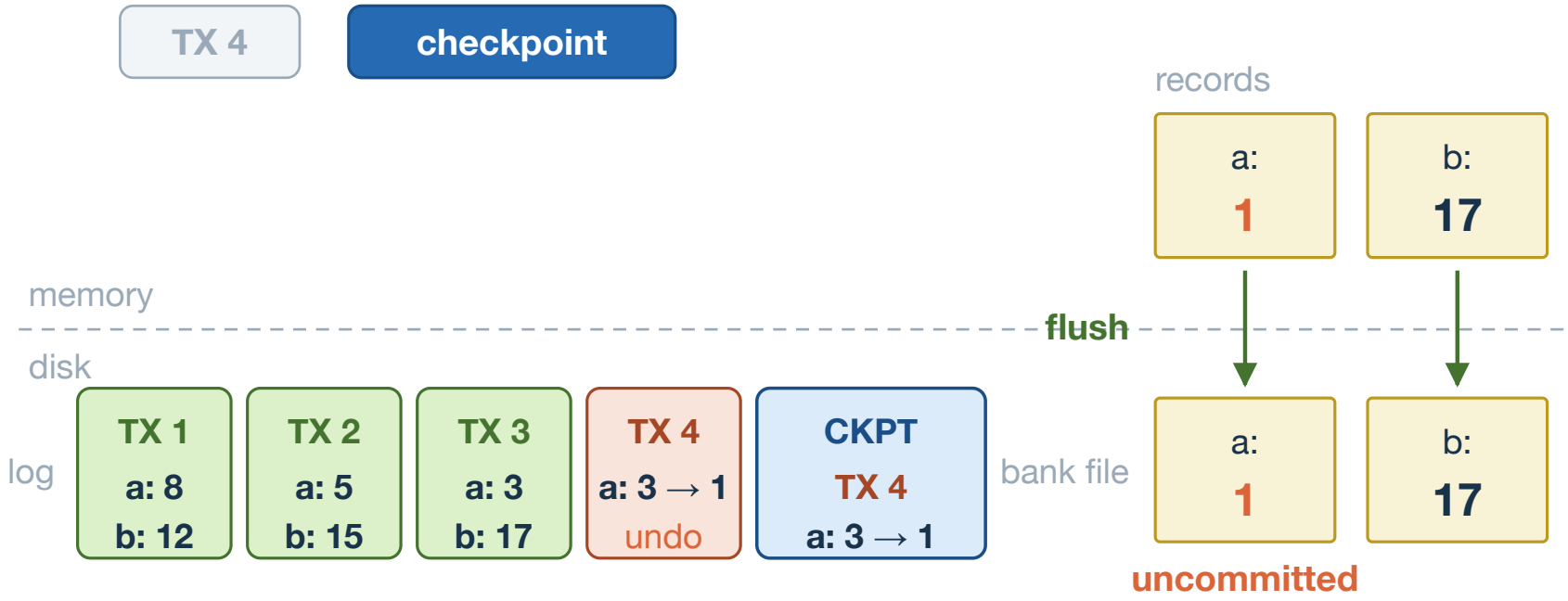
Solution: keep the in-progress TXs' log in a CKPT record

The logs are kept in a CKPT record to survive the log truncation during the checkpoint

Method: wait till no operation is in progress, to ensure the CKPT entry is complete

- An operation is short, so this wait always ends

Then we can do the checkpoint



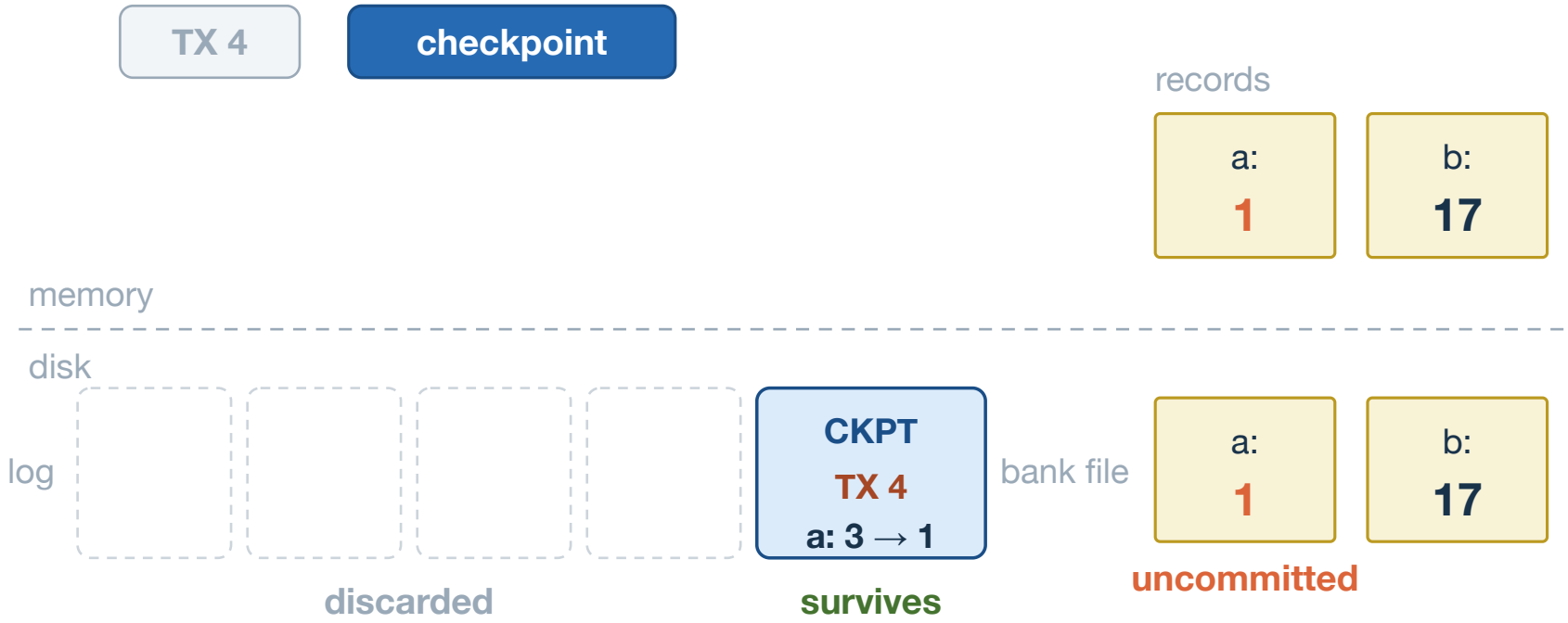
Solution: keep the in-progress TXs' log in a CKPT record

The logs are kept in a CKPT record to survive the log truncation during the checkpoint

Method: wait till no operation is in progress, to ensure the CKPT entry is complete

- An operation is short, so this wait always ends

Then we can do the checkpoint



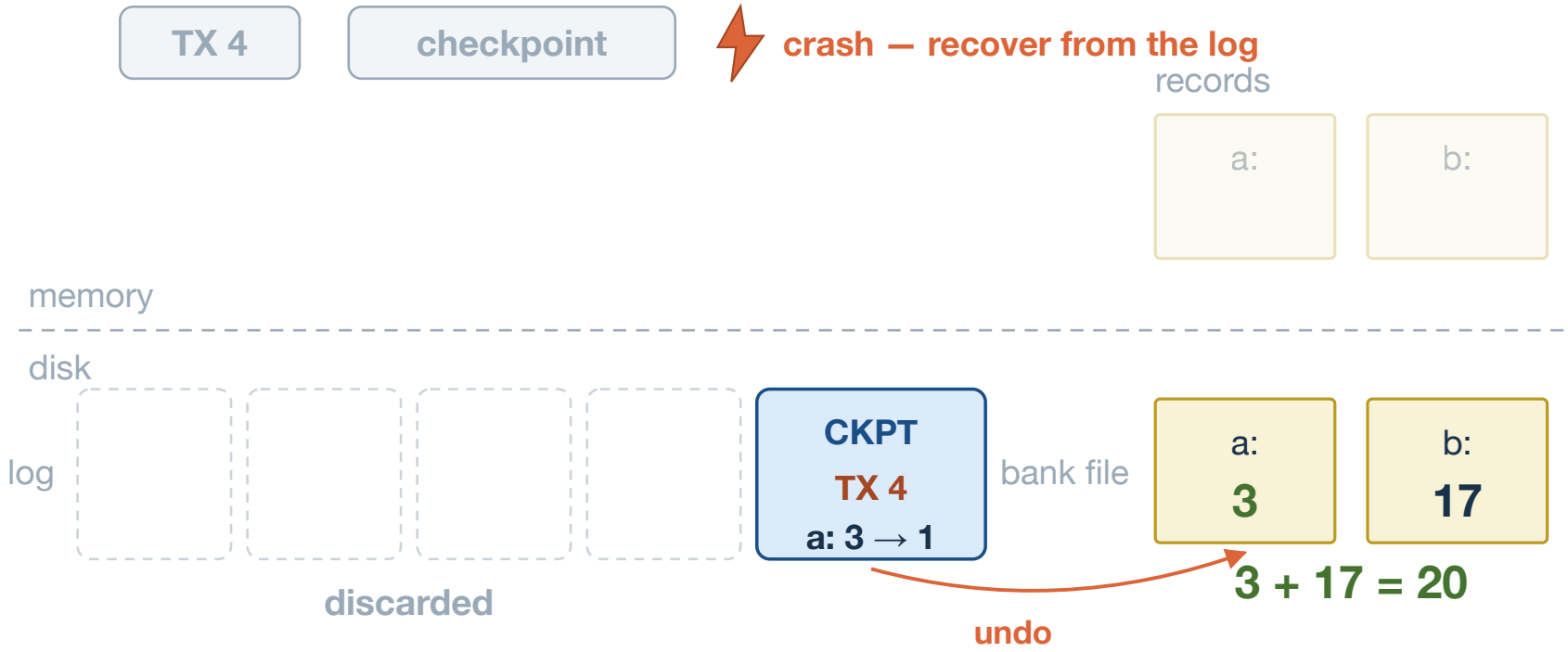
Solution: keep the in-progress TXs' log in a CKPT record

The logs are kept in a CKPT record to survive the log truncation during the checkpoint

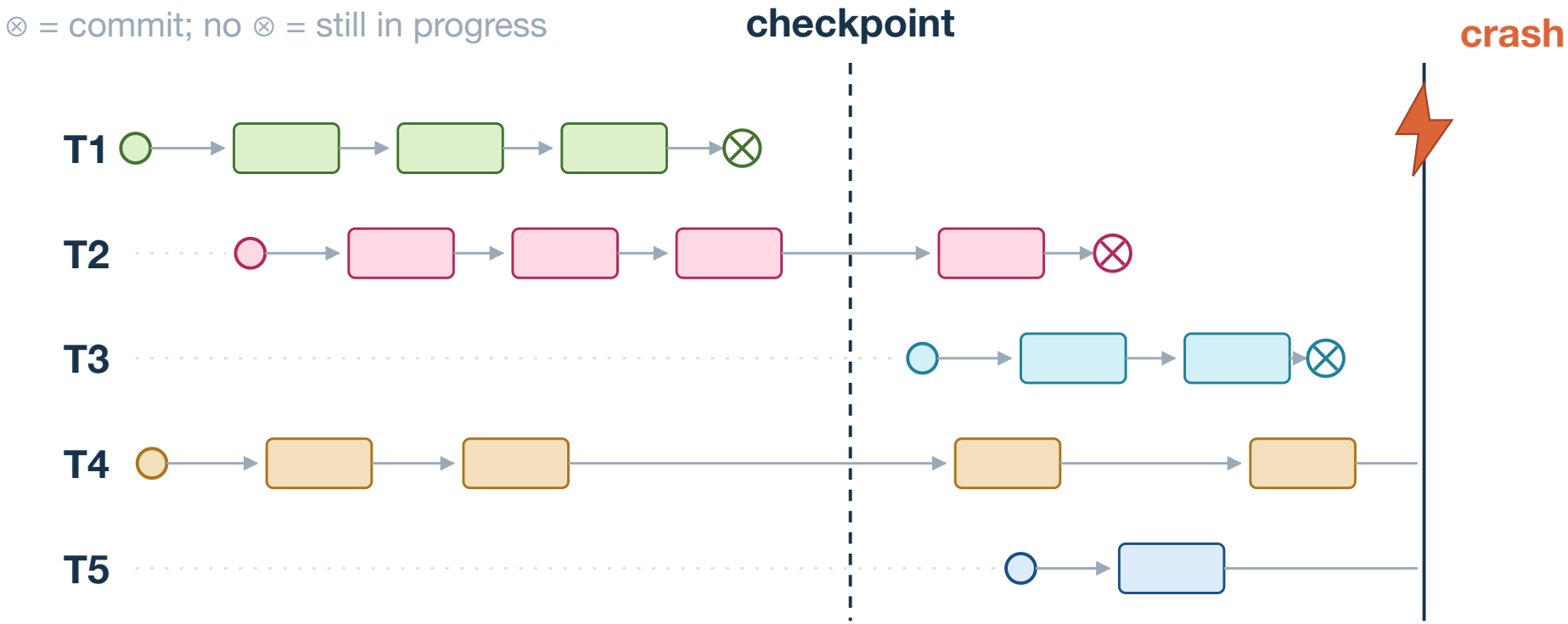
Method: wait till no operation is in progress, to ensure the CKPT entry is complete

- An operation is short, so this wait always ends

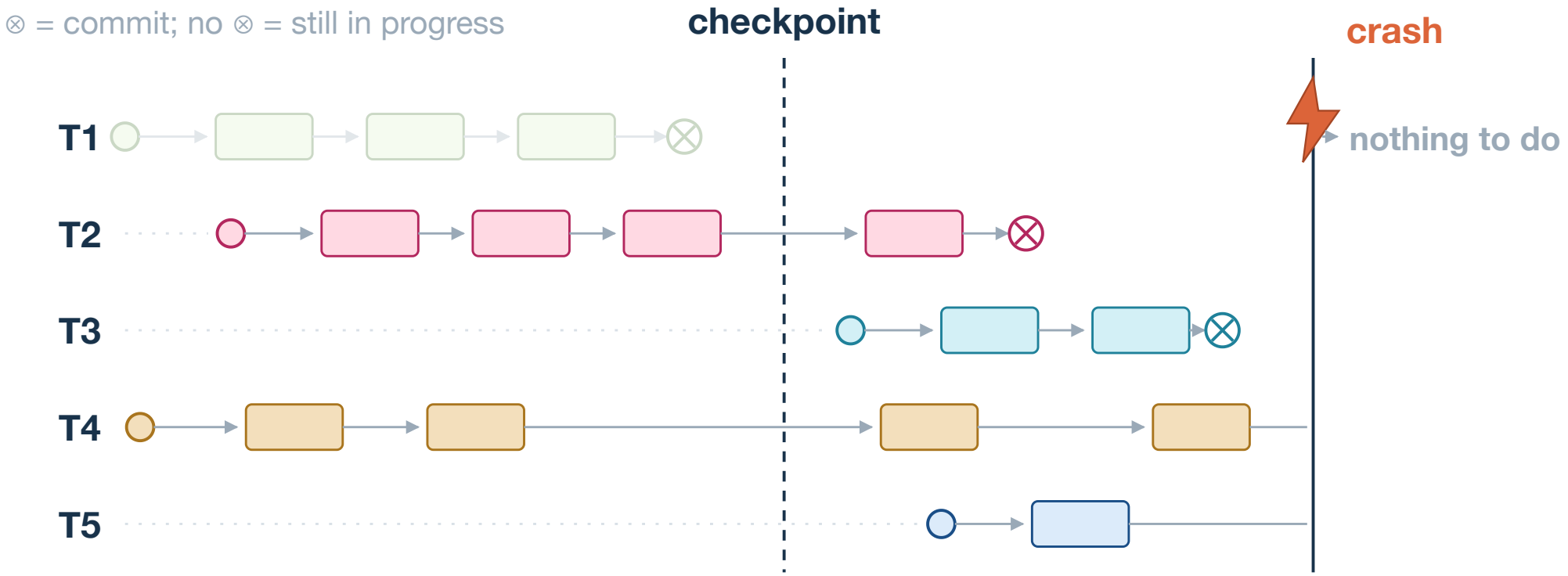
Then we can do the checkpoint



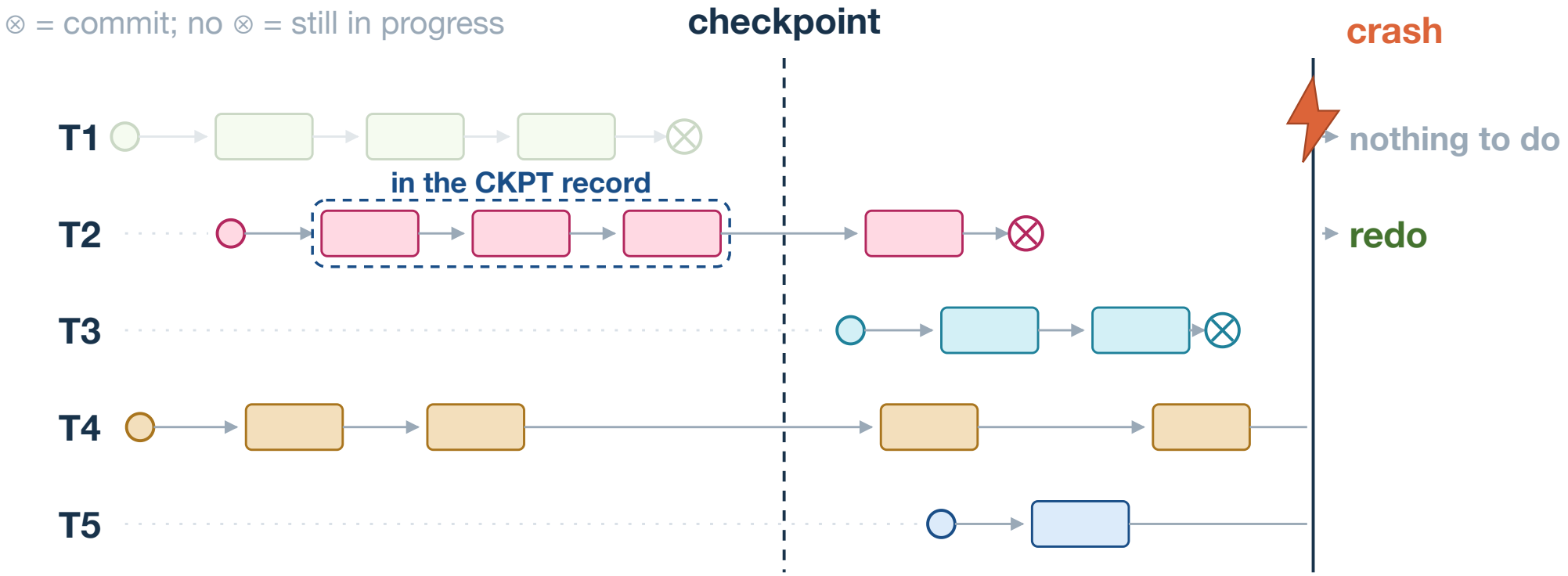
Recovery with a checkpoint



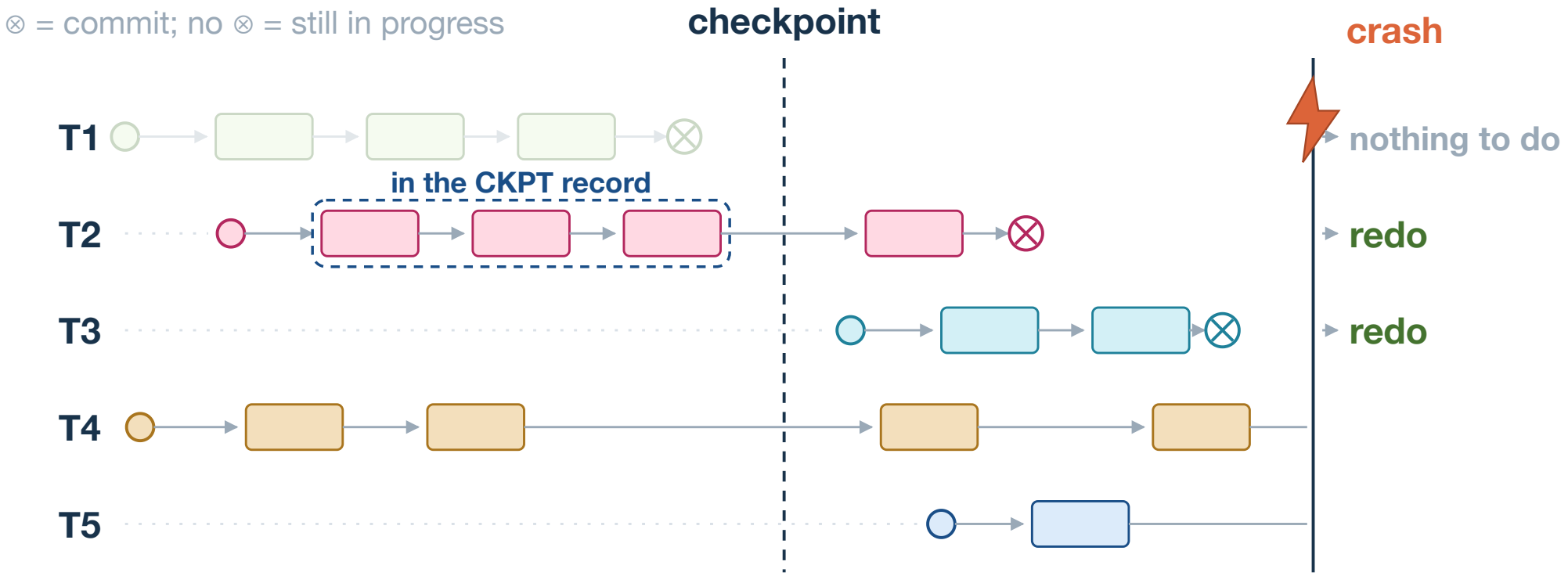
Recovery with a checkpoint



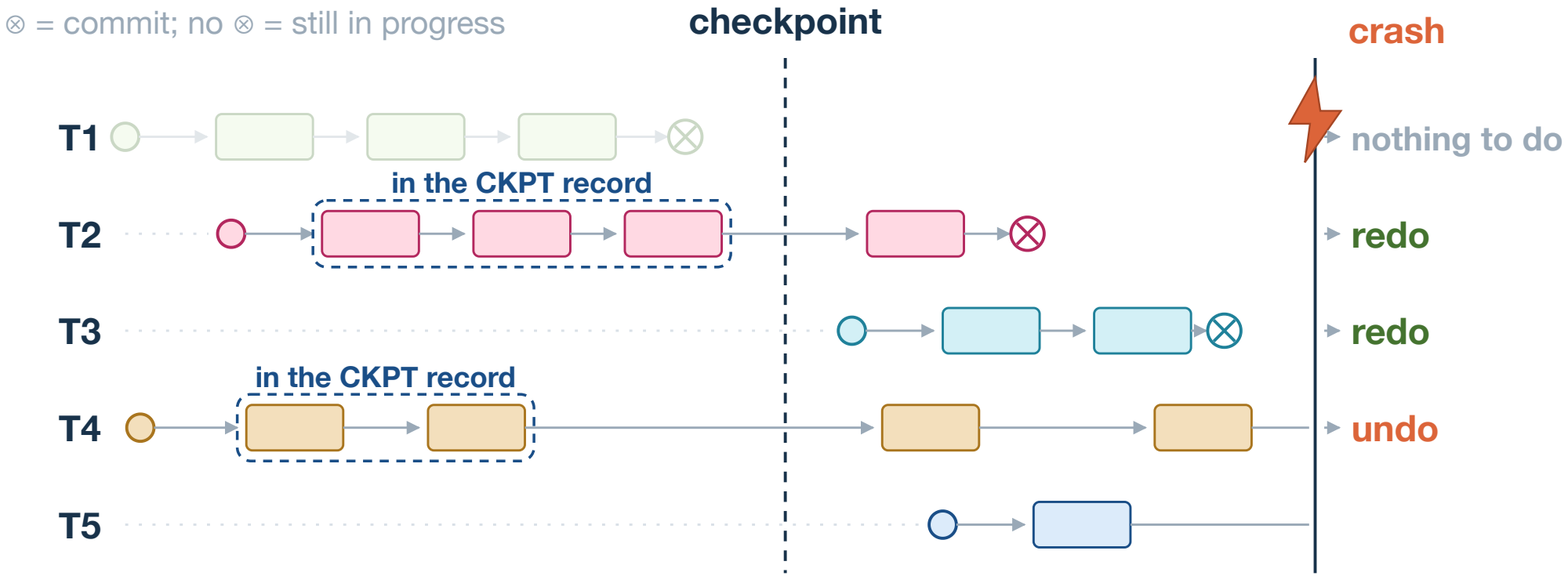
Recovery with a checkpoint



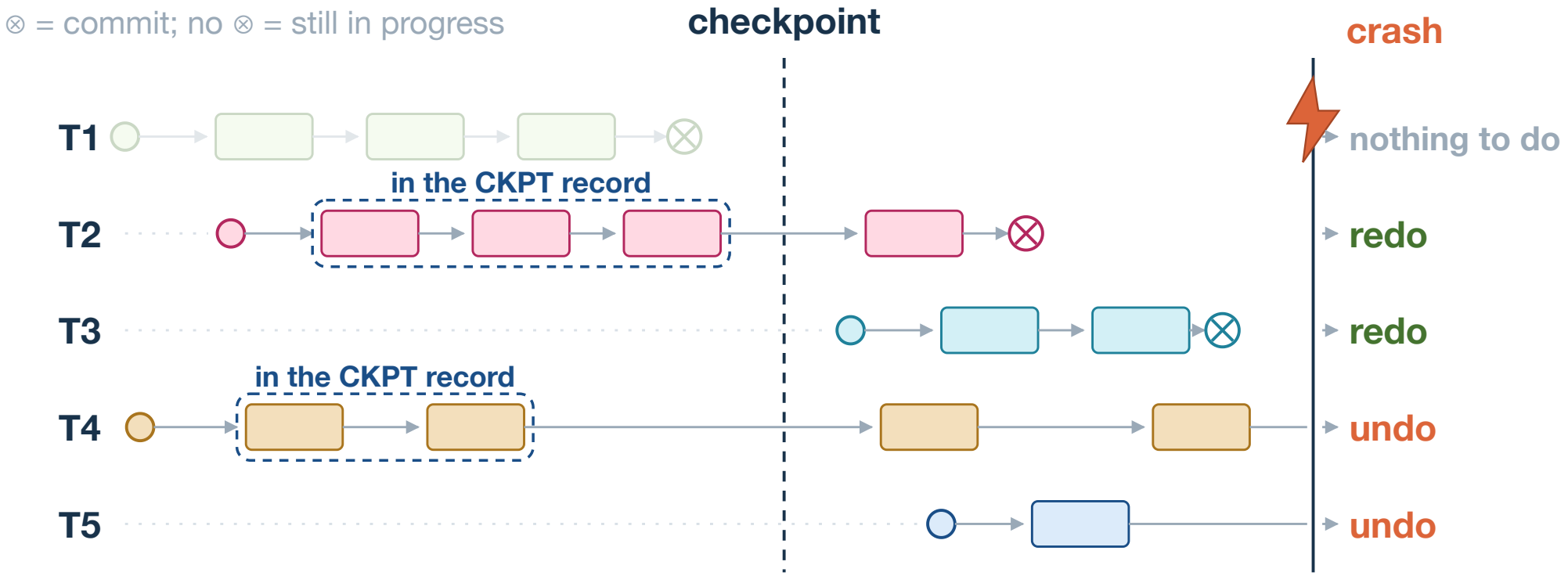
Recovery with a checkpoint



Recovery with a checkpoint



Recovery with a checkpoint



Question: what to do with T1–T5?

Undo-redo vs. redo-only: how to choose

It depends on the system goals, e.g., performance

Two questions

- Which one is faster during execution?
- Which one is faster during recovery?

Redo-only is generally faster

- Fewer disk operations: one transaction writes one entry
- Recovery needs only one scan of the log, no walking back and forth

Redo-only by default
undo-redo only for transactions too large for memory

(Aside) What about undo-only logging

Rules

- Append the undo record before flushing a state modification
- But: all modifications must be flushed before the transaction commits (so no redo needed)

Rarely used

- Much slower than undo-redo during execution: commit waits for the data to sync (random writes)
- Recovery is faster: only undo is needed, but that matters less than execution efficiency

Summary

What we have talked about today

- The new problem under failure: the disk is left in an intermediate state, possibly incorrect
- The guarantee we want: all-or-nothing atomicity — happens completely or not at all
- Two ways to build it: shadow copy and logging (journaling is a special case of logging)
- Logging needs checkpointing, or the log keeps growing and recovery keeps getting slower

This lecture assumes fail-stop failures, no partial failures

- Multiple machines and partial failures come in later lectures
- The log need not be a file: anything with atomic append that is itself fault tolerant will do, e.g., an in-memory log replicated on many servers

参考文献

[1] All File Systems Are Not Created Equal: On the Complexity of Crafting Crash-Consistent Applications — Pillai et al., 2014. OSDI (P5)

[2] Journaling the Linux ext2fs Filesystem — Stephen C. Tweedie, 1998. LinuxExpo (P21)

[3] Principles of Computer System Design: An Introduction — Saltzer & Kaashoek, 2009. Morgan Kaufmann (P29)

[4] ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks Using Write-Ahead Logging — Mohan et al., 1992. ACM TODS (P46)

[5] Transaction Processing: Concepts and Techniques — Gray & Reuter, 1993. Morgan Kaufmann (P49)